rent_prices_cleaning

March 4, 2026

Rent price prediction in Berlin

The goal of this project is to create the most suitable ML model to predict rent prices (Kaltmiete) in Berlin. Since Warmmiete (rent that includes other costs like electricity, heating, etc) isn't available in all listings, it was more suitable to use Kaltmiete instead. In this case, the best place to find the data for our dataset is real estate and apartment-search websites like Immowelt, WG-Gesucht, Immosuchmaschine, Kleineanzeigen, etc. For this project, I decided to use only Immowelt, since it included more data overall per listing, therefore more features can be included in the dataset. Also, it has a lot of new listings per day, so it is sufficient to build a proper dataset with enough samples if we gather data regularly.

Data Cleaning & Preparation

Part 1: Handling missing values

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

[2]:
```
df = pd.read_csv('immowelt_listings.csv')
df.head(5)
```

[2]:
```
                                        listing_url cold_rent  \
0  https://www.immowelt.de/expose/f6e9dd76-dbe1-4…   1,499 €
1  https://www.immowelt.de/expose/56cf72dd-00ea-4…   1,178 €
2  https://www.immowelt.de/expose/591c45d0-01dd-4…   5,130 €
3  https://www.immowelt.de/expose/8e225383-b57c-4…     360 €
4  https://www.immowelt.de/expose/03a3d0b6-9fab-4…   1,399 €

           warm_rent deposit     area     rooms        floor   free_from  \
0                NaN    4500  58,3 m²  2 Zimmer  5. Geschoss  15.02.2026
1  1.350 € Warmmiete    3534  45,7 m²  2 Zimmer  4. Geschoss  01.03.2026
2  6.130 € Warmmiete   15390   270 m²  3 Zimmer  3. Geschoss      sofort
3    475 € Warmmiete    1080    93 m²  1 Zimmer  2. Geschoss  29.12.2025
4  1.649 € Warmmiete     NaN  65,8 m²  2 Zimmer  1. Geschoss  01.02.2026

                                         location has_balkon  … badewanne  \
0  Friedenauer Höhe 23a, Friedenau, Berlin (12159)       True  …       NaN
1            Spreestraße, Köpenick, Berlin (12555)        NaN  …       NaN
```

```
2     Zeughofstrasse 20, Kreuzberg, Berlin (10799)        NaN  …       NaN
3          Bahnhofstraße, Köpenick, Berlin (12555)        NaN  …      True
4   Friedenauer Höhe 16, Friedenau, Berlin (12159)       True  …       NaN

  dusche     flooring_type energy_source     heating_type  \
0   True  fliesen, parkett     Fernwärme  Fußbodenheizung
1   True           parkett           NaN    Etagenheizung
2    NaN               NaN     Fernwärme   Zentralheizung
3   True        holzdielen           NaN              NaN
4   True               NaN     Fernwärme  Fußbodenheizung

             property_condition year_built schufa_check  \
0                     Erstbezug       2025         True
1                           NaN       2025         True
2  Altbau (bis 1945), neuwertig       1902         True
3                           NaN       2025         True
4                     neuwertig       2024         True

  number_of_images_posted        scraped_at
0                    12.0  12/28/2025 21:25
1                    12.0  12/28/2025 21:26
2                     9.0  12/28/2025 21:26
3                     9.0  12/28/2025 21:26
4                    20.0  12/28/2025 21:27

[5 rows x 27 columns]
```

[3]:
```
df.shape
```

[3]:
```
(22687, 27)
```

22687 samples and 27 features is a pretty good dataset

[4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22687 entries, 0 to 22686
Data columns (total 27 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   listing_url             22687 non-null  object
 1   cold_rent               22289 non-null  object
 2   warm_rent               12409 non-null  object
 3   deposit                 8311 non-null   object
 4   area                    21656 non-null  object
 5   rooms                   21679 non-null  object
 6   floor                   18815 non-null  object
 7   free_from               13097 non-null  object
```

```
 8   location                 21515 non-null  object
 9   has_balkon               10969 non-null  object
 10  has_terrasse             5991 non-null   object
 11  has_garten               2138 non-null   object
 12  elevator                 9914 non-null   object
 13  parking                  3460 non-null   object
 14  keller                   10178 non-null  object
 15  barrierefrei             2649 non-null   object
 16  einbaukueche             12632 non-null  object
 17  badewanne                5288 non-null   object
 18  dusche                   5612 non-null   object
 19  flooring_type            7795 non-null   object
 20  energy_source            9344 non-null   object
 21  heating_type             12388 non-null  object
 22  property_condition       9035 non-null   object
 23  year_built               11127 non-null  object
 24  schufa_check             22687 non-null  bool
 25  number_of_images_posted  14269 non-null  float64
 26  scraped_at               22687 non-null  object
dtypes: bool(1), float64(1), object(25)
memory usage: 4.5+ MB
```

[5]:
```
df.isnull().sum()
```

[5]:
```
listing_url                   0
cold_rent                   398
warm_rent                 10278
deposit                   14376
area                       1031
rooms                      1008
floor                      3872
free_from                  9590
location                   1172
has_balkon                11718
has_terrasse              16696
has_garten                20549
elevator                  12773
parking                   19227
keller                    12509
barrierefrei              20038
einbaukueche              10055
badewanne                 17399
dusche                    17075
flooring_type             14892
energy_source             13343
heating_type              10299
property_condition        13652
```

```
year_built                 11560
schufa_check                   0
number_of_images_posted     8418
scraped_at                     0
dtype: int64
```

Only 3 features don't have any missing values: 'listing_url' which sort of serves as an identifier, schufa_check which aws already taken care of by the scraper, it was set to fasle if it was not found and scrapet_at which is the time of scraping, a metadata provided by the scraper. We shouldn't be concerned with most of them having so many null values because they are boolean features and we can easily replace null with False.

[6]:
```
# We might start with cleaning by renaming the german features into english to␣
↪be consistent.
df = df.rename(columns={'keller': 'has_basement','barrierefrei':
↪'is_barrier-free','einbaukueche':'has_built-in_kitchen','badewanne':
↪'has_bathtub',
                        'dusche':'has_shower'})
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22687 entries, 0 to 22686
Data columns (total 27 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   listing_url              22687 non-null  object
 1   cold_rent                22289 non-null  object
 2   warm_rent                12409 non-null  object
 3   deposit                  8311 non-null   object
 4   area                     21656 non-null  object
 5   rooms                    21679 non-null  object
 6   floor                    18815 non-null  object
 7   free_from                13097 non-null  object
 8   location                 21515 non-null  object
 9   has_balkon               10969 non-null  object
 10  has_terrasse             5991 non-null   object
 11  has_garten               2138 non-null   object
 12  elevator                 9914 non-null   object
 13  parking                  3460 non-null   object
 14  has_basement             10178 non-null  object
 15  is_barrier-free          2649 non-null   object
 16  has_built-in_kitchen     12632 non-null  object
 17  has_bathtub              5288 non-null   object
 18  has_shower               5612 non-null   object
 19  flooring_type            7795 non-null   object
 20  energy_source            9344 non-null   object
 21  heating_type             12388 non-null  object
 22  property_condition       9035 non-null   object
```

```
 23  year_built                11127 non-null  object
 24  schufa_check              22687 non-null  bool
 25  number_of_images_posted   14269 non-null  float64
 26  scraped_at                22687 non-null  object
dtypes: bool(1), float64(1), object(25)
memory usage: 4.5+ MB
```

We will assume that all the listings have advertised everything in their listing. Meaning if they haven't mentioned having an elevator in the advertisiment then the scraper has nothing to scrape and has returned null. For that we assume that the apartment does not have an elevator and replace 'null' with 'false'. We do this for all boolean features.

[7]:
```
cols = [
    'has_balkon','has_terrasse','has_garten','elevator','parking',
    'has_basement','is_barrier-free','has_built-in_kitchen',
    'has_bathtub','has_shower'
]

df[cols] = df[cols].astype("boolean").fillna(False)
```

[8]:
```
df.isnull().sum()
```

[8]:
```
listing_url                   0
cold_rent                   398
warm_rent                 10278
deposit                   14376
area                       1031
rooms                      1008
floor                      3872
free_from                  9590
location                   1172
has_balkon                    0
has_terrasse                  0
has_garten                    0
elevator                      0
parking                       0
has_basement                  0
is_barrier-free               0
has_built-in_kitchen          0
has_bathtub                   0
has_shower                    0
flooring_type             14892
energy_source             13343
heating_type              10299
property_condition        13652
year_built                11560
schufa_check                  0
number_of_images_posted    8418
```

```
scraped_at                    0
dtype: int64
```

Now we are left with 14 features to clean. We will start with the features that are easy to fix like 'number_of_images_posted' and featrues that have very little missing areas like cold_rent. For the cold_rent which is the predicted feature, luckily we only have 324 missing values which is nothing compared to the number of samples our dataset has. We can either drop them, or use an unsupervised learning algorithm to estimate those values.

[9]:
```
# For that we have to check the samples with the missing values to see if they␣
↪are good samples or not.
df[df.cold_rent.isnull()].sample(15)
```

[9]:
```
                                          listing_url cold_rent warm_rent  \
22662  https://www.immowelt.de/expose/d8d336ff-79d1-4…       NaN       NaN
11529  https://www.immowelt.de/projekte/expose/k2mv83…       NaN       NaN
16610  https://www.immowelt.de/expose/ef47ce6c-b644-4…       NaN       NaN
15987  https://www.immowelt.de/expose/8f18b94e-f39e-4…       NaN       NaN
20066  https://www.immowelt.de/projekte/expose/k2n6a3…       NaN       NaN
22242  https://www.immowelt.de/projekte/expose/k2ug93…       NaN       NaN
11553  https://www.immowelt.de/expose/23n8s2j9w2lz?se…       NaN       NaN
22657  https://www.immowelt.de/expose/4113ce60-9512-4…       NaN       NaN
6230   https://www.immowelt.de/expose/674e72b2-51fa-4…       NaN       NaN
4561   https://www.immowelt.de/projekte/expose/k2mv83…       NaN       NaN
12626  https://www.immowelt.de/projekte/expose/k2e7a3…       NaN       NaN
19018  https://www.immowelt.de/projekte/expose/k2vj93…       NaN       NaN
39     https://www.immowelt.de/projekte/expose/k2mv83…       NaN       NaN
6666   https://www.immowelt.de/expose/cd4823aa-c95b-4…       NaN       NaN
19158  https://www.immowelt.de/expose/c7ee14eb-ef20-4…       NaN       NaN

        deposit                 area           rooms                    floor  \
22662       NaN                  NaN             NaN                      NaN
11529       NaN        60 bis 142 m²  2 bis 4 Zimmer  Bezugsbereit 01.03.2025
16610       NaN                  NaN             NaN                      NaN
15987       NaN                  NaN             NaN                      NaN
20066       NaN         28 bis 77 m²  1 bis 3 Zimmer                      NaN
22242       NaN        87 bis 111 m²  1 bis 2 Zimmer        Bezugsbereit 2024
11553       NaN                  NaN             NaN                      NaN
22657       NaN                  NaN             NaN                      NaN
6230        NaN                  NaN             NaN                      NaN
4561        NaN        60 bis 142 m²  2 bis 4 Zimmer  Bezugsbereit 01.03.2025
12626       NaN         55 bis 71 m²  1 bis 2 Zimmer   Bezugsbereit ab sofort
19018       NaN        37 bis 130 m²  1 bis 3 Zimmer        Bezugsbereit 2025
39          NaN        60 bis 142 m²  2 bis 4 Zimmer  Bezugsbereit 01.03.2025
6666   500000 €  1.500 m² Grundstück       24 Zimmer                      NaN
19158       NaN                72 m²        1 Zimmer              5. Geschoss

      free_from                                          location  \
```

```
22662       NaN                                                NaN
11529       NaN   Friedenauer Höhe 19A, Mariendorf, Berlin (12159)
16610       NaN                                                NaN
15987       NaN                                                NaN
20066       NaN           Buschkurgallee 64, Britz, Berlin (12359)
22242       NaN             Johannisplatz 3, Mitte, Berlin (10117)
11553       NaN                                                NaN
22657       NaN                                                NaN
6230        NaN                                                NaN
4561        NaN   Friedenauer Höhe 19A, Mariendorf, Berlin (12159)
12626       NaN     Meta-London-Straße 20, Spandau, Berlin (13587)
19018       NaN            Wöhlertstraße 21, Mitte, Berlin (10115)
39          NaN   Friedenauer Höhe 19A, Mariendorf, Berlin (12159)
6666        NaN  Forckenbeckstraße 55, Schmargendorf, Berlin (1…
19158    sofort                            Spandau, Berlin (13595)

       has_balkon  …  has_bathtub  has_shower  \
22662       False  …        False       False
11529        True  …        False       False
16610       False  …        False       False
15987       False  …        False       False
20066        True  …        False       False
22242       False  …        False       False
11553       False  …        False       False
22657       False  …        False       False
6230        False  …        False       False
4561         True  …        False       False
12626       False  …        False       False
19018       False  …        False       False
39           True  …        False       False
6666         True  …         True        True
19158       False  …        False        True

                                     flooring_type  energy_source  \
22662                                          NaN            NaN
11529                                          NaN      Fernwärme
16610                                          NaN            NaN
15987                                          NaN            NaN
20066                                          NaN            NaN
22242                                          NaN      Fernwärme
11553                                          NaN            NaN
22657                                          NaN            NaN
6230                                           NaN            NaN
4561                                           NaN      Fernwärme
12626                                          NaN            NaN
19018                                          NaN      Fernwärme
39                                             NaN      Fernwärme
```

```
6666   holzdielen, parkett, estrich, stein, teppich            Öl
19158                          holzdielen, laminat        Elektro

         heating_type  property_condition  year_built  schufa_check  \
22662             NaN                 NaN         NaN         False
11529             NaN                 NaN         NaN         False
16610             NaN                 NaN         NaN         False
15987             NaN                 NaN         NaN         False
20066             NaN                 NaN         NaN         False
22242  Zentralheizung                 NaN         NaN         False
11553             NaN                 NaN         NaN         False
22657             NaN                 NaN         NaN         False
6230              NaN                 NaN         NaN         False
4561              NaN                 NaN         NaN         False
12626             NaN                 NaN         NaN         False
19018             NaN                 NaN         NaN         False
39                NaN                 NaN         NaN         False
6666   Zentralheizung                 NaN        1923          True
19158             NaN                 NaN         NaN          True

       number_of_images_posted           scraped_at
22662                      NaN  2026-01-31 17:55:11
11529                     13.0  2026-01-19 15:48:05
16610                      NaN  2026-01-24 14:16:58
15987                      NaN  2026-01-24 12:07:47
20066                      NaN  2026-01-30 19:48:55
22242                      NaN  2026-01-31 16:47:16
11553                      NaN  2026-01-19 15:50:21
22657                      NaN  2026-01-31 17:54:52
6230                       NaN  2026-01-14 22:12:36
4561                      13.0  2026-01-11 13:56:24
12626                     13.0  2026-01-22 11:07:10
19018                      NaN  2026-01-30 16:18:38
39                        13.0  2025-12-28 21:43:30
6666                       6.0  2026-01-15 21:53:37
19158                      NaN  2026-01-30 16:50:41

[15 rows x 27 columns]
```

[10]:
```python
with pd.option_context('display.max_colwidth', None):
    print(df[df.cold_rent.isnull()].sample(5)['listing_url'])
```

```
51                                                                https://www
.immowelt.de/projekte/expose/k2vj932?serp_view=list&search=distributionTypes%3DR
ent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D2#ln=cla
ssified_search_results&m=classified_search_results_classified_classified_detail_
XL
6247                                          https://www.immowelt.de/expose/c
```

```
0859acc-77af-4e62-b47e-
efec51bc4387?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D13#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_XL
20076                        https://www.immowelt.de/projekte/expose/k2n6a32?ser
p_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26
locations%3DAD08DE8634%26page%3D38&sort_order_ab_test=sort_order_reference#ln=cl
assified_search_results&m=classified_search_results_classified_classified_detail
_M
22658    https://www.immowelt.de/expose/4716aea8-c84b-4250-82ab-dfa7ed38146e?ser
p_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26
locations%3DAD08DE8634%26page%3D74&sort_order_ab_test=sort_order_reference#ln=cl
assified_search_results&m=classified_search_results_classified_classified_detail
_M
15477                                                                  https://ww
w.immowelt.de/projekte/expose/k2vj932?serp_view=list&search=distributionTypes%3D
Rent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26page%3D8#ln=cl
assified_search_results&m=classified_search_results_classified_classified_detail
_L
Name: listing_url, dtype: object
```

By checking the urls we notice that these links redirect to listings that don't belong to a specific apartment but are rather from agencies that advertise multiple other listings that we already scrapped, meaning these samples are invalid and should be dropped.

[11]:
```
old_rows = df.shape[0]
df = df.dropna(subset=["cold_rent"])
new_rows = df.shape[0]
print(f"Rows removed: {old_rows - new_rows}")
```

```
Rows removed: 398
```

Now we decide how to handle samples with missing values in 'area' and 'rooms', which occur in nearly equal numbers. Unfortunately the samples that we dropped mostly had both these features, so probably we still have around 900 rows left. Before we decide on what to do with the samples we have to first inspect them.

[12]:
```
df[df.area.isnull()].sample(5)
```

[12]:
```
                                          listing_url cold_rent  \
17808  https://www.immowelt.de/expose/26jdurflzj91?se…    2660 €
17700  https://www.immowelt.de/expose/1f3d4869-5911-4…    1700 €
8378   https://www.immowelt.de/expose/14fb6a83-2f54-4…    5529 €
11021  https://www.immowelt.de/expose/083c5b83-f2bb-4…     997 €
12565  https://www.immowelt.de/expose/258fgi1mmldc?se…    1255 €

               warm_rent deposit area     rooms floor free_from  \
17808  2.820 € Warmmiete     NaN  NaN       NaN   NaN       NaN
17700                NaN     NaN  NaN       NaN   NaN       NaN
```

```
8378                 NaN     NaN  NaN  3 Zimmer   NaN    sofort
11021                NaN     NaN  NaN       NaN   NaN       NaN
12565  1.488 € Warmmiete     NaN  NaN       NaN   NaN       NaN

                                                 location  has_balkon  …  \
17808                                                 NaN       False  …
17700                                                 NaN       False  …
8378   Reichenhaller Straße 27, Schmargendorf, Berlin…        True  …
11021                                                 NaN       False  …
12565                                                 NaN       False  …

       has_bathtub  has_shower  flooring_type  energy_source  heating_type  \
17808        False       False            NaN            NaN           NaN
17700        False       False            NaN            NaN           NaN
8378         False        True        parkett            NaN           NaN
11021        False       False            NaN            NaN           NaN
12565        False       False            NaN            NaN           NaN

        property_condition  year_built  schufa_check  number_of_images_posted  \
17808                  NaN         NaN          True                      NaN
17700                  NaN         NaN          True                      NaN
8378   renoviert / saniert         NaN          True                     18.0
11021                  NaN         NaN          True                      9.0
12565                  NaN         NaN          True                     13.0

                scraped_at
17808  2026-01-26 13:48:18
17700  2026-01-26 13:31:52
8378   2026-01-16 21:04:27
11021  2026-01-19 14:17:53
12565  2026-01-22 10:57:58

[5 rows x 27 columns]
```

After sampling several times different samples, these samples had mostly no features other than the prices so they don't serve our model in anyway, therefore we can safely drop them.

[13]:
```
df = df.dropna(subset=["area"])
```

[14]:
```
int(df.rooms.isnull().sum())
```

[14]: 161

[15]:
```
# Luckily, by dropping the samples with null area, the number of samples will␣
↪null rooms dropped drastically
df[df.rooms.isnull()].sample(5)
```

[15]:

```
                                          listing_url cold_rent  \
4570   https://www.immowelt.de/expose/33a8e5f4-8212-4…     610 €
4550   https://www.immowelt.de/expose/f34cdc34-e4d2-4…    1035 €
19940  https://www.immowelt.de/expose/f347b5a8-da49-4…     650 €
19177  https://www.immowelt.de/expose/36388fb6-10bc-4…     873 €
20394  https://www.immowelt.de/expose/dd3c1569-8b68-4…     676 €

                            warm_rent deposit     area rooms        floor  \
4570            610 €/Monat Kaltmiete  1260 €    30 m²   NaN  5. Geschoss
4550                1.335 € Warmmiete  1335 €    51 m²   NaN          NaN
19940           650 €/Monat Kaltmiete  1500 €    16 m²   NaN  1. Geschoss
19177           873 €/Monat Warmmiete  2146 €  28,8 m²   NaN  1. Geschoss
20394  676 €/Monat Kaltmiete zzgl. NK  2028 €    30 m²   NaN  2. Geschoss

        free_from                                          location  \
4570   01.02.2026                      Schmargendorf, Berlin (14199)
4550       sofort    Koloniestraße 38, Gesundbrunnen, Berlin (13359)
19940  16.02.2026                           Steglitz, Berlin (12167)
19177      sofort          Pankstr 89, Gesundbrunnen, Berlin (13357)
20394      sofort  Grünauer Str. 123, Köpenick, Berlin-Köpenick (…

       has_balkon  …  has_bathtub  has_shower  flooring_type  energy_source  \
4570         True  …         True       False            NaN      Fernwärme
4550        False  …        False        True            NaN            NaN
19940       False  …        False        True            NaN            NaN
19177        True  …         True       False            NaN      Fernwärme
20394        True  …         True       False            NaN            Gas

          heating_type  property_condition  year_built  schufa_check  \
4570               NaN            Gepflegt        1930          True
4550    Zentralheizung                 NaN         NaN          True
19940              NaN                 NaN        1985          True
19177  Fußbodenheizung           Erstbezug        2025          True
20394   Zentralheizung                 NaN        1997          True

       number_of_images_posted           scraped_at
4570                      12.0  2026-01-11 13:57:54
4550                      11.0  2026-01-11 13:54:30
19940                      NaN  2026-01-30 19:37:50
19177                      NaN  2026-01-30 16:55:13
20394                      NaN  2026-01-30 20:16:05

[5 rows x 27 columns]
```

Interestingly this time the samples seems very legit, with most of them having all the features, even features like warm rent and deposit which are missing in many samples, therefore I don't want to drop them but rather find a reason why these values are missing in order to replace them and keep these samples. That's why I will inspect some listings first by sampling their urls.

[16]:
```
with pd.option_context('display.max_colwidth', None):
    print(df[df.rooms.isnull()].sample(5)['listing_url'])
```

```
21252    https://www.immowelt.de/expose/1eda4792-d7f8-4c41-8bb4-
68b0264f2685?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D7&sort_order_ab_test=sort_order_
reference#ln=classified_search_results&m=classified_search_results_classified_cl
assified_detail_XL
3670             https://www.immowelt.de/expose/ea186bf6-7c4e-4b09-9646-
0a68d796e84e?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D25&feature_priceDropVariantB=tru
e#ln=classified_search_results&m=classified_search_results_classified_classified
_detail_XL
18276                                            https://www.immowelt.de/expose/f
347b5a8-da49-45dc-8e88-
301fbab59363?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D29#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_XL
14832                                            https://www.immowelt.de/expose/a
978c403-d079-451f-b28f-
23ba0da0e7bd?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D91#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_XL
3243             https://www.immowelt.de/expose/dd3c1569-8b68-4194-9533-
837d2664438a?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D38&feature_priceDropVariantB=tru
e#ln=classified_search_results&m=classified_search_results_classified_classified
_detail_XL
Name: listing_url, dtype: object
```

After inspecting several posts, I noticed that all the samples were valid and that these samples, with missing number of rooms were either WG apartments or simply it just happend to miss that value. Now we can either drop the values or come up with a way to estimate them. Now to estimated, we can use features that impact the number of rooms like area and number of images. We cannot use the price, that would cause data leakage.

[17]:
```
# Is it really worth it to do imputations? Let's check what part of the data␣
↪set that would be (in percentage):
float(df.rooms.isnull().mean())*100
```

[17]: 0.751669078855222

[18]:
```
# 0.74%, it is basically irrelevant, we don't lose anything by dropping them.
# Doing imputations with these would add unnecessary noise
df = df.dropna(subset=["rooms"])
```

[19]:
```
# Let's recheck the remaning features with null values
df.isnull().sum()[df.isnull().sum() > 0]
```

[19]:
```
warm_rent                   9508
deposit                    13102
floor                       2753
free_from                   8530
location                     326
flooring_type              13698
energy_source              12139
heating_type                9063
property_condition         12411
year_built                 10233
number_of_images_posted     7794
dtype: int64
```

Let's check the location since it has the least null values. Also, if you notice the samples who don't have location has also dropped nearly the same as the number of samples that we dropped.

[20]:
```
df[df.location.isnull()].sample(5)
```

[20]:
```
                                          listing_url  cold_rent  \
12721  https://www.immowelt.de/expose/e4035a0f-c11d-4…     1449 €
12375  https://www.immowelt.de/expose/b3004878-930b-4…      850 €
17572  https://www.immowelt.de/expose/2106d3a4-2170-4…     2099 €
5986   https://www.immowelt.de/expose/d047c4df-f426-4…     2050 €
17936  https://www.immowelt.de/expose/ae6318b8-66e8-4…  1973.32 €

               warm_rent deposit      area     rooms        floor   free_from  \
12721                NaN     NaN     75 m²  3 Zimmer  4. Geschoss         NaN
12375  1.020 € Warmmiete     NaN   28,9 m²  1 Zimmer  1. Geschoss  01.03.2026
17572  2.433 € Warmmiete     NaN   83,3 m²  4 Zimmer  1. Geschoss  01.03.2026
5986   2.278 € Warmmiete     NaN   84,4 m²  4 Zimmer           EG  15.01.2026
17936  2.468 € Warmmiete     NaN  117,3 m²  4 Zimmer  4. Geschoss         NaN

      location  has_balkon  …  has_bathtub  has_shower  flooring_type  \
12721      NaN       False  …        False       False            NaN
12375      NaN       False  …        False       False            NaN
17572      NaN       False  …        False       False            NaN
5986       NaN       False  …        False       False            NaN
17936      NaN       False  …        False       False            NaN

       energy_source  heating_type  property_condition  year_built  \
12721            NaN           NaN                 NaN         NaN
12375            NaN           NaN                 NaN         NaN
17572            NaN           NaN                 NaN         NaN
5986             NaN           NaN                 NaN         NaN
17936            NaN           NaN                 NaN         NaN

       schufa_check  number_of_images_posted           scraped_at
12721          True                     10.0  2026-01-22 11:20:58
```

```
12375          True                      12.0  2026-01-22 10:28:30
17572          True                       NaN  2026-01-26 13:06:33
5986           True                       8.0  2026-01-14 21:29:13
17936          True                       NaN  2026-01-26 14:07:00

[5 rows x 27 columns]
```

All the samples seem good, so there is no good reason to just drop them. Despite location being a very difficult feature to deal with, it is indeed a very powerful feature for our data set. It is one of the main features to determine the rent of the apartment. I suggest, since it is a categorical feature we replace the missing values with 'Unknown'. By doing so,even if we derive simple features from location like post number, street name and name of area, they are still gonna be categorical. Deriving a continuous feature like 'distance from city-center' or similar things is unlikely because we don't have the precise location. The model may find it very useful patterns with these 'Unknown' values because for example they might be missing the owner might not have put it for being a bad location and fearing it would lower the value of his property. We also have to consider the risk of this apartments not being in Berlin at all, without the location we cannot ensure rather that is true or false. These apartments are most likely going to by in Berlin since that's where we scraped the data. But I want to ensure first by sampling and checking some listings in order to find more information.

[21]:
```python
with pd.option_context('display.max_colwidth', None):
    print(df[df.location.isnull()].sample(5)['listing_url'])
```

```
12303    https://www.immowelt.de/expose/1a72185e-bbf8-4185-8c2d-
9e324f601d20?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D26#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_XL
17744     https://www.immowelt.de/expose/64c42e2f-96fc-4e05-994e-
259f2829eaff?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D28#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
17734     https://www.immowelt.de/expose/9d337be0-e38a-4be1-96c3-
903a47f661ef?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D27#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
17763     https://www.immowelt.de/expose/685538cf-8594-4343-b3a3-
c1e2153ecacc?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D29#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
17631    https://www.immowelt.de/expose/600371a0-1c3f-4f84-8cbd-
7e147649e9d8?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D11#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_XL
Name: listing_url, dtype: object
```

By inspecting some samples, I noticed that the location was in Berlin. It was present in all the postings but the reason why it is missing in our data set was either because they didn't put the feature but rather put it in the description where our scraper can't get it, or our scraper simply

didn't scrape it even though it was present. Apperantly in some rare cases (less than 0.1%) the scraper failed to scrape it even though it was programmed to do it. Now the complexity of the decision-making increases, because the data wasn't missing for the reason I thought. Even further, the locations were in avarage-priced locations, so not the worst places. If we replace the missing values with 'Unknown', we would encode scraper failure as a real-world category, which is wrong. If we drop the values we lose almost 2% of our data set and they are all meaningful data. We postpone the decision on how to handle these samples until the remaining preprocessing steps are completed, because the number of these samples may drop near 0.

[22]:
```
df[df.number_of_images_posted.isnull()].sample(5)
```

[22]:
```
                                          listing_url  cold_rent  \
22294  https://www.immowelt.de/expose/1b977aec-447a-4…      560 €
16014  https://www.immowelt.de/expose/a1877ff0-3acc-4…     1170 €
18467  https://www.immowelt.de/expose/d8cf58ed-2c30-4…      786 €
20433  https://www.immowelt.de/expose/c9cbf60f-f4b0-4…     4487 €
18251  https://www.immowelt.de/expose/a4c7668f-f51a-4…  1328.13 €

               warm_rent    deposit     area     rooms        floor free_from  \
22294                NaN        NaN    38 m²  1 Zimmer          NaN       NaN
16014  1.570 € Warmmiete     1570 €    65 m²  2 Zimmer  5 Geschosse    sofort
18467                NaN        NaN    87 m²  3 Zimmer  4. Geschoss       NaN
20433                NaN        NaN    68 m²  3 Zimmer          NaN    sofort
18251  1.746 € Warmmiete  3984.39 €  98,4 m²  3 Zimmer           EG    sofort

                                       location  has_balkon  …  \
22294                      Mitte, Berlin (10555)       False  …
16014                    Neukölln, Berlin (12051)        True  …
18467                   Kreuzberg, Berlin (10999)       False  …
20433  Friedrichstraße 135A, Mitte, Berlin (10117)        True  …
18251  Pepitapromenade 11, Spandau, Berlin (13587)        True  …

       has_bathtub  has_shower  flooring_type  energy_source    heating_type  \
22294        False       False            NaN            NaN             NaN
16014         True       False            NaN            NaN  Zentralheizung
18467         True       False            NaN            NaN             NaN
20433        False        True        parkett            NaN             NaN
18251         True       False            NaN      Fernwärme  Zentralheizung

       property_condition  year_built  schufa_check  number_of_images_posted  \
22294                 NaN         NaN          True                      NaN
16014           neuwertig         NaN          True                      NaN
18467                 NaN         NaN          True                      NaN
20433                 NaN         NaN          True                      NaN
18251           neuwertig        2018          True                      NaN

                scraped_at
```

```
22294  2026-01-31 16:57:29
16014  2026-01-24 12:13:01
18467  2026-01-27 20:23:58
20433  2026-01-30 20:19:24
18251  2026-01-27 19:31:37

[5 rows x 27 columns]
```

[23]:
```python
with pd.option_context('display.max_colwidth', None):
    print(df[df.number_of_images_posted.isnull()].sample(5)['listing_url'])
```

```
16424                                                https://www.immowelt.de/expose/
17c70de7-57d0-4e75-af4b-
711689447fe7?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D60#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
19849                              https://www.immowelt.de/expose/26q9ve97g9ea?ser
p_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26
locations%3DAD08DE8634%26page%3D30&sort_order_ab_test=sort_order_reference#ln=cl
assified_search_results&m=classified_search_results_classified_classified_detail
_M
14864                                                https://www.immowelt.de/expose/
2375e466-113d-4430-ae90-
c60a64a8f4fc?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D94#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
20055    https://www.immowelt.de/expose/76a0b61c-1e61-480c-b5e9-
fd3a9c2ca296?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D37&sort_order_ab_test=sort_order
_reference#ln=classified_search_results&m=classified_search_results_classified_c
lassified_detail_L
22202    https://www.immowelt.de/expose/73c7729c-460c-4bef-8595-
d282c47015ce?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D58&sort_order_ab_test=sort_order
_reference#ln=classified_search_results&m=classified_search_results_classified_c
lassified_detail_M
Name: listing_url, dtype: object
```

Before inspecting the samples with missing number of values, I was intuitively considering replacing NaN values with 0 thinking that the reason why they were NaN was because there were no images to scrape and therefore it didn't scrape anything. But almost all of the listings had images and for some reason it was just the scraper failing to scrape the text. Caution: The scraper doesn't scrape the images themself but rather the text saying 'there are x images' and from there it filters the number, but when there was no image, there was no text in the images and therefore no images.

[24]:
```python
# Before we consider further, let's check what values we have:
print(np.sort(df["number_of_images_posted"].unique()))
```

```
[ 3.  4.  5.  6.  7.  8.  9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20.
```

```
 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38.
 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 50. 51. 52. 53. 55. 61. 67. 71.
 nan]
```

We have values like 1 and 2 missing, which is odd, especially considering the fact that in the samples, I just observed, there were many posting with 1 or 2 images. In this case dropping the rows in not an option, considering the fact that number_of_images has little to no impact on the price. Rather it could be more useful to provide information about other features, in case we need to do any imputations. In our data set we have many features and keeping a feature with 20% missing values and no way of imputing them, so it wouldbe a great idea to just drop it

[25]:
```
df = df.drop(columns=["number_of_images_posted"])
```

[26]:
```
# floor => replace with unknown
df["floor"].unique()
# free-from => transform the imediately, later, unknown
```

[26]:
```
array(['5. Geschoss', '4. Geschoss', '3. Geschoss', '2. Geschoss',
       '1. Geschoss', '7. Geschoss', nan, 'EG', '11. Geschoss',
       '10. Geschoss', 'Geschoss 6/7', 'Geschoss -2/4', '6. Geschoss',
       'Geschoss 3/5', 'Geschoss 4/5', '4 Geschosse', '9. Geschoss',
       '8. Geschoss', '13. Geschoss', '17. Geschoss', 'Geschoss EG/3',
       '14. Geschoss', 'Geschoss -4/5', '15. Geschoss', 'Geschoss 4/6',
       'Geschoss 2/3', 'Geschoss 3/4', '12. Geschoss', 'Geschoss 15/21',
       'Geschoss 1/2', '20. Geschoss', 'Geschoss 2/4', 'Geschoss 2/2',
       'Geschoss EG/2', 'Geschoss EG/5', 'Geschoss 1/5', 'Geschoss EG/1',
       '5 Geschosse', 'Geschoss EG/6', '18. Geschoss', 'Geschoss 5/6',
       'Geschoss 2/5', '2 Geschosse', 'Geschoss 5/8', 'Geschoss -1/3',
       'Geschoss 1/8', '6 Geschosse', 'Geschoss 3/6', 'Geschoss 1/4',
       'Geschoss 2/6', 'Geschoss 3/3', 'Geschoss 1/6', '3 Geschosse',
       'Geschoss 4/7', 'Geschoss 1/3', 'Geschoss 3/9', 'Geschoss 6/6',
       'Geschoss 5/5', 'Geschoss -5/5', 'Geschoss 1/13', 'Geschoss EG/4',
       'Geschoss 5/7', '9 Geschosse', 'Geschoss 4/4', 'Geschoss -5/4',
       '7 Geschosse', 'Geschoss -1/5', 'Geschoss 1/7', 'Geschoss -2/3',
       'Geschoss 3/12', 'Geschoss 7/8', '16. Geschoss', 'Geschoss 2/10',
       'Geschoss EG/11', 'Geschoss 3/10', '21. Geschoss', 'Geschoss 7/7',
       'Geschoss 8/9', 'Geschoss 2/8', 'Geschoss 1/1', 'Geschoss -1/4',
       'Geschoss 9/11', 'Geschoss 10/20', 'Geschoss EG/8',
       'Geschoss EG/7', 'Geschoss 4/12', 'Geschoss 8/10'], dtype=object)
```

[27]:
```
with pd.option_context('display.max_colwidth', None):
    print(df[df.floor.isnull()].sample(5)['listing_url'])
```

```
1903     https://www.immowelt.de/expose/f8b718b0-4f98-4f61-a098-
a728b1c21304?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D84&feature_priceDropVariantB=tru
e#ln=classified_search_results&m=classified_search_results_classified_classified
_detail_L
2456                              https://www.immowelt.de/expose/25gbrw23k59k?ser
```

```
p_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26
locations%3DAD08DE8634%26page%3D65&feature_priceDropVariantB=true#ln=classified_
search_results&m=classified_search_results_classified_classified_detail_M
7907
https://www.immowelt.de/expose/7fafb45e-9433-4bce-bcd4-
95d9435e0146?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D24#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
3100    https://www.immowelt.de/expose/62a61c65-a7f6-4250-939e-
9c452e291d5a?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D42&feature_priceDropVariantB=tru
e#ln=classified_search_results&m=classified_search_results_classified_classified
_detail_XL
587                                     https://www.immowelt.de/expose/bd0cd23c-
60fc-4b95-b655-
a60145c1dc27?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D155#ln=classified_search_results
&m=classified_search_results_classified_classified_detail_M
Name: listing_url, dtype: object
```

[28]:
```python
with pd.option_context('display.max_colwidth', None):
    print(df[df.floor == '9 Geschosse'].sample(5)['listing_url'])
```

```
20438    https://www.immowelt.de/expose/307c62c5-40fb-4314-9542-
a6f3f12d839e?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D50&sort_order_ab_test=sort_order
_reference#ln=classified_search_results&m=classified_search_results_classified_c
lassified_detail_L
6944                                            https://www.immowelt.de/expose/
307c62c5-40fb-4314-9542-
a6f3f12d839e?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D33#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_L
13278                                           https://www.immowelt.de/expose/
307c62c5-40fb-4314-9542-
a6f3f12d839e?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D45#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_L
3727              https://www.immowelt.de/expose/307c62c5-40fb-4314-9542-
a6f3f12d839e?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D23&feature_priceDropVariantB=tru
e#ln=classified_search_results&m=classified_search_results_classified_classified
_detail_L
18744                                           https://www.immowelt.de/expose/
307c62c5-40fb-4314-9542-
a6f3f12d839e?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D51#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_L
```

```
Name: listing_url, dtype: object
```

[29]:
```python
with pd.option_context('display.max_colwidth', None):
    print(df[df.floor == 'Geschoss -2/3'].sample(5)['listing_url'])
```

```
16434                                                https://www.immowelt.de/expose/
26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D61#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
21074    https://www.immowelt.de/expose/26fknlg5hddf?serp_view=list&search=distr
ibutionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%2
6page%3D71&sort_order_ab_test=sort_order_reference#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
14623                                                https://www.immowelt.de/expose/
26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D66#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
10216                                                https://www.immowelt.de/expose/
26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D54#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
8679                                                 https://www.immowelt.de/expose/
26fknlg5hddf?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D50#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
Name: listing_url, dtype: object
```

[30]:
```python
with pd.option_context('display.max_colwidth', None):
    print(df[df.floor == 'Geschoss -1/3'].sample(2)['listing_url'])
```

```
8098                                     https://www.immowelt.de/expose/261flizrfk
cq?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHouse%2CApartm
ent%26locations%3DAD08DE8634%26page%3D30#ln=classified_search_results&m=classifi
ed_search_results_classified_classified_detail_M
3014    https://www.immowelt.de/expose/246a1kdzqnik?serp_view=list&search=distri
butionTypes%3DRent%26estateTypes%3DHouse%2CApartment%26locations%3DAD08DE8634%26
page%3D45&feature_priceDropVariantB=true#ln=classified_search_results&m=classifi
ed_search_results_classified_classified_detail_M
Name: listing_url, dtype: object
```

After inspecting the values and some listings, we reach the conclusion that we have 6 types of values: 1. Good values like '5. Geschoss' from which we can simply extract the floor number 2. Other good values like 'EG' just stands for 'ground floor' meaning floor number 0 3. Bad values that just state the number of total floors '9 Geschosse', but not in which floor the apartment is, we cannot conclude any information from it, like the apartment being on the first, ground floor or the highest floor because by inspecting some listings, we noticed that it simply wasn't a particular reason why the specific floor wasn't mentioned. This values are to be considered as null values. 4. Good values in a fraction format shapped - these valuse contain the number of the floor and the number of the total floors. We are only going to extract the number of the floor from these values.

5. Confusing negative fraction values - I called them confusing because some of them really meant negative floors like -1, but values like -2, -3, -4 and -5 meant actually floor number 2,3 and so on rather than apartments buried into the ground. 6. NaN values The best solution in this case is to get the floor number from each feature and leave them as nan values as they are, since we want to have a continuous feature rather then replacing it with 'Unknown' and converting the feature to a categorical one just for one type of value. For the tricky negative-value values we only leave -1 and convert the rest as postive.

[31]:
```
# We will write a custom function to do the formating

import re

def format_floor(value):
    # Case 0: already numeric -> we don't do anything
    if isinstance(value, (int, float)) and not pd.isna(value):
        return int(value)

    # Case 1: NaN
    if pd.isna(value):
        return np.nan

    if not isinstance(value, str):
        return np.nan

    value = value.strip()

    # Case 2: Ground floor
    if value == "EG":
        return 0

    # Case 3: Simple format like "5. Geschoss"
    match = re.match(r"(-?\d+)\.\s*Geschoss", value)
    if match:
        floor = int(match.group(1))
        return floor if floor == -1 else abs(floor)

    # Case 4: Fraction format like "Geschoss 3/5" or "Geschoss -2/4"
    match = re.match(r"Geschoss\s+(-?\d+)\s*/\s*(\d+)", value)
    if match:
        floor = int(match.group(1))
        return floor if floor == -1 else abs(floor)

    # Case 5: Only building height like "9 Geschosse"
    if re.match(r"\d+\s+Geschosse", value):
        return np.nan

    # Anything else: unknown / unusable
```

```
    return np.nan
```

[32]:
```
df["floor"] = np.sort(df["floor"].apply(format_floor).astype("Int64"))
df["floor"].unique()
```

[32]:
```
array([-1.,  0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11.,
       12., 13., 14., 15., 16., 17., 18., 20., 21., nan])
```

[33]:
```
df.isnull().sum()
```

[33]:
```
listing_url                0
cold_rent                  0
warm_rent               9508
deposit                13102
area                       0
rooms                      0
floor                   3122
free_from               8530
location                 326
has_balkon                 0
has_terrasse               0
has_garten                 0
elevator                   0
parking                    0
has_basement               0
is_barrier-free            0
has_built-in_kitchen       0
has_bathtub                0
has_shower                 0
flooring_type          13698
energy_source          12139
heating_type            9063
property_condition     12411
year_built             10233
schufa_check               0
scraped_at                 0
dtype: int64
```

Features like warm_rent and deposit are directly linked with cold_rent - the target variable. warm_rent = cold_rent + utilities deposit is often a direct multiple of rent (e.g. 2–3×) Using them to train a model would cause data leakage, therefore we don't need them to train our model. We have to drop them.

[34]:
```
df = df.drop(columns=["warm_rent","deposit"])
```

[35]:
```
# Let's check what values does 'free_from' has:
df["free_from"].unique()
```

[35]:
```
array(['15.02.2026', '01.03.2026', 'sofort', '29.12.2025', '01.02.2026',
       '01.01.2026', 'ab sofort', nan, 'nach absprache', 'nach wunsch',
       '15.01.2026', '31.12.2025', '10.01.2026', 'nach vereinbarung',
       'ab 01.02.2026', '08.01.2026', '28.02.2026', '1.06.2025',
       'herbst 2025', 'auf anfrage', '14.02.2026', 'ab 01.04.2026',
       '13.02.2026', 'neuwertig', '12.01.2026', '01.04.2026',
       '16.01.2026', '31.01.2026', 'jan 2026', '29.01.2026', '02.02.2026',
       'mitte januar', '09.02.2026', '23.02.2026', '12.02.2026',
       '21.01.2026', 'ab märz 2026', 'ab 01.10.2025', '23.01.2026',
       '10.02.2026', 'ab 01.11.2025', '11.01.2026', '18.01.2026',
       '01.05.2026', 'bezugsfrei', '16.02.2026', '08.02.2026',
       '14.01.2026', '15.01.2026 sofort', '2013', '30.01.2026',
       '05.02.2026', '20.02.2026', '20.01.2026', '19.01.2026',
       '15.03.2026', '26.01.2026', '13.04.2026', '03.03.2026',
       '07.02.2026', '09.03.2026', '01.10.2026', 'an sofort',
       '1.02.2026 (abhängig vom baufortschritt)', '10.04.2026',
       '13.01.2026', '25.01.2026', '07.03.2026', '17.01.2026',
       '24.01.2026', '06.02.2026', '31.05.2026', '1.12.2025',
       '28.01.2026', '27.01.2026', '17.02.2026', '15.04.2026',
       '18.02.2026', '31.03.2026', '21.02.2026', '05.03.2026',
       'sofort möglich', '22.02.2026', '15.06.2026', '06.04.2026',
       '14.03.2026', '08.03.2026', '03.02.2026', 'verfügbar',
       '24.02.2026', '11.02.2026', '25.02.2026', '16.03.2026',
       '04.02.2026', '26.03.2026', 'erstbezug', '10.03.2026',
       '30.10.2026', '27.02.2026'], dtype=object)
```

There are different dates, words / group of words that mostly say that the date can be discussed or immediately available and the missing values (nan). We could transform this feature into number of days using 'scraped_at' feature (metadata from scraper), but that would only work only if every post was scraped on the same day that it was posted, which was not possible. The least we can do is transform the feature into a categorical feature using these 3 values: - 'immediately' - convert the text values because they all say so - 'not immediately' - dates, values that contain numbers like 'herbst 2025' or text that says 'depends' and similar things| - 'unknown' - nan values

[36]:
```python
# Let's create a function to format these values based on what was suggested

def format_availability(value):
    # Missing values
    if pd.isna(value):
        return "unknown"

    # If already formatted
    if value in {"immediately", "not_immediately", "unknown"}:
        return value

    # Only strings
    if not isinstance(value, str):
        return "unknown"
```

```
    v = value.strip().lower()

    # "now / free / immediately" type signals
    if re.search(r"\b(sofort|verfügbar|bezugsfrei|immediate)\b", v):
        return "immediately"

    # --- NOT IMMEDIATE ---
    # Any numeric reference → date or time
    if re.search(r"\d", v):
        return "not_immediately"

    # Temporal words (general, not specific dates)
    if re.search(
        ␣
↪r"\b(ab|nach|bis|seit|herbst|winter|sommer|frühjahr|spring|summer|fall|autumn)\b",
        v,
    ):
        return "not_immediately"

    # If none of the cases then it's unknown
    return "unknown"
```

[37]:
```
df["free_from"] = df["free_from"].apply(format_availability)
df["free_from"].value_counts()
```

[37]:
```
free_from
unknown            8565
immediately        8251
not_immediately    4442
Name: count, dtype: int64
```

If you observe carefully, this feature had 8530 missing values and after formatting 8565 were mapped as unknown, meaning there almost no problematic values.

[38]:
```
df.isnull().sum()[df.isnull().sum() > 0]
```

[38]:
```
floor                  3122
location                326
flooring_type         13698
energy_source         12139
heating_type           9063
property_condition    12411
year_built            10233
dtype: int64
```

[39]:
```
df["year_built"].unique()
```

[39]:
```
array(['2025', '1902', '2024', '2023', '2021', '2018', '2022', '1918',
       '2004', '1970', '2020', '2017', '2015', '1899', '1888', nan,
       '1954', '1900', '1986', '1911', '1995', '1908', '1905', '1980',
       '2026', '1910', '1906', '1992', '1994', '1912', '1890', '1938',
       '1950', '2019', '2010', '1903', '1935', '2016', '1998', '1955',
       '1960', '1996', '1977', '1978', '1965', '1920', '2000', '2007',
       '1976', '1930', '1989', '1982', '1988', '1962', '1997', '1894',
       '1880', '1993', '1979', '1961', '1974', '1904', '1967', '1963',
       '2003', '1984', '2011', '1946', '1969', '1966', '1957', '1892',
       '1959', '2005', '1907', '2006', '1878', '1971', '1956', '2008',
       '1864', '1973', '1896', '2001', '1765', '2009', '1913', '2014',
       '1968', '1915', '1929', '1958', '2012', '1925', '1934', '2013',
       '1936', '1916', '1928', '1983', '1990', '1895', '1919', '1999',
       '1953', 'neuwertig', '1901', '1937', '1870', '1926', '1985',
       '1927', '1012', '1887', '1839', '1949', '1972', '1850', '1877',
       '1975', '1907/2025', '1939', '1932', '1952', '1923', '2002',
       '1897', '1886', '1951', '1964', '1862', '2004/2018', '1893',
       '1933', '1991', '1914', '1875', '1931', '1874', '1909', '1898'],
      dtype=object)
```

[40]:
```
# we take the year built instead of year renovated in values like 'year_built/
↪year_renovated' and
# remove anomalies that could be cause by types like '1012' by replacing years␣
↪earlier than 1700 by nan
df["year_built"] = (
    df["year_built"]
    .astype(str)
    .str.extract(r"(\d{4})")[0]
    .astype("Int64")
)

# Remove implausible years
df.loc[df["year_built"] < 1700, "year_built"] = pd.NA
```

[41]:
```
df["year_built"].describe()
```

[41]:
```
count        11014.0
mean     1997.833031
std        43.552269
min           1765.0
25%           1995.0
50%           2021.0
75%           2024.0
max           2026.0
Name: year_built, dtype: Float64
```

Now the data looks good and we keep nan values.

[42]:
```
df["flooring_type"].unique()
```

[42]:
```
array(['fliesen, parkett', 'parkett', nan, 'holzdielen', 'teppich',
       'fliesen, kunststoff', 'fliesen, holzdielen, parkett', 'laminat',
       'fliesen', 'fliesen, parkett, granit', 'parkett, laminat',
       'fliesen, holzdielen', 'fliesen, laminat', 'kunststoff',
       'fliesen, laminat, terrakotta', 'holzdielen, kunststoff',
       'holzdielen, laminat, stein', 'holzdielen, parkett',
       'holzdielen, parkett, stein', 'fliesen, laminat, kunststoff',
       'laminat, kunststoff', 'fliesen, holzdielen, laminat',
       'parkett, kunststoff', 'holzdielen, laminat',
       'holzdielen, parkett, laminat', 'parkett, terrakotta',
       'kunststoff, stein', 'fliesen, parkett, laminat', 'linoleum',
       'marmor, holzdielen', 'holzdielen, parkett, terrakotta', 'stein',
       'holzdielen, stein', 'fliesen, parkett, kunststoff',
       'kunststoff, linoleum', 'fliesen, holzdielen, kunststoff',
       'fliesen, laminat, stein', 'marmor, laminat', 'parkett, stein',
       'fliesen, holzdielen, laminat, kunststoff',
       'holzdielen, terrakotta',
       'fliesen, holzdielen, parkett, kunststoff',
       'fliesen, parkett, stein', 'laminat, kunststoff, linoleum',
       'fliesen, holzdielen, kunststoff, linoleum', 'laminat, stein',
       'fliesen, teppich', 'fliesen, holzdielen, parkett, stein',
       'fliesen, laminat, stein, teppich', 'holzdielen, teppich',
       'fliesen, marmor, parkett', 'fliesen, linoleum, teppich',
       'fliesen, estrich, granit', 'marmor, parkett',
       'laminat, kunststoff, teppich', 'parkett, laminat, stein',
       'estrich', 'fliesen, parkett, teppich', 'parkett, doppelboden',
       'fliesen, holzdielen, parkett, laminat, kunststoff',
       'parkett, laminat, kunststoff',
       'fliesen, parkett, laminat, kunststoff', 'fliesen, linoleum',
       'fliesen, laminat, kunststoff, stein', 'fliesen, laminat, teppich',
       'laminat, kunststoff, stein', 'fliesen, holzdielen, terrakotta',
       'fliesen, parkett, laminat, teppich',
       'fliesen, parkett, kunststoff, teppich',
       'marmor, holzdielen, stein'], dtype=object)
```

[43]:
```
df["energy_source"].unique()
```

[43]:
```
array(['Fernwärme', nan, 'Luft-/Wasser-Wärmepumpe', 'Gas', 'Öl',
       'Blockheizkraftwerk', 'Gas, Solar', 'Fernwärme, Elektro, Solar',
       'Elektro', 'Fernwärme, Luft-/Wasser-Wärmepumpe', 'Gas,',
       'Fernwärme, Gas, Blockheizkraftwerk', 'Solar', 'Erdwärme',
       'Elektro, Gas', 'Gas, Öl', 'Fernwärme, Öl',
       'Erdwärme, Luft-/Wasser-Wärmepumpe', 'Fernwärme, Erdwärme',
       'Elektro, Luft-/Wasser-Wärmepumpe',
       'Gas, Solar, Luft-/Wasser-Wärmepumpe', 'Fernwärme, Gas',
```

```
       'Gas, Blockheizkraftwerk', 'Solar, Blockheizkraftwerk',
       'Fernwärme, Elektro', 'Elektro, Solar', 'Gas, Flüssiggas',
       'Gas, Luft-/Wasser-Wärmepumpe', 'Fernwärme, Blockheizkraftwerk',
       'Pellets', 'Solar, Luft-/Wasser-Wärmepumpe',
       'Elektro, Solar, Luft-/Wasser-Wärmepumpe',
       'Gas, Flüssiggas, Solar', 'Gas, fossile Brennstoffe',
       'Elektro, Öl', 'Erdwärme, Solar, Luft-/Wasser-Wärmepumpe',
       'Kohle, Gas, Öl', 'Fernwärme, Elektro, Gas',
       'Elektro, Gas, Blockheizkraftwerk', 'fossile Brennstoffe',
       'Fernwärme, Elektro, Blockheizkraftwerk', 'Flüssiggas'],
      dtype=object)
```

[44]:
```
df["heating_type"].unique()
```

[44]:
```
array(['Fußbodenheizung', 'Etagenheizung', 'Zentralheizung', nan,
       'Zentralheizung: Fußbodenheizung', 'individuell',
       'Zentralheizung: Heizkörper', 'Etagenheizung: Fußbodenheizung',
       'Ofen', 'individuell: Fußbodenheizung', 'individuell: Heizkörper'],
      dtype=object)
```

[45]:
```
df["property_condition"].unique()
```

[45]:
```
array(['Erstbezug', nan, 'Altbau (bis 1945), neuwertig', 'neuwertig',
       'Neubau, Erstbezug', 'Neubau, neuwertig', 'renoviert / saniert',
       'Gepflegt', 'Altbau (bis 1945), renoviert / saniert', 'Neubau',
       'Modernisiert', 'Neubau, renoviert / saniert', 'Neubau, Gepflegt',
       'Altbau (bis 1945), Gepflegt', 'Altbau (bis 1945)', 'Saniert',
       'Massivhaus, neuwertig', 'Massivhaus',
       'Massivhaus, renoviert / saniert',
       'renovierungsbedürftig / sanierungsbedürftig',
       'Neubau, Massivhaus', 'Massivhaus, Gepflegt', 'teilsaniert',
       'Erstbezug nach Renovierung', 'Massivhaus, Erstbezug',
       'Erstbezug, renoviert / saniert', 'neuwertig, renoviert / saniert',
       'Gepflegt, renoviert / saniert', ', renoviert / saniert',
       'Neubau, Erstbezug, renoviert / saniert',
       'Neubau, neuwertig, renoviert / saniert',
       'Altbau (bis 1945), neuwertig, renoviert / saniert',
       'Saniert, renoviert / saniert', 'Fertighaus, renoviert / saniert',
       'Erstbezug nach Renovierung, renoviert / saniert',
       'Altbau (bis 1945), Saniert, renoviert / saniert',
       'Neubau, Massivhaus, Gepflegt'], dtype=object)
```

For the remaining 4 features: 'flooring_type', 'energy_source', 'heating_type' and 'property_condition', since they are all categorical features, we just replace missing values with 'Unknown' and then for take care of them individually when we go through proper data formatting.

[46]:
```
df["flooring_type"] = df["flooring_type"].fillna("Unknown")
df["energy_source"] = df["energy_source"].fillna("Unknown")
```

```
df["heating_type"] = df["heating_type"].fillna("Unknown")
df["property_condition"] = df["property_condition"].fillna("Unknown")

df.isnull().sum()[df.isnull().sum() > 0]
```

[46]:
```
floor          3122
location        326
year_built    10244
dtype: int64
```

[47]:
```
# We drop the samples with location missing and check how many samples we have.
df = df.dropna(subset=["location"])
df.shape
```

[47]:
```
(20932, 24)
```

We just dropped (22687, 27) ==> (20932, 24) 1755 values and 3 columns

[48]:
```
1755/22687
```

[48]:
```
0.07735707673998325
```

Losing only 7% of the data set is a very good achievement. Usually, I drop 10-20% of the data set, so we did really well in this one.

Part 2: Handling dublications

Theoretically, we are not supposed to have dupplicated records in this data set, because the scraper is implemented in such a way that it reads all values from stored URLs, stores them in a list and does not scrape them again even if they appear, but rather skips the link. We still have to check for duplications because we cannot fully trust the scraper and also there might be listings who are reposted under a different listing, therefore having a different URL but the same data. For this reasons, first we will check first for duplicated URL / records and then for records that might be duplicated.

[49]:
```
df.duplicated().any()
```

[49]:
```
np.False_
```

No identical rows. That was expected since we have a unique feature: the 'scraped at'. Even if they have identical URL, that's why we check the duplicated records excluding these 2 features.

[50]:
```
# exclude the URL and  feature that surves as primary key
df.drop(columns=["listing_url","scraped_at"]).duplicated().any()
```

[50]:
```
np.True_
```

[51]:
```
df[df.drop(columns=["listing_url","scraped_at"]).duplicated()].head(5)
```

[51]:

```
                                          listing_url cold_rent   area  \
119  https://www.immowelt.de/expose/288c5511-4b4f-4…     360 €  93 m²
120  https://www.immowelt.de/expose/6806cf09-9057-4…     360 €  93 m²
548  https://www.immowelt.de/expose/25baabdc-7848-4…     232 €  34 m²
860  https://www.immowelt.de/expose/aa61a4ba-c557-4…     415 €  48 m²
900  https://www.immowelt.de/expose/e32e3e45-f2e6-4…     500 €  45 m²

        rooms  floor        free_from  \
119  1 Zimmer    0.0  not_immediately
120  1 Zimmer    0.0  not_immediately
548  1 Zimmer    0.0          unknown
860  2 Zimmer    0.0          unknown
900  2 Zimmer    0.0          unknown

                                  location  has_balkon  has_terrasse  \
119  Bahnhofstraße, Köpenick, Berlin (12555)       False         False
120  Bahnhofstraße, Köpenick, Berlin (12555)       False         False
548         Prenzlauer Berg, Berlin (10405)       False         False
860          Friedrichshain, Berlin (10249)       False         False
900                Neukölln, Berlin (12053)       False         False

     has_garten  …  has_built-in_kitchen  has_bathtub  has_shower  \
119       False  …                  True         True        True
120       False  …                  True         True        True
548       False  …                 False        False       False
860       False  …                 False        False       False
900       False  …                 False        False       False

     flooring_type  energy_source  heating_type  property_condition  \
119     holzdielen        Unknown       Unknown             Unknown
120     holzdielen        Unknown       Unknown             Unknown
548        Unknown        Unknown       Unknown             Unknown
860        Unknown        Unknown       Unknown             Unknown
900        Unknown        Unknown       Unknown             Unknown

    year_built schufa_check           scraped_at
119       2025         True  2025-12-28 21:59:05
120       2025         True  2025-12-28 21:59:47
548       <NA>         True  2025-12-28 23:22:15
860       <NA>         True  2025-12-31 15:35:16
900       <NA>         True  2025-12-31 15:41:47

[5 rows x 24 columns]
```

[52]:

```python
# in case data has different formats
df["scraped_at"] = pd.to_datetime(df["scraped_at"], format="mixed",␣
↪dayfirst=True)
```

```
# sort them in order to keep the newest versions
df = df.sort_values("scraped_at")

# drop duplicates with the earliest date and keep the latest version of the␣
↪samples
df = df.drop_duplicates(
    subset=df.columns.difference(["listing_url", "scraped_at"]),
    keep="last"
)
```

[53]:
```
df.shape
```

[53]: (16021, 24)

20% of the dataset contained duplicated rows.

There might be a chance that there are almost identical duplicated, but with one value difference. In rare cases where the scraper had scrapped the same listing to times, but field to scrape all the values in one of the cases. That is the reason why I want to check if they are duplicated rows without one or two of the other non-unique features.

[54]:
```
from itertools import combinations

# We totally exclude these columns
unique_cols = ["listing_url", "scraped_at"]

# We exclude the features that are more unique than the others like location,␣
↪price and area
# There is no point checking if samples are unique without them
# We exclude them only from iteration, not from sample-unique-check
static_cols = ["location", "cold_rent", "area", "floor"]

# All columns we want to check for duplicates
compare_cols = [c for c in df.columns if c not in unique_cols]

# Columns we'll iterate through for exclusion (excluding the static ones)
iter_cols = [c for c in compare_cols if c not in static_cols]

# Number of columns to exclude at a time
X = 1

SENTINEL = "__MISSING__"
problematic_rows = set()
found_combinations = []

# Try all combinations of X columns to exclude (only from iter_cols)
for excluded_cols in combinations(iter_cols, X):
```

```python
    # Columns to actually compare (all compare_cols except the excluded ones)
    cols_to_check = [c for c in compare_cols if c not in excluded_cols]

    # Find duplicates based on these columns
    temp_df = df[cols_to_check].astype("string").fillna(SENTINEL)
    dup_mask = temp_df.duplicated(keep=False)

    if dup_mask.any():
        dup_indices = df.index[dup_mask].tolist()
        problematic_rows.update(dup_indices)

        print(f"Excluded: {excluded_cols}")
        print(f"  Found {dup_mask.sum()} duplicate rows")
        found_combinations.append({
            'excluded_cols': excluded_cols,
            'num_duplicates': dup_mask.sum(),
            'duplicate_indices': dup_indices
        })
        print()

print(f"\nTotal unique problematic rows: {len(problematic_rows)}")
```

```
Excluded: ('rooms',)
  Found 4 duplicate rows

Excluded: ('free_from',)
  Found 86 duplicate rows

Excluded: ('has_balkon',)
  Found 18 duplicate rows

Excluded: ('has_terrasse',)
  Found 34 duplicate rows

Excluded: ('has_garten',)
  Found 10 duplicate rows

Excluded: ('elevator',)
  Found 26 duplicate rows

Excluded: ('parking',)
  Found 8 duplicate rows

Excluded: ('has_basement',)
  Found 12 duplicate rows

Excluded: ('is_barrier-free',)
  Found 4 duplicate rows
```

```
Excluded: ('has_built-in_kitchen',)
  Found 4 duplicate rows

Excluded: ('has_bathtub',)
  Found 4 duplicate rows

Excluded: ('has_shower',)
  Found 6 duplicate rows

Excluded: ('flooring_type',)
  Found 14 duplicate rows

Excluded: ('energy_source',)
  Found 12 duplicate rows

Excluded: ('heating_type',)
  Found 100 duplicate rows

Excluded: ('property_condition',)
  Found 94 duplicate rows

Excluded: ('year_built',)
  Found 10 duplicate rows


Total unique problematic rows: 440
```

[55]:
```
# Now we have to inspect all of these potential duplicates
df[df.drop(columns=["listing_url","scraped_at","rooms"]).duplicated(keep=False)]
```

[55]:
```
                                          listing_url cold_rent  \
209    https://www.immowelt.de/expose/b056646c-dea4-4…    1000 €
410    https://www.immowelt.de/expose/6c28dbf7-34ed-4…    1000 €
20311  https://www.immowelt.de/expose/e1cf3cb9-28e9-4…    2200 €
21451  https://www.immowelt.de/expose/cc8f124c-586b-4…    2200 €

                    area     rooms  floor    free_from  \
209                70 m²  2 Zimmer    0.0      unknown
410                70 m²  3 Zimmer    0.0      unknown
20311  916 m² Grundstück  4 Zimmer    NaN  immediately
21451  916 m² Grundstück  3 Zimmer    NaN  immediately

                            location  has_balkon  has_terrasse  has_garten  \
209    Prenzlauer Berg, Berlin (10437)       False         False       False
410    Prenzlauer Berg, Berlin (10437)       False         False       False
20311          Spandau, Berlin (13589)       False          True        True
```

```
21451          Spandau, Berlin (13589)       False          True        True

       …  has_built-in_kitchen  has_bathtub  has_shower  flooring_type  \
209    …                 False        False       False        Unknown
410    …                 False        False       False        Unknown
20311  …                  True        False        True        Unknown
21451  …                  True        False        True        Unknown

       energy_source    heating_type  property_condition year_built  \
209          Unknown         Unknown             Unknown       <NA>
410          Unknown         Unknown             Unknown       <NA>
20311             Öl  Zentralheizung          Massivhaus       <NA>
21451             Öl  Zentralheizung          Massivhaus       <NA>

      schufa_check          scraped_at
209           True 2025-12-28 22:19:21
410           True 2025-12-28 22:55:42
20311         True 2026-01-30 20:09:14
21451         True 2026-01-31 13:55:18

[4 rows x 24 columns]
```

This is the perfect case to show that there were indeed duplicates with little changes. We have 2 listings made within 2 days, possibly they deleted the original listing and posted it again later. Could be due the wrong number of rooms, therefore we will assume that the one with the most recent date is the true sample.

[56]:
```
df = df.drop(index=[209, 20311])
```

[57]:
```
df[df.drop(columns=["listing_url","scraped_at","free_from"]).
 ↪duplicated(keep=False)].head(10)
```

[57]:
```
                                          listing_url  cold_rent  \
3223  https://www.immowelt.de/expose/c74aeffc-e6f4-4…      771 €
3241  https://www.immowelt.de/expose/c5afedd9-ed30-4…     2650 €
3283  https://www.immowelt.de/expose/1a86d9ed-9730-4…      966 €
3285  https://www.immowelt.de/expose/d8e40997-88aa-4…     1239 €
3979  https://www.immowelt.de/expose/c74aeffc-e6f4-4…      771 €
3995  https://www.immowelt.de/expose/c5afedd9-ed30-4…     2650 €
4288  https://www.immowelt.de/expose/653de956-40d3-4…     1239 €
4342  https://www.immowelt.de/expose/1a86d9ed-9730-4…      966 €
5842  https://www.immowelt.de/expose/a6b0219f-740f-4…  2916.95 €
5870  https://www.immowelt.de/expose/25swdlasqtaq?se…     1250 €

                   area     rooms  floor        free_from  \
3223            52,8 m²  2 Zimmer    1.0  not_immediately
3241  300 m² Grundstück  4 Zimmer    1.0  not_immediately
3283              19 m²  1 Zimmer    1.0  not_immediately
```

3285	32 m²	1 Zimmer	1.0	not_immediately
3979	52,8 m²	2 Zimmer	1.0	immediately
3995	300 m² Grundstück	4 Zimmer	1.0	immediately
4288	32 m²	1 Zimmer	1.0	immediately
4342	19 m²	1 Zimmer	1.0	immediately
5842	158,1 m²	5 Zimmer	2.0	immediately
5870	70 m²	2 Zimmer	2.0	not_immediately

	location	has_balkon	has_terrasse	\
3223	Spandau, Berlin-Spandau (13583)	True	False	
3241	Tiniusstr. 35, Heinersdorf, Berlin (13089)	False	True	
3283	Köpenicker Straße 43, Mitte, Berlin (10179)	False	False	
3285	Köpenicker Straße 43, Mitte, Berlin (10179)	False	False	
3979	Spandau, Berlin-Spandau (13583)	True	False	
3995	Tiniusstr. 35, Heinersdorf, Berlin (13089)	False	True	
4288	Köpenicker Straße 43, Mitte, Berlin (10179)	False	False	
4342	Köpenicker Straße 43, Mitte, Berlin (10179)	False	False	
5842	Blumeneschenweg 14, Kaulsdorf, Berlin (12621)	False	True	
5870	Köpenick, Berlin (12555)	True	False	

	has_garten	…	has_built-in_kitchen	has_bathtub	has_shower	\
3223	False	…	True	False	True	
3241	True	…	True	False	False	
3283	True	…	False	False	True	
3285	True	…	False	False	True	
3979	False	…	True	False	True	
3995	True	…	True	False	False	
4288	True	…	False	False	True	
4342	True	…	False	False	True	
5842	True	…	True	False	False	
5870	False	…	True	False	False	

	flooring_type	energy_source	heating_type	\
3223	laminat	Gas	Zentralheizung	
3241	fliesen, kunststoff	Luft-/Wasser-Wärmepumpe	Fußbodenheizung	
3283	Unknown	Fernwärme	Zentralheizung	
3285	Unknown	Fernwärme	Zentralheizung	
3979	laminat	Gas	Zentralheizung	
3995	fliesen, kunststoff	Luft-/Wasser-Wärmepumpe	Fußbodenheizung	
4288	Unknown	Fernwärme	Zentralheizung	
4342	Unknown	Fernwärme	Zentralheizung	
5842	Unknown	Elektro	Unknown	
5870	Unknown	Gas	Zentralheizung	

	property_condition	year_built	schufa_check	scraped_at
3223	Massivhaus	1954	True	2025-12-31 22:57:33
3241	Erstbezug	2025	True	2025-12-31 23:01:18

```
3283             Unknown       2014         True 2025-12-31 23:11:21
3285             Unknown       2014         True 2025-12-31 23:11:52
3979          Massivhaus       1954         True 2026-01-10 16:31:38
3995           Erstbezug       2025         True 2026-01-10 16:34:59
4288             Unknown       2014         True 2026-01-11 13:01:12
4342             Unknown       2014         True 2026-01-11 13:11:43
5842             Unknown       2026         True 2026-01-13 15:03:29
5870            Gepflegt       <NA>         True 2026-01-13 15:09:07

[10 rows x 24 columns]
```

[58]:
```
# we drop the old ones and keep the most recent listing

subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
 ↪"free_from"]]

print(f"Before: {df.shape[0]} samples")

df = (
    df
    .sort_values("scraped_at", ascending=False)
    .drop_duplicates(subset=subset, keep="first")
)

print(f"After: {df.shape[0]} samples")
```

```
Before: 16019 samples
After: 15976 samples
```

[59]:
```
df[df.drop(columns=["listing_url","scraped_at","has_balkon"]).
 ↪duplicated(keep=False)]
```

[59]:
```
                                          listing_url cold_rent     area  \
21491  https://www.immowelt.de/expose/6b9f4760-2e38-4…    1900 €   125 m²
19487  https://www.immowelt.de/expose/910aef75-a3dc-4…    1900 €   125 m²
16967  https://www.immowelt.de/expose/910aef75-a3dc-4…    1900 €   125 m²
16920  https://www.immowelt.de/expose/6b9f4760-2e38-4…    1900 €   125 m²
11913  https://www.immowelt.de/expose/910aef75-a3dc-4…    1900 €   125 m²
11868  https://www.immowelt.de/expose/6b9f4760-2e38-4…    1900 €   125 m²
7890   https://www.immowelt.de/expose/910aef75-a3dc-4…    1900 €   125 m²
7837   https://www.immowelt.de/expose/6b9f4760-2e38-4…    1900 €   125 m²
5776   https://www.immowelt.de/expose/6e395239-885f-4…     680 €  27,8 m²
5135   https://www.immowelt.de/expose/6b9f4760-2e38-4…    1900 €   125 m²
5071   https://www.immowelt.de/expose/910aef75-a3dc-4…    1900 €   125 m²
4405   https://www.immowelt.de/expose/c4e61499-bc1f-4…     765 €    32 m²
4396   https://www.immowelt.de/expose/5df5bd39-e128-4…  966.67 €  68,5 m²
4095   https://www.immowelt.de/expose/f3800ef4-be39-4…  431.72 €  42,1 m²
4039   https://www.immowelt.de/expose/aee1f5f1-5af4-4…     680 €  27,8 m²
```

```
3451   https://www.immowelt.de/expose/9da55270-dfcb-4…     765 €    32 m²
3280   https://www.immowelt.de/expose/5df5bd39-e128-4…  966.67 €  68,5 m²
3237   https://www.immowelt.de/expose/f3800ef4-be39-4…  431.72 €  42,1 m²

          rooms  floor        free_from  \
21491  4 Zimmer    NaN      immediately
19487  4 Zimmer    NaN      immediately
16967  4 Zimmer    5.0      immediately
16920  4 Zimmer    5.0      immediately
11913  4 Zimmer    3.0      immediately
11868  4 Zimmer    3.0      immediately
7890   4 Zimmer    2.0      immediately
7837   4 Zimmer    2.0      immediately
5776   1 Zimmer    1.0      immediately
5135   4 Zimmer    1.0      immediately
5071   4 Zimmer    1.0      immediately
4405   1 Zimmer    1.0  not_immediately
4396   2 Zimmer    1.0      immediately
4095   1 Zimmer    1.0      immediately
4039   1 Zimmer    1.0      immediately
3451   1 Zimmer    1.0  not_immediately
3280   2 Zimmer    1.0      immediately
3237   1 Zimmer    1.0      immediately

                                              location  has_balkon  \
21491  Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…       False
19487  Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…        True
16967  Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…        True
16920  Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…       False
11913  Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…        True
11868  Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…       False
7890   Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…        True
7837   Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…       False
5776        Rennbahnstraße 83, Weißensee, Berlin (13086)        True
5135   Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…       False
5071   Elsa-Neumann-Straße 1, Siemensstadt, Berlin (1…        True
4405              Lindenstraße, Köpenick, Berlin (12555)        True
4396   Ingeborg-Feustel-Straße 9, Hellersdorf, Berlin…        True
4095           Emser Straße 88, Neukölln, Berlin (12051)        True
4039        Rennbahnstraße 83, Weißensee, Berlin (13086)       False
3451              Lindenstraße, Köpenick, Berlin (12555)       False
3280   Ingeborg-Feustel-Straße 9, Hellersdorf, Berlin…       False
3237           Emser Straße 88, Neukölln, Berlin (12051)       False

       has_terrasse  has_garten  …  has_built-in_kitchen  has_bathtub  \
21491         False       False  …                  True        False
19487         False       False  …                  True        False
```

16967	False	False	…	True	False
16920	False	False	…	True	False
11913	False	False	…	True	False
11868	False	False	…	True	False
7890	False	False	…	True	False
7837	False	False	…	True	False
5776	False	False	…	True	False
5135	False	False	…	True	False
5071	False	False	…	True	False
4405	False	False	…	True	False
4396	False	False	…	False	False
4095	False	False	…	True	False
4039	False	False	…	True	False
3451	False	False	…	True	False
3280	False	False	…	False	False
3237	False	False	…	True	False

	has_shower	flooring_type	energy_source	\
21491	False	fliesen, parkett	Fernwärme	
19487	False	fliesen, parkett	Fernwärme	
16967	False	fliesen, parkett	Fernwärme	
16920	False	fliesen, parkett	Fernwärme	
11913	False	fliesen, parkett	Fernwärme	
11868	False	fliesen, parkett	Fernwärme	
7890	False	fliesen, parkett	Fernwärme	
7837	False	fliesen, parkett	Fernwärme	
5776	True	fliesen	Gas	
5135	False	fliesen, parkett	Fernwärme	
5071	False	fliesen, parkett	Fernwärme	
4405	True	teppich	Unknown	
4396	True	Unknown	Fernwärme	
4095	True	Unknown	Fernwärme	
4039	True	fliesen	Gas	
3451	True	teppich	Unknown	
3280	True	Unknown	Fernwärme	
3237	True	Unknown	Fernwärme	

	heating_type	property_condition	year_built	\
21491	Zentralheizung: Fußbodenheizung	Gepflegt	2024	
19487	Zentralheizung: Fußbodenheizung	Gepflegt	2024	
16967	Zentralheizung: Fußbodenheizung	Gepflegt	2024	
16920	Zentralheizung: Fußbodenheizung	Gepflegt	2024	
11913	Zentralheizung: Fußbodenheizung	Gepflegt	2024	
11868	Zentralheizung: Fußbodenheizung	Gepflegt	2024	
7890	Zentralheizung: Fußbodenheizung	Gepflegt	2024	
7837	Zentralheizung: Fußbodenheizung	Gepflegt	2024	
5776	Zentralheizung	neuwertig	2023	

```
5135   Zentralheizung: Fußbodenheizung             Gepflegt       2024
5071   Zentralheizung: Fußbodenheizung             Gepflegt       2024
4405                           Unknown              Unknown       2017
4396                   Fußbodenheizung              Unknown       2023
4095                    Zentralheizung  renoviert / saniert       1900
4039                    Zentralheizung            neuwertig       2023
3451                           Unknown              Unknown       2017
3280                   Fußbodenheizung              Unknown       2023
3237                    Zentralheizung  renoviert / saniert       1900

      schufa_check          scraped_at
21491         True 2026-01-31 14:00:33
19487         True 2026-01-30 18:00:19
16967         True 2026-01-25 11:13:57
16920         True 2026-01-25 11:02:07
11913         True 2026-01-20 22:11:45
11868         True 2026-01-20 21:58:15
7890          True 2026-01-16 19:52:24
7837          True 2026-01-16 19:43:57
5776          True 2026-01-13 14:50:30
5135          True 2026-01-11 15:50:05
5071          True 2026-01-11 15:37:44
4405          True 2026-01-11 13:24:42
4396          True 2026-01-11 13:22:43
4095          True 2026-01-10 17:00:22
4039          True 2026-01-10 16:45:37
3451          True 2025-12-31 23:46:58
3280          True 2025-12-31 23:10:35
3237          True 2025-12-31 23:00:27

[18 rows x 24 columns]
```

[60]:
```python
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"has_balkon"]]

print(f"Before: {df.shape[0]} samples")

df = (
    df
    .sort_values("scraped_at", ascending=False)
    .drop_duplicates(subset=subset, keep="first")
)

print(f"After: {df.shape[0]} samples")
```

```
Before: 15976 samples
After: 15967 samples
```

[61]:
```
df[df.drop(columns=["listing_url","scraped_at","has_terrasse"]).
 ↪duplicated(keep=False)]
```

[61]:
```
                                          listing_url cold_rent      area  \
5714  https://www.immowelt.de/expose/4da5fdaf-a792-4…    2299 €  100,5 m²
5710  https://www.immowelt.de/expose/afa02248-6b1f-4…    2480 €   84,5 m²
5701  https://www.immowelt.de/expose/82da02a1-d358-4…    1549 €   60,1 m²
5700  https://www.immowelt.de/expose/1d3ece94-51bf-4…    1469 €   55,5 m²
5699  https://www.immowelt.de/expose/31e16e72-7fab-4…    2050 €   87,5 m²
5663  https://www.immowelt.de/expose/abecabb0-2fab-4…    2349 €  121,4 m²
5653  https://www.immowelt.de/expose/8d9c9ef0-3a2f-4…    3239 €  178,3 m²
5652  https://www.immowelt.de/expose/2968d626-7614-4…    1189 €   43,4 m²
5638  https://www.immowelt.de/expose/c7e7a447-8aab-4…     995 €   36,2 m²
5633  https://www.immowelt.de/expose/7a0b51f4-5e93-4…    1675 €   69,2 m²
5611  https://www.immowelt.de/expose/c7e7a447-8aab-4…     995 €   36,2 m²
5606  https://www.immowelt.de/expose/7a0b51f4-5e93-4…    1675 €   69,2 m²
5597  https://www.immowelt.de/expose/abecabb0-2fab-4…    2349 €  121,4 m²
5595  https://www.immowelt.de/expose/971b5f79-ce03-4…    2299 €  100,5 m²
5581  https://www.immowelt.de/expose/8d9c9ef0-3a2f-4…    3239 €  178,3 m²
5580  https://www.immowelt.de/expose/2968d626-7614-4…    1189 €   43,4 m²
5500  https://www.immowelt.de/expose/82da02a1-d358-4…    1549 €   60,1 m²
5499  https://www.immowelt.de/expose/1d3ece94-51bf-4…    1469 €   55,5 m²
5498  https://www.immowelt.de/expose/31e16e72-7fab-4…    2050 €   87,5 m²
5457  https://www.immowelt.de/expose/afa02248-6b1f-4…    2480 €   84,5 m²
5162  https://www.immowelt.de/expose/270862dc-a9a1-4…    1169 €   51,2 m²
5097  https://www.immowelt.de/expose/7bbd68d8-b447-4…    1150 €     74 m²
5070  https://www.immowelt.de/expose/aceecbf2-4776-4…    1150 €     74 m²
4994  https://www.immowelt.de/expose/428c0787-7dd8-4…    1169 €   51,2 m²
4898  https://www.immowelt.de/expose/52561aa8-0ffa-4…    1025 €   36,7 m²
4807  https://www.immowelt.de/expose/b089bf00-2f80-4…     897 €   48,5 m²
4332  https://www.immowelt.de/expose/6c0cd9c1-7a37-4…  988.24 €   62,1 m²
3999  https://www.immowelt.de/expose/f5530ea4-d5a0-4…  1439.4 €     96 m²
3998  https://www.immowelt.de/expose/8bb0f98f-88d4-4…    1642 €     78 m²
3624  https://www.immowelt.de/expose/02aff49f-e60b-4…    1025 €   36,7 m²
3570  https://www.immowelt.de/expose/b364e497-a8a7-4…     897 €   48,5 m²
3264  https://www.immowelt.de/expose/6c0cd9c1-7a37-4…  988.24 €   62,1 m²
3157  https://www.immowelt.de/expose/f5530ea4-d5a0-4…  1439.4 €     96 m²
2862  https://www.immowelt.de/expose/8bb0f98f-88d4-4…    1642 €     78 m²

         rooms  floor        free_from  \
5714  3 Zimmer    1.0      immediately
5710  4 Zimmer    1.0      immediately
5701  2 Zimmer    1.0  not_immediately
5700  2 Zimmer    1.0  not_immediately
5699  3 Zimmer    1.0  not_immediately
5663  4 Zimmer    1.0      immediately
5653  4 Zimmer    1.0  not_immediately
```

```
5652  2 Zimmer    1.0  not_immediately
5638  1 Zimmer    1.0      immediately
5633  3 Zimmer    1.0      immediately
5611  1 Zimmer    1.0      immediately
5606  3 Zimmer    1.0      immediately
5597  4 Zimmer    1.0      immediately
5595  3 Zimmer    1.0      immediately
5581  4 Zimmer    1.0  not_immediately
5580  2 Zimmer    1.0  not_immediately
5500  2 Zimmer    1.0  not_immediately
5499  2 Zimmer    1.0  not_immediately
5498  3 Zimmer    1.0  not_immediately
5457  4 Zimmer    1.0      immediately
5162  2 Zimmer    1.0  not_immediately
5097  2 Zimmer    1.0      immediately
5070  2 Zimmer    1.0      immediately
4994  2 Zimmer    1.0  not_immediately
4898  1 Zimmer    1.0  not_immediately
4807  2 Zimmer    1.0  not_immediately
4332  2 Zimmer    1.0      immediately
3999  2 Zimmer    1.0      immediately
3998  2 Zimmer    1.0      immediately
3624  1 Zimmer    1.0  not_immediately
3570  2 Zimmer    1.0  not_immediately
3264  2 Zimmer    1.0      immediately
3157  2 Zimmer    1.0      immediately
2862  2 Zimmer    1.0      immediately

                                                 location  has_balkon  \
5714  Carl-Spindler-Straße 19, Köpenick, Berlin (12555)        True
5710                              Mitte, Berlin (10115)       False
5701  Archenholdstrasse 21, Friedrichsfelde, Berlin …        True
5700  Archenholdstrasse 19, Friedrichsfelde, Berlin …        True
5699  Archenholdstrasse 21, Friedrichsfelde, Berlin …        True
5663        Wäscherinnenweg 2, Köpenick, Berlin (12555)        True
5653  Danziger Straße 73, Prenzlauer Berg, Berlin (1…        True
5652  Danziger Straße 73, Prenzlauer Berg, Berlin (1…        True
5638      Peter-Lennè-Straße 49, Dahlem, Berlin (14195)        True
5633      Peter-Lennè-Straße 43, Dahlem, Berlin (14195)        True
5611      Peter-Lennè-Straße 49, Dahlem, Berlin (14195)        True
5606      Peter-Lennè-Straße 43, Dahlem, Berlin (14195)        True
5597        Wäscherinnenweg 2, Köpenick, Berlin (12555)        True
5595  Carl-Spindler-Straße 19, Köpenick, Berlin (12555)        True
5581  Danziger Straße 73, Prenzlauer Berg, Berlin (1…        True
5580  Danziger Straße 73, Prenzlauer Berg, Berlin (1…        True
5500  Archenholdstrasse 21, Friedrichsfelde, Berlin …        True
5499  Archenholdstrasse 19, Friedrichsfelde, Berlin …        True
```

5498	Archenholdstrasse 21, Friedrichsfelde, Berlin …	True
5457	Mitte, Berlin (10115)	False
5162	Am Maselakepark 35, Spandau, Berlin (13587)	True
5097	Heinrich-Hertz-Straße 4, Siemensstadt, Berlin …	True
5070	Heinrich-Hertz-Straße 4, Siemensstadt, Berlin …	True
4994	Am Maselakepark 35, Spandau, Berlin (13587)	True
4898	Bornholmer Straße, Prenzlauer Berg, Berlin (10…	True
4807	Lion-Feuchtwanger-Straße, Kaulsdorf, Berlin (1…	True
4332	Gadebuscher Straße 25A, Kaulsdorf, Berlin (12619)	True
3999	Boxhagener Straße 27, Friedrichshain, Berlin (…	True
3998	Gesundbrunnen, Berlin (13347)	True
3624	Bornholmer Straße, Prenzlauer Berg, Berlin (10…	True
3570	Lion-Feuchtwanger-Straße, Kaulsdorf, Berlin (1…	True
3264	Gadebuscher Straße 25A, Kaulsdorf, Berlin (12619)	True
3157	Boxhagener Straße 27, Friedrichshain, Berlin (…	True
2862	Gesundbrunnen, Berlin (13347)	True

	has_terrasse	has_garten	…	has_built-in_kitchen	has_bathtub	\
5714	False	False	…	True	False	
5710	False	False	…	True	True	
5701	False	False	…	True	False	
5700	False	False	…	True	False	
5699	False	False	…	True	True	
5663	False	False	…	True	False	
5653	False	False	…	True	False	
5652	False	False	…	True	False	
5638	False	False	…	True	False	
5633	False	False	…	True	False	
5611	True	False	…	True	False	
5606	True	False	…	True	False	
5597	True	False	…	True	False	
5595	True	False	…	True	False	
5581	True	False	…	True	False	
5580	True	False	…	True	False	
5500	True	False	…	True	False	
5499	True	False	…	True	False	
5498	True	False	…	True	True	
5457	True	False	…	True	True	
5162	True	False	…	True	False	
5097	True	False	…	True	False	
5070	False	False	…	True	False	
4994	False	False	…	True	False	
4898	True	False	…	True	False	
4807	True	True	…	True	False	
4332	True	False	…	False	False	
3999	True	True	…	False	True	
3998	True	False	…	True	True	

3624	False	False	…	True	False
3570	False	True	…	True	False
3264	False	False	…	False	False
3157	False	True	…	False	True
2862	False	False	…	True	True

	has_shower	flooring_type	energy_source	\
5714	True	parkett	Fernwärme	
5710	True	fliesen, parkett	Fernwärme	
5701	True	parkett	Fernwärme	
5700	True	parkett	Fernwärme	
5699	True	parkett	Fernwärme	
5663	True	parkett	Fernwärme	
5653	False	Unknown	Fernwärme	
5652	False	Unknown	Fernwärme	
5638	False	Unknown	Fernwärme	
5633	False	Unknown	Fernwärme	
5611	False	Unknown	Fernwärme	
5606	False	Unknown	Fernwärme	
5597	True	parkett	Fernwärme	
5595	True	parkett	Fernwärme	
5581	False	Unknown	Fernwärme	
5580	False	Unknown	Fernwärme	
5500	True	parkett	Fernwärme	
5499	True	parkett	Fernwärme	
5498	True	parkett	Fernwärme	
5457	True	fliesen, parkett	Fernwärme	
5162	False	Unknown	Fernwärme	
5097	False	fliesen, parkett	Fernwärme	
5070	False	fliesen, parkett	Fernwärme	
4994	False	Unknown	Fernwärme	
4898	True	holzdielen	Unknown	
4807	True	Unknown	Unknown	
4332	True	Unknown	Fernwärme	
3999	False	holzdielen	Fernwärme	
3998	True	Unknown	Gas	
3624	True	holzdielen	Unknown	
3570	True	Unknown	Unknown	
3264	True	Unknown	Fernwärme	
3157	False	holzdielen	Fernwärme	
2862	True	Unknown	Gas	

	heating_type	property_condition	year_built	\
5714	Fußbodenheizung	Erstbezug	2024	
5710	Fußbodenheizung	neuwertig	2021	
5701	Fußbodenheizung	Erstbezug	2025	
5700	Fußbodenheizung	Erstbezug	2025	

5699		Fußbodenheizung	Erstbezug	2025
5663		Fußbodenheizung	Erstbezug	2024
5653		Zentralheizung	renoviert / saniert	2015
5652		Zentralheizung	renoviert / saniert	2015
5638		Fußbodenheizung	Neubau, Erstbezug	2025
5633		Fußbodenheizung	Neubau, Erstbezug	2025
5611		Fußbodenheizung	Neubau, Erstbezug	2025
5606		Fußbodenheizung	Neubau, Erstbezug	2025
5597		Fußbodenheizung	Erstbezug	2024
5595		Fußbodenheizung	Erstbezug	2024
5581		Zentralheizung	renoviert / saniert	2015
5580		Zentralheizung	renoviert / saniert	2015
5500		Fußbodenheizung	Erstbezug	2025
5499		Fußbodenheizung	Erstbezug	2025
5498		Fußbodenheizung	Erstbezug	2025
5457		Fußbodenheizung	neuwertig	2021
5162		Zentralheizung	Unknown	2024
5097	Zentralheizung:	Fußbodenheizung	Gepflegt	2024
5070	Zentralheizung:	Fußbodenheizung	Gepflegt	2024
4994		Zentralheizung	Unknown	2024
4898	Etagenheizung:	Fußbodenheizung	Unknown	2019
4807	Etagenheizung:	Fußbodenheizung	Unknown	2023
4332		Fußbodenheizung	Unknown	2020
3999		Zentralheizung	Gepflegt	<NA>
3998		Zentralheizung	renoviert / saniert	1971
3624	Etagenheizung:	Fußbodenheizung	Unknown	2019
3570	Etagenheizung:	Fußbodenheizung	Unknown	2023
3264		Fußbodenheizung	Unknown	2020
3157		Zentralheizung	Gepflegt	<NA>
2862		Zentralheizung	renoviert / saniert	1971

	schufa_check	scraped_at
5714	True	2026-01-13 14:38:24
5710	True	2026-01-13 14:37:41
5701	True	2026-01-13 14:36:04
5700	True	2026-01-13 14:35:50
5699	True	2026-01-13 14:35:36
5663	True	2026-01-13 14:28:06
5653	True	2026-01-13 14:26:24
5652	True	2026-01-13 14:26:17
5638	True	2026-01-13 14:24:18
5633	True	2026-01-13 14:23:40
5611	True	2026-01-12 14:16:26
5606	True	2026-01-12 14:15:45
5597	True	2026-01-11 17:18:49
5595	True	2026-01-11 17:18:27
5581	True	2026-01-11 17:15:50

```
5580          True 2026-01-11 17:15:38
5500          True 2026-01-11 17:01:17
5499          True 2026-01-11 17:01:06
5498          True 2026-01-11 17:00:55
5457          True 2026-01-11 16:52:44
5162          True 2026-01-11 15:55:01
5097          True 2026-01-11 15:42:48
5070          True 2026-01-11 15:37:35
4994          True 2026-01-11 15:23:25
4898          True 2026-01-11 15:06:20
4807          True 2026-01-11 14:50:26
4332          True 2026-01-11 13:09:25
3999          True 2026-01-10 16:35:47
3998          True 2026-01-10 16:35:30
3624          True 2026-01-10 15:10:14
3570          True 2026-01-10 14:59:06
3264          True 2025-12-31 23:06:44
3157          True 2025-12-31 22:43:22
2862          True 2025-12-31 21:42:43

[34 rows x 24 columns]
```

[62]:
```
# We keep the ones with true 'has_tarrase'
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"has_terrasse"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_terrasse", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15950 samples
```

[63]:
```
df[df.drop(columns=["listing_url","scraped_at","has_garten"]).
↪duplicated(keep=False)]
```

[63]:
```
                                          listing_url cold_rent     area  \
4001  https://www.immowelt.de/expose/3f918689-673a-4…     650 €    45 m²
4110  https://www.immowelt.de/expose/49d10714-984a-4…     650 €    60 m²
2923  https://www.immowelt.de/expose/3f918689-673a-4…     650 €    45 m²
2977  https://www.immowelt.de/expose/49d10714-984a-4…     650 €    60 m²
3575  https://www.immowelt.de/expose/b2efac81-2bca-4…     431 €    16 m²
3572  https://www.immowelt.de/expose/535a8a2f-a240-4…    1056 €  53,7 m²
3512  https://www.immowelt.de/expose/48940456-e12b-4…    1030 €    42 m²
4735  https://www.immowelt.de/expose/9696b5f0-551b-4…    1030 €    42 m²
```

```
4831  https://www.immowelt.de/expose/696e7f4b-6034-4…     431 €    16 m²
4808  https://www.immowelt.de/expose/1cb45d7f-486c-4…    1056 €  53,7 m²

         rooms  floor        free_from  \
4001  2 Zimmer    1.0          unknown
4110  2 Zimmer    1.0          unknown
2923  2 Zimmer    1.0          unknown
2977  2 Zimmer    1.0          unknown
3575  1 Zimmer    1.0  not_immediately
3572  2 Zimmer    1.0  not_immediately
3512  1 Zimmer    1.0  not_immediately
4735  1 Zimmer    1.0  not_immediately
4831  1 Zimmer    1.0  not_immediately
4808  2 Zimmer    1.0  not_immediately

                                     location  has_balkon  has_terrasse  \
4001                   Köpenick, Berlin (12555)        True          True
4110                 Schöneberg, Berlin (10823)        True          True
2923                   Köpenick, Berlin (12555)        True          True
2977                 Schöneberg, Berlin (10823)        True          True
3575  Teterower Ring, Kaulsdorf, Berlin (12619)        True         False
3572  Teterower Ring, Kaulsdorf, Berlin (12619)        True         False
3512     Emdener Straße, Moabit, Berlin (10551)       False         False
4735     Emdener Straße, Moabit, Berlin (10551)       False         False
4831  Teterower Ring, Kaulsdorf, Berlin (12619)        True         False
4808  Teterower Ring, Kaulsdorf, Berlin (12619)        True         False

      has_garten  …  has_built-in_kitchen  has_bathtub  has_shower  \
4001        True  …                 False         True       False
4110        True  …                  True        False       False
2923       False  …                 False         True       False
2977       False  …                  True        False       False
3575        True  …                  True        False        True
3572        True  …                  True        False        True
3512       False  …                  True        False        True
4735        True  …                  True        False        True
4831       False  …                  True        False        True
4808       False  …                  True        False        True

               flooring_type  energy_source                    heating_type  \
4001  holzdielen, kunststoff        Unknown                  Zentralheizung
4110                laminat        Unknown                  Zentralheizung
2923  holzdielen, kunststoff        Unknown                  Zentralheizung
2977                laminat        Unknown                  Zentralheizung
3575                Unknown        Unknown  Etagenheizung: Fußbodenheizung
3572                Unknown        Unknown  Etagenheizung: Fußbodenheizung
3512    holzdielen, teppich        Unknown                         Unknown
```

```
4735     holzdielen, teppich        Unknown                          Unknown
4831                 Unknown        Unknown  Etagenheizung: Fußbodenheizung
4808                 Unknown        Unknown  Etagenheizung: Fußbodenheizung

      property_condition year_built schufa_check          scraped_at
4001             Unknown       <NA>         True 2026-01-10 16:36:20
4110             Unknown       <NA>         True 2026-01-10 17:04:26
2923             Unknown       <NA>         True 2025-12-31 21:55:22
2977             Unknown       <NA>         True 2025-12-31 22:05:45
3575             Unknown       2023         True 2026-01-10 15:00:08
3572             Unknown       2023         True 2026-01-10 14:59:36
3512             Unknown       1930         True 2026-01-10 14:46:57
4735             Unknown       1930         True 2026-01-11 14:34:47
4831             Unknown       2023         True 2026-01-11 14:54:38
4808             Unknown       2023         True 2026-01-11 14:50:40

[10 rows x 24 columns]
```

[64]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"has_garten"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_garten", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15945 samples
```

For boolean values we won't inspect the samples anymore, we will just immediately drop them.

[65]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"elevator"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("elevator", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15932 samples
```

[66]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"parking"]]
```

```
df = (
    df
    # True first, then False, then NaN
    .sort_values("parking", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15928 samples
```

[67]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
 ↪"has_basement"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_basement", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15922 samples
```

[68]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
 ↪"is_barrier-free"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("is_barrier-free", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15920 samples
```

[69]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
 ↪"has_built-in_kitchen"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_built-in_kitchen", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15918 samples
```

[70]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"has_bathtub"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_bathtub", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15916 samples
```

[71]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"has_shower"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("has_shower", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15913 samples
```

[72]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"flooring_type"]]

df = (
    df
    # True first, then False, then NaN
    .sort_values("flooring_type", ascending=False, na_position="last")
    .drop_duplicates(subset=subset, keep="first")
)
print(f"After: {df.shape[0]} samples")
```

```
After: 15908 samples
```

[73]:
```
df[df.drop(columns=["listing_url","scraped_at","energy_source"]).
↪duplicated(keep=False)]
```

[73]:
```
                                          listing_url cold_rent     area  \
6386  https://www.immowelt.de/expose/05310f59-6821-4…     665 €  26,5 m²
7629  https://www.immowelt.de/expose/23007f52-c757-4…     730 €  29,2 m²
6378  https://www.immowelt.de/expose/64d88100-c1b6-4…     700 €  27,4 m²
6379  https://www.immowelt.de/expose/4435f29c-4172-4…     700 €  27,8 m²
7533  https://www.immowelt.de/expose/4435f29c-4172-4…     700 €  27,8 m²
```

```
7609  https://www.immowelt.de/expose/a6fc625a-beb4-4…     625 €  24,9 m²
7532  https://www.immowelt.de/expose/64d88100-c1b6-4…     700 €  27,4 m²
5833  https://www.immowelt.de/expose/23007f52-c757-4…     730 €  29,2 m²
7542  https://www.immowelt.de/expose/05310f59-6821-4…     665 €  26,5 m²
6211  https://www.immowelt.de/expose/a6fc625a-beb4-4…     625 €  24,9 m²
7610  https://www.immowelt.de/expose/45f0109a-a961-4…     740 €  29,5 m²
6227  https://www.immowelt.de/expose/45f0109a-a961-4…     740 €  29,5 m²

         rooms  floor        free_from  \
6386  1 Zimmer    2.0  not_immediately
7629  1 Zimmer    2.0  not_immediately
6378  1 Zimmer    2.0  not_immediately
6379  1 Zimmer    2.0  not_immediately
7533  1 Zimmer    2.0  not_immediately
7609  1 Zimmer    2.0  not_immediately
7532  1 Zimmer    2.0  not_immediately
5833  1 Zimmer    2.0  not_immediately
7542  1 Zimmer    2.0  not_immediately
6211  1 Zimmer    2.0  not_immediately
7610  1 Zimmer    2.0  not_immediately
6227  1 Zimmer    2.0  not_immediately

                                     location  has_balkon  has_terrasse  \
6386  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
7629  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
6378  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
6379  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
7533  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
7609  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
7532  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
5833  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
7542  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
6211  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
7610  Donaustraße 42c, Neukölln, Berlin (12043)        True         False
6227  Donaustraße 42c, Neukölln, Berlin (12043)        True         False

      has_garten  …  has_built-in_kitchen  has_bathtub  has_shower  \
6386       False  …                  True        False       False
7629       False  …                  True        False       False
6378       False  …                  True        False       False
6379       False  …                  True        False       False
7533       False  …                  True        False       False
7609       False  …                  True        False       False
7532       False  …                  True        False       False
5833       False  …                  True        False       False
7542       False  …                  True        False       False
6211       False  …                  True        False       False
```

```
7610       False  …                  True        False       False
6227       False  …                  True        False       False

      flooring_type  energy_source  heating_type  property_condition  \
6386        Unknown        Unknown       Unknown   Neubau, neuwertig
7629        Unknown      Fernwärme       Unknown   Neubau, neuwertig
6378        Unknown        Unknown       Unknown   Neubau, neuwertig
6379        Unknown        Unknown       Unknown   Neubau, neuwertig
7533        Unknown      Fernwärme       Unknown   Neubau, neuwertig
7609        Unknown      Fernwärme       Unknown   Neubau, neuwertig
7532        Unknown      Fernwärme       Unknown   Neubau, neuwertig
5833        Unknown        Unknown       Unknown   Neubau, neuwertig
7542        Unknown      Fernwärme       Unknown   Neubau, neuwertig
6211        Unknown        Unknown       Unknown   Neubau, neuwertig
7610        Unknown      Fernwärme       Unknown   Neubau, neuwertig
6227        Unknown        Unknown       Unknown   Neubau, neuwertig

     year_built schufa_check          scraped_at
6386       2023         True 2026-01-15 21:08:47
7629       2023         True 2026-01-16 19:01:08
6378       2023         True 2026-01-15 21:07:48
6379       2023         True 2026-01-15 21:07:54
7533       2023         True 2026-01-16 18:44:45
7609       2023         True 2026-01-16 18:57:37
7532       2023         True 2026-01-16 18:44:39
5833       2023         True 2026-01-13 15:01:56
7542       2023         True 2026-01-16 18:45:48
6211       2023         True 2026-01-14 22:08:55
7610       2023         True 2026-01-16 18:57:48
6227       2023         True 2026-01-14 22:11:57

[12 rows x 24 columns]
```

[74]:
```
# We keep the samples with that have the 'energy_source' value and drop those␣
↪with 'unknown' values
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"energy_source"]]

to_drop = df[
    df.duplicated(subset=subset, keep=False) &
    (df["energy_source"] == "Unknown")
].index

df = df.drop(index=to_drop, errors="ignore")
```

[75]:
```
print(f"After: {df.shape[0]} samples")
```

```
After: 15902 samples
```

[76]:
```
df[df.drop(columns=["listing_url","scraped_at","heating_type"]).
 ↪duplicated(keep=False)].head(10)
```

[76]:
```
                                          listing_url  cold_rent     area  \
3409   https://www.immowelt.de/expose/43778d73-a328-4…     1149 €  60,5 m²
4310   https://www.immowelt.de/expose/43778d73-a328-4…     1149 €  60,5 m²
11328  https://www.immowelt.de/expose/8404f011-be75-4…    574.8 €  18,8 m²
11327  https://www.immowelt.de/expose/8753be53-a399-4… 1013.24 €    48 m²
12192  https://www.immowelt.de/expose/dfbd4c17-bd71-4…    532.6 €  18,1 m²
11325  https://www.immowelt.de/expose/1bd95b25-231b-4…    460.6 €  23,6 m²
12146  https://www.immowelt.de/expose/73c79502-aa68-4…      530 €  11,8 m²
12987  https://www.immowelt.de/expose/6a211cbc-1803-4…      525 €    11 m²
11320  https://www.immowelt.de/expose/567f9a33-d4d7-4…      917 €    49 m²
5705   https://www.immowelt.de/expose/a821c056-d519-4…      518 €  11,3 m²

          rooms  floor        free_from  \
3409   2 Zimmer    1.0      immediately
4310   2 Zimmer    1.0      immediately
11328  1 Zimmer    3.0  not_immediately
11327  2 Zimmer    3.0  not_immediately
12192  1 Zimmer    3.0  not_immediately
11325  1 Zimmer    3.0  not_immediately
12146  1 Zimmer    3.0  not_immediately
12987  1 Zimmer    3.0  not_immediately
11320  2 Zimmer    3.0  not_immediately
5705   1 Zimmer    1.0  not_immediately

                                                location  has_balkon  \
3409                              Grünau, Berlin (12527)        True
4310                              Grünau, Berlin (12527)        True
11328    Rathenaustraße, Oberschöneweide, Berlin (12459)       False
11327  Allee der Kosmonauten, Friedrichsfelde, Berlin…       False
12192    Rathenaustraße, Oberschöneweide, Berlin (12459)       False
11325    Rathenaustraße, Oberschöneweide, Berlin (12459)       False
12146      Ostendstraße, Oberschöneweide, Berlin (12459)       False
12987      Ostendstraße, Oberschöneweide, Berlin (12459)       False
11320  Lion-Feuchtwanger-Straße, Kaulsdorf, Berlin (1…       False
5705       Ostendstraße, Oberschöneweide, Berlin (12459)       False

       has_terrasse  has_garten  …  has_built-in_kitchen  has_bathtub  \
3409          False       False  …                  True        False
4310          False       False  …                  True        False
11328         False       False  …                  True        False
11327         False       False  …                  True        False
12192         False       False  …                  True        False
11325         False       False  …                  True        False
12146         False       False  …                  True        False
```

```
12987         False       False  …                  True        False
11320         False       False  …                  True        False
5705          False       False  …                  True        False

       has_shower  flooring_type  energy_source  \
3409         True        parkett       Erdwärme
4310         True        parkett       Erdwärme
11328        True        Unknown        Unknown
11327        True        Unknown        Unknown
12192        True        Unknown        Unknown
11325        True        Unknown        Unknown
12146        True        Unknown        Unknown
12987        True        Unknown        Unknown
11320        True        Unknown        Unknown
5705         True        Unknown        Unknown

                         heating_type     property_condition year_built  \
3409                    Zentralheizung  Massivhaus, Erstbezug       2025
4310   Zentralheizung: Fußbodenheizung  Massivhaus, Erstbezug       2025
11328                          Unknown                Unknown       2022
11327                          Unknown                Unknown       2020
12192                          Unknown                Unknown       2022
11325                          Unknown                Unknown       2022
12146                          Unknown                Unknown       2018
12987                    Etagenheizung                Unknown       2018
11320                          Unknown                Unknown       2023
5705                           Unknown                Unknown       2018

      schufa_check          scraped_at
3409          True 2025-12-31 23:37:54
4310          True 2026-01-11 13:05:28
11328         True 2026-01-19 15:09:44
11327         True 2026-01-19 15:09:34
12192         True 2026-01-22 09:57:00
11325         True 2026-01-19 15:09:14
12146         True 2026-01-22 09:50:45
12987         True 2026-01-22 19:03:24
11320         True 2026-01-19 15:08:25
5705          True 2026-01-13 14:36:45

[10 rows x 24 columns]
```

[77]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"heating_type"]]
to_drop = df[
    df.duplicated(subset=subset, keep=False) &
    (df["heating_type"] == "Unknown")
```

```
].index
df = df.drop(index=to_drop, errors="ignore")
```

[78]:
```
df[df.drop(columns=["listing_url", "scraped_at", "property_condition"]).
 ↪duplicated(keep=False)].head(10)
```

[78]:
```
                                          listing_url cold_rent     area  \
5736   https://www.immowelt.de/expose/b1f1fe89-c0b1-4…    1239 €    51 m²
5432   https://www.immowelt.de/expose/b1f1fe89-c0b1-4…    1239 €    51 m²
17058  https://www.immowelt.de/expose/9ee635c2-3f38-4…    1170 €    36 m²
17327  https://www.immowelt.de/expose/53cdcf05-0953-4…    1170 €    36 m²
5718   https://www.immowelt.de/expose/3ffdf53b-a483-4…    1650 €  82,2 m²
11763  https://www.immowelt.de/expose/283e71a5-6467-4…    1700 €    70 m²
13004  https://www.immowelt.de/expose/283e71a5-6467-4…    1700 €    70 m²
5470   https://www.immowelt.de/expose/3ffdf53b-a483-4…    1650 €  82,2 m²
4335   https://www.immowelt.de/expose/f0d38787-0c2f-4…     400 €    36 m²
3095   https://www.immowelt.de/expose/f0d38787-0c2f-4…     400 €    36 m²

          rooms  floor    free_from  \
5736   2 Zimmer    1.0  immediately
5432   2 Zimmer    1.0  immediately
17058  1 Zimmer    5.0  immediately
17327  1 Zimmer    5.0  immediately
5718   3 Zimmer    1.0  immediately
11763  2 Zimmer    3.0  immediately
13004  2 Zimmer    3.0  immediately
5470   3 Zimmer    1.0  immediately
4335   1 Zimmer    1.0      unknown
3095   1 Zimmer    1.0      unknown

                                                location  has_balkon  \
5736                            Lankwitz, Berlin (12247)       False
5432                            Lankwitz, Berlin (12247)       False
17058  Kurfürstendamm 167-168, Wilmersdorf, Berlin (1…       False
17327  Kurfürstendamm 167-168, Wilmersdorf, Berlin (1…       False
5718   Frankfurter Allee 55 A, Friedrichshain, Berlin…        True
11763          Niemetzstraße 2, Neukölln, Berlin (12055)        True
13004          Niemetzstraße 2, Neukölln, Berlin (12055)        True
5470   Frankfurter Allee 55 A, Friedrichshain, Berlin…        True
4335                             Spandau, Berlin (13627)        True
3095                             Spandau, Berlin (13627)        True

       has_terrasse  has_garten  …  has_built-in_kitchen  has_bathtub  \
5736           True        True  …                 False        False
5432           True        True  …                 False        False
17058         False       False  …                 False        False
17327         False       False  …                 False        False
```

```
5718          False        True  …                  True         True
11763         False       False  …                  True        False
13004         False       False  …                  True        False
5470          False        True  …                  True         True
4335           True       False  …                  True         True
3095           True       False  …                  True         True

       has_shower  flooring_type                            energy_source  \
5736         True        parkett  Elektro, Solar, Luft-/Wasser-Wärmepumpe
5432         True        parkett  Elektro, Solar, Luft-/Wasser-Wärmepumpe
17058        True        parkett                                Fernwärme
17327        True        parkett                                Fernwärme
5718        False        parkett                                Fernwärme
11763       False        parkett                                      Gas
13004       False        parkett                                      Gas
5470        False        parkett                                Fernwärme
4335        False     kunststoff                                  Unknown
3095        False     kunststoff                                  Unknown

                        heating_type  \
5736   Zentralheizung: Fußbodenheizung
5432   Zentralheizung: Fußbodenheizung
17058                  Zentralheizung
17327                  Zentralheizung
5718   Zentralheizung: Fußbodenheizung
11763                 Fußbodenheizung
13004                 Fußbodenheizung
5470   Zentralheizung: Fußbodenheizung
4335                           Unknown
3095                           Unknown

                                  property_condition year_built  \
5736                                         Unknown       2025
5432                       Neubau, renoviert / saniert       2025
17058           Altbau (bis 1945), renoviert / saniert       1900
17327                                        Unknown       1900
5718                                         Unknown       1905
11763                                        Unknown       1900
13004                                       Gepflegt       1900
5470   Altbau (bis 1945), Saniert, renoviert / saniert       1905
4335                               renoviert / saniert       <NA>
3095                                         Unknown       <NA>

      schufa_check          scraped_at
5736          True 2026-01-13 14:42:55
5432          True 2026-01-11 16:47:57
17058         True 2026-01-25 11:36:16
```

```
17327          True 2026-01-26 09:26:44
5718           True 2026-01-13 14:39:30
11763          True 2026-01-20 21:35:30
13004          True 2026-01-22 19:06:51
5470           True 2026-01-11 16:55:16
4335           True 2026-01-11 13:10:02
3095           True 2025-12-31 22:29:30

[10 rows x 24 columns]
```

[79]:
```
subset = [c for c in df.columns if c not in ["listing_url", "scraped_at",␣
↪"property_condition"]]

df["scraped_at"] = pd.to_datetime(df["scraped_at"], errors="coerce")

df = (
    df
    # prefer non-Unknown
    .assign(_known=lambda x: x["property_condition"].ne("Unknown"))
    # sort so best row comes first within each duplicate group:
    # 1) known property_condition (True first)
    # 2) most recent scraped_at (newest first)
    .sort_values(["_known", "scraped_at"], ascending=[False, False],␣
↪na_position="last")
    # keep one per group
    .drop_duplicates(subset=subset, keep="first")
    .drop(columns=["_known"])
)

print(f"After: {df.shape[0]} samples")
```

```
After: 15808 samples
```

[80]:
```
df[df.drop(columns=["listing_url", "scraped_at", "year_built"]).
↪duplicated(keep=False)]
```

[80]:
```
                                          listing_url cold_rent     area  \
7989  https://www.immowelt.de/expose/2554ujfn17u6?se…    1090 €  77,6 m²
5919  https://www.immowelt.de/expose/2554ujfn17u6?se…    1090 €  77,6 m²
7617  https://www.immowelt.de/expose/fe0d7373-7586-4…     360 €    93 m²
7539  https://www.immowelt.de/expose/043525f5-c584-4…    1287 €    48 m²
7531  https://www.immowelt.de/expose/b102b2ac-c8da-4…   526.4 €  18,4 m²
6374  https://www.immowelt.de/expose/11c74691-d14e-4…    1200 €  49,3 m²
6039  https://www.immowelt.de/expose/d395cd36-eae0-4…    1200 €  49,3 m²
6029  https://www.immowelt.de/expose/936dbfc8-a050-4…     360 €    93 m²
6012  https://www.immowelt.de/expose/1a7ea234-41d5-4…    1287 €    48 m²
6011  https://www.immowelt.de/expose/18f98470-b59d-4…   526.4 €  18,4 m²
```

```
         rooms  floor        free_from  \
7989  2 Zimmer    2.0  not_immediately
5919  2 Zimmer    2.0  not_immediately
7617  1 Zimmer    2.0  not_immediately
7539  2 Zimmer    2.0  not_immediately
7531  1 Zimmer    2.0  not_immediately
6374  2 Zimmer    2.0  not_immediately
6039  2 Zimmer    2.0  not_immediately
6029  1 Zimmer    2.0  not_immediately
6012  2 Zimmer    2.0  not_immediately
6011  1 Zimmer    2.0  not_immediately

                                          location  has_balkon  \
7989     Leobschützer Straße 60, Karow, Berlin (13125)        True
5919     Leobschützer Straße 60, Karow, Berlin (13125)        True
7617           Bahnhofstraße, Köpenick, Berlin (12555)       False
7539  Allee der Kosmonauten, Friedrichsfelde, Berlin…       False
7531    Rathenaustraße, Oberschöneweide, Berlin (12459)       False
6374        Am Köllnischen Park, Mitte, Berlin (10179)       False
6039        Am Köllnischen Park, Mitte, Berlin (10179)       False
6029           Bahnhofstraße, Köpenick, Berlin (12555)       False
6012  Allee der Kosmonauten, Friedrichsfelde, Berlin…       False
6011    Rathenaustraße, Oberschöneweide, Berlin (12459)       False

      has_terrasse  has_garten  …  has_built-in_kitchen  has_bathtub  \
7989          True       False  …                  True        False
5919          True       False  …                  True        False
7617         False       False  …                  True         True
7539         False       False  …                  True        False
7531         False       False  …                  True        False
6374         False       False  …                  True        False
6039         False       False  …                  True        False
6029         False       False  …                  True         True
6012         False       False  …                  True        False
6011         False       False  …                  True        False

      has_shower  flooring_type  energy_source    heating_type  \
7989       False        Unknown            Gas  Zentralheizung
5919       False        Unknown            Gas  Zentralheizung
7617        True        Unknown        Unknown         Unknown
7539        True        Unknown        Unknown         Unknown
7531        True        Unknown        Unknown         Unknown
6374        True        Unknown        Unknown         Unknown
6039        True        Unknown        Unknown         Unknown
6029        True        Unknown        Unknown         Unknown
6012        True        Unknown        Unknown         Unknown
```

```
6011        True        Unknown        Unknown         Unknown

      property_condition year_built schufa_check          scraped_at
7989           neuwertig       1996         True 2026-01-16 20:07:31
5919           neuwertig       2025         True 2026-01-13 15:18:39
7617             Unknown       2026         True 2026-01-16 18:58:56
7539             Unknown       2020         True 2026-01-16 18:45:25
7531             Unknown       2022         True 2026-01-16 18:44:32
6374             Unknown       2021         True 2026-01-15 21:07:01
6039             Unknown       <NA>         True 2026-01-14 21:38:55
6029             Unknown       <NA>         True 2026-01-14 21:37:12
6012             Unknown       <NA>         True 2026-01-14 21:34:11
6011             Unknown       <NA>         True 2026-01-14 21:34:02

[10 rows x 24 columns]
```

[81]:
```python
df[df.duplicated(subset=[c for c in df.columns if c not in ["listing_url",␣
↪"scraped_at", "year_built"]], keep=False) & df["year_built"].isna()]
```

[81]:
```
                                          listing_url cold_rent     area  \
6039  https://www.immowelt.de/expose/d395cd36-eae0-4…    1200 €  49,3 m²
6029  https://www.immowelt.de/expose/936dbfc8-a050-4…     360 €    93 m²
6012  https://www.immowelt.de/expose/1a7ea234-41d5-4…    1287 €    48 m²
6011  https://www.immowelt.de/expose/18f98470-b59d-4…   526.4 €  18,4 m²

         rooms  floor        free_from  \
6039  2 Zimmer    2.0  not_immediately
6029  1 Zimmer    2.0  not_immediately
6012  2 Zimmer    2.0  not_immediately
6011  1 Zimmer    2.0  not_immediately

                                          location  has_balkon  \
6039         Am Köllnischen Park, Mitte, Berlin (10179)       False
6029            Bahnhofstraße, Köpenick, Berlin (12555)       False
6012  Allee der Kosmonauten, Friedrichsfelde, Berlin…       False
6011    Rathenaustraße, Oberschöneweide, Berlin (12459)       False

      has_terrasse  has_garten  …  has_built-in_kitchen  has_bathtub  \
6039         False       False  …                  True        False
6029         False       False  …                  True         True
6012         False       False  …                  True        False
6011         False       False  …                  True        False

      has_shower  flooring_type  energy_source  heating_type  \
6039        True        Unknown        Unknown       Unknown
6029        True        Unknown        Unknown       Unknown
6012        True        Unknown        Unknown       Unknown
```

```
6011        True        Unknown        Unknown       Unknown

      property_condition year_built schufa_check          scraped_at
6039             Unknown       <NA>         True 2026-01-14 21:38:55
6029             Unknown       <NA>         True 2026-01-14 21:37:12
6012             Unknown       <NA>         True 2026-01-14 21:34:11
6011             Unknown       <NA>         True 2026-01-14 21:34:02

[4 rows x 24 columns]
```

[82]:
```
# We drop them those 4 with NA, but we would be left with 1 duplicate since␣
↪there were 10
df = df.drop(index=df[df.duplicated(subset=[c for c in df.columns if c not in␣
↪["listing_url", "scraped_at", "year_built"]], keep=False) & df["year_built"].
↪isna()].index)
```

[83]:
```
df[df.drop(columns=["listing_url", "scraped_at", "year_built"]).
↪duplicated(keep=False)]
```

[83]:
```
                                          listing_url cold_rent     area  \
7989  https://www.immowelt.de/expose/2554ujfn17u6?se…    1090 €  77,6 m²
5919  https://www.immowelt.de/expose/2554ujfn17u6?se…    1090 €  77,6 m²

         rooms  floor        free_from  \
7989  2 Zimmer    2.0  not_immediately
5919  2 Zimmer    2.0  not_immediately

                                         location  has_balkon  has_terrasse  \
7989  Leobschützer Straße 60, Karow, Berlin (13125)        True          True
5919  Leobschützer Straße 60, Karow, Berlin (13125)        True          True

      has_garten  …  has_built-in_kitchen  has_bathtub  has_shower  \
7989       False  …                  True        False       False
5919       False  …                  True        False       False

      flooring_type  energy_source    heating_type  property_condition  \
7989        Unknown            Gas  Zentralheizung           neuwertig
5919        Unknown            Gas  Zentralheizung           neuwertig

     year_built schufa_check          scraped_at
7989       1996         True 2026-01-16 20:07:31
5919       2025         True 2026-01-13 15:18:39

[2 rows x 24 columns]
```

[84]:
```
# We keep the one with the original year (Which also happens to be the newer␣
↪listing)
```

```
df = df.drop(index=[5919])
```

[85]:
```
df.shape
```

[85]: (15803, 24)

During deduplication we dropped from 20932 to 15803 22687 ==> 20932 ==> 15803

[86]:
```
(1-(15803/22687))*100
```

[86]: 30.34336844889144

We lost 30.34% of our dataset

[87]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 15803 entries, 22653 to 3
Data columns (total 24 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   listing_url           15803 non-null  object
 1   cold_rent             15803 non-null  object
 2   area                  15803 non-null  object
 3   rooms                 15803 non-null  object
 4   floor                 13832 non-null  float64
 5   free_from             15803 non-null  object
 6   location              15803 non-null  object
 7   has_balkon            15803 non-null  boolean
 8   has_terrasse          15803 non-null  boolean
 9   has_garten            15803 non-null  boolean
 10  elevator              15803 non-null  boolean
 11  parking               15803 non-null  boolean
 12  has_basement          15803 non-null  boolean
 13  is_barrier-free       15803 non-null  boolean
 14  has_built-in_kitchen  15803 non-null  boolean
 15  has_bathtub           15803 non-null  boolean
 16  has_shower            15803 non-null  boolean
 17  flooring_type         15803 non-null  object
 18  energy_source         15803 non-null  object
 19  heating_type          15803 non-null  object
 20  property_condition    15803 non-null  object
 21  year_built            7839 non-null   Int64
 22  schufa_check          15803 non-null  bool
 23  scraped_at            15803 non-null  datetime64[ns]
dtypes: Int64(1), bool(1), boolean(10), datetime64[ns](1), float64(1),
object(10)
memory usage: 2.0+ MB
```

Only the year built and the floor type have missing values

Part 3: Handling inconsistent formats

1. Formatting 'cold rent' feature

[88]:
```
df["cold_rent"].sample(5)
```

[88]:
```
4849      980 €
21943    2660 €
8533     1100 €
7367     2500 €
2614     1200 €
Name: cold_rent, dtype: object
```

[89]:
```
# check if the there are any irregular values (values other than 'number +
↪currency' or 'number' format)
pattern = r'^\s*(?:\d{1,3}(?:[.,]\d{3})+|\d+)(?:[.,]\d+)?\s*€?\s*$'
mask = df['cold_rent'].astype(str).str.match(pattern)
df_invalid = df[~mask]
df_invalid['cold_rent']
```

[89]:
```
Series([], Name: cold_rent, dtype: object)
```

[90]:
```
# Let's also check if any of the values have comma in case we need to replace
↪it with a period.
mask = df['cold_rent'].astype(str).str.contains(',')
df_invalid = df[mask]
df_invalid['cold_rent'].sample(5)
```

[90]:
```
4        1,399 €
20    1,586.85 €
0        1,499 €
14       1,095 €
8        1,330 €
Name: cold_rent, dtype: object
```

[91]:
```
# Let's format the column so it contains only the numerical values
s = (df['cold_rent'].astype(str)
     .str.strip()
     .str.replace(r'[€\s]', '', regex=True))

# Sampled values had US format, but it is reccomended to ensure to treat both
us = s.str.contains(r',\d{3}(\D|$)') & s.str.contains(r'\.\d+$')   # 1,234.56
eu = s.str.contains(r'\.\d{3}(\D|$)') & s.str.contains(r',\d+$')   # 1.234,56

out = pd.Series(index=s.index, dtype='float64')

# US: remove thousands commas
```

```
out[us] = pd.to_numeric(s[us].str.replace(',', '', regex=False),␣
 ↪errors='coerce')

# EU: remove thousands dots, swap decimal comma -> dot
out[eu] = pd.to_numeric(
    s[eu].str.replace('.', '', regex=False).str.replace(',', '.', regex=False),
    errors='coerce'
)

# Plain: no thousands separators, maybe dot-decimal
plain = ~(us | eu)
out[plain] = pd.to_numeric(s[plain].str.replace(',', '.', regex=False),␣
 ↪errors='coerce')

df['cold_rent'] = out
```

```
C:\Users\Perdorues\AppData\Local\Temp\ipykernel_18060\2430145821.py:7:
UserWarning: This pattern is interpreted as a regular expression, and has match
groups. To actually get the groups, use str.extract.
  us = s.str.contains(r',\d{3}(\D|$)') & s.str.contains(r'\.\d+$')   # 1,234.56
C:\Users\Perdorues\AppData\Local\Temp\ipykernel_18060\2430145821.py:8:
UserWarning: This pattern is interpreted as a regular expression, and has match
groups. To actually get the groups, use str.extract.
  eu = s.str.contains(r'\.\d{3}(\D|$)') & s.str.contains(r',\d+$')   # 1.234,56
```

[92]:
```
df["cold_rent"].sample(10)
```

[92]:
```
11443    1080.00
10581    1500.00
14344     715.00
19957     945.00
22153     571.07
22193    1935.00
780      1320.00
20584    2315.94
20671    1145.00
19011    1499.00
Name: cold_rent, dtype: float64
```

[93]:
```
df['cold_rent'].describe()
```

[93]:
```
count    15803.000000
mean      1313.803160
std       1188.053211
min          1.095000
25%        678.000000
50%       1100.000000
75%       1642.000000
```

```
max      35000.000000
Name: cold_rent, dtype: float64
```

The values seem reasonable, the outliers are expected, but nothing very extreme and unexpected. We will deal with outliers in the later stages.

2. Formatting 'area' feature

[94]:
```
df["area"].sample(5)
```

[94]:
```
7392     10,2 m²
22583      57 m²
18466    48,5 m²
5        30,6 m²
7367       90 m²
Name: area, dtype: object
```

[95]:
```
pattern = r'^\s*(?:\d{1,3}(?:[.,]\d{3})+|\d+)(?:[.,]\d+)?\s*m(?:²|2)\s*$'
mask = df['area'].astype(str).str.match(pattern)
df_invalid = df[~mask]
df_invalid['area'].sample(5)
```

[95]:
```
15677     400 m² Grundstück
22018     500 m² Grundstück
13356     161 m² Grundstück
18795    93,9 m² Grundstück
14646     310 m² Grundstück
Name: area, dtype: object
```

[96]:
```
# For some reason they have large values and I decided to also sample all the␣
↪records instead of only the values
df_invalid.sample(5)
```

[96]:
```
                                            listing_url  cold_rent  \
5477   https://www.immowelt.de/expose/ff9ec6cf-77cb-4…    2484.18
13080  https://www.immowelt.de/expose/51bd788e-3911-4…    2890.00
3101   https://www.immowelt.de/expose/63b7fdd9-a3e6-4…    1784.10
8742   https://www.immowelt.de/expose/63a83697-84ab-4…    2500.00
8477   https://www.immowelt.de/expose/23214a55-1e46-4…   23520.00

                     area      rooms  floor        free_from  \
5477    230 m² Grundstück   5 Zimmer    1.0      immediately
13080   483 m² Grundstück   5 Zimmer    3.0  not_immediately
3101   93,9 m² Grundstück   3 Zimmer    1.0      immediately
8742    310 m² Grundstück   5 Zimmer    2.0  not_immediately
8477    580 m² Grundstück  35 Zimmer    2.0      immediately

                                          location  has_balkon  has_terrasse  \
```

```
5477   Dorfstraße 2, Wartenberg, Berlin (13059)       False         False
13080         Niederschönhausen, Berlin (13156)       False          True
3101    Dorfplatz 8J, Bohnsdorf, Berlin (12526)        True          True
8742                  Kaulsdorf, Berlin (12621)       False          True
8477                    Spandau, Berlin (13597)        True          True

       has_garten  …  has_built-in_kitchen  has_bathtub  has_shower  \
5477        False  …                 False        False       False
13080        True  …                  True         True       False
3101        False  …                  True         True        True
8742         True  …                  True         True        True
8477         True  …                  True        False       False

          flooring_type                 energy_source  \
5477            Unknown       Gas, Blockheizkraftwerk
13080           Unknown                           Gas
3101   fliesen, parkett       Luft-/Wasser-Wärmepumpe
8742   fliesen, laminat       Luft-/Wasser-Wärmepumpe
8477            Unknown  Gas, Luft-/Wasser-Wärmepumpe

                         heating_type     property_condition year_built  \
5477                   Fußbodenheizung     Neubau, Massivhaus       2025
13080                   Zentralheizung               Gepflegt       2013
3101                           Unknown              Erstbezug       2024
8742                   Fußbodenheizung              Erstbezug       2025
8477   Zentralheizung: Fußbodenheizung  Massivhaus, Erstbezug       2025

      schufa_check          scraped_at
5477          True 2026-01-11 16:56:41
13080         True 2026-01-22 19:20:10
3101          True 2025-12-31 22:31:12
8742          True 2026-01-16 21:55:15
8477          True 2026-01-16 21:22:04

[5 rows x 24 columns]
```

The prices justify the big values of area. It just happend to be that for large areas the formatting doesn't match because of additional words like 'Grundstück'.

[97]:
```
mask = df['area'].astype(str).str.contains(',')
df_invalid = df[mask]
df_invalid['area'].sample(5)
```

[97]:
```
16398    112,3 m²
5365      99,9 m²
19523     26,7 m²
22118     50,8 m²
22084     50,5 m²
```

```
Name: area, dtype: object
```

[98]:
```
mask = df['area'].astype(str).str.contains(r'\.')
df_invalid = df[mask]
df_invalid['area'].sample(5)
```

[98]:
```
10283    1.151 m² Grundstück
16468    2.100 m² Grundstück
22261    2.100 m² Grundstück
8892     1.151 m² Grundstück
13214    3.750 m² Grundstück
Name: area, dtype: object
```

[99]:
```
s = df['area'].astype(str).str.strip()

# Keep only the numeric part before the unit or before text.
s = s.str.replace(r'\s*(m²|m2)\b.*$', '', regex=True)

# Extract the first "number-like" token
num = s.str.extract(r'(\d+(?:[.,]\d+)*)')[0]

# Decide if it's "thousands grouping" like 3.750 or 1,234,567
# (one or more groups of exactly 3 digits after separators)
is_thousands = num.str.match(r'^\d{1,3}(?:[.,]\d{3})+$', na=False)

out = pd.Series(index=df.index, dtype='float64')

# Thousands-style: remove separators entirely -> integer
out[is_thousands] = pd.to_numeric(
    num[is_thousands].str.replace(r'[.,]', '', regex=True),
    errors='coerce'
)

# Non-thousands-style: treat comma as decimal, keep dot as decimal
out[~is_thousands] = pd.to_numeric(
    num[~is_thousands].str.replace(',', '.', regex=False),
    errors='coerce'
)

out.sample(5)
```

[99]:
```
18130     70.2
457      123.0
14494     75.2
1488      44.0
11415     21.0
dtype: float64
```

[100]:
```
# Format values look good so we save the changes.
df['area'] = out
df['area'].describe()
```

[100]:
```
count    15803.000000
mean        82.181301
std        141.620615
min          1.000000
25%         50.800000
50%         66.000000
75%         87.000000
max       5251.000000
Name: area, dtype: float64
```

No matter how careful we are with the formatting, outliers will always be present. We cannot avoid them.

3. Formatting 'rooms' feature

[101]:
```
df["rooms"].unique()
```

[101]:
```
array(['2 Zimmer', '1 Zimmer', '4 Zimmer', '5 Zimmer', '3 Zimmer',
       '1,5 Zimmer', '3,5 Zimmer', '8 Zimmer', '7 Zimmer', '2,5 Zimmer',
       '6 Zimmer', '5,5 Zimmer', '4,5 Zimmer', '35 Zimmer', '2,2 Zimmer',
       '6,5 Zimmer', '2,9 Zimmer', '10 Zimmer', '22 Zimmer'], dtype=object)
```

Most values seem legitimate, expect the decimal ones. First let's remove the 'Zimmer' part and then have a detailed look into the unusual values with decimeters to get more information about them, in order to decide how to deal with them.

[102]:
```
df['rooms'] = df['rooms'].astype(str).str.strip().replace(r'\s*(Zimmer)\b.*$',␣
 ↪'', regex=True)

# and convert to floating numbers
df["rooms"] = (
    df["rooms"]
        .str.replace(",", ".", regex=False)   # turn 3,5 → 3.5
        .astype(float)

)

df["rooms"].unique()
```

[102]:
```
array([ 2. ,  1. ,  4. ,  5. ,  3. ,  1.5,  3.5,  8. ,  7. ,  2.5,  6. ,
        5.5,  4.5, 35. ,  2.2,  6.5,  2.9, 10. , 22. ])
```

[103]:
```
mask = df['rooms'] % 1 != 0
df_invalid = df[mask]
df_invalid.sample(5)
```

[103]:

```
                                          listing_url  cold_rent  area  \
18771  https://www.immowelt.de/expose/db6101f2-99a9-4…     1100.0  98.0
22611  https://www.immowelt.de/expose/4cc19695-6732-4…      891.0  99.0
8141   https://www.immowelt.de/expose/5bbbcd0a-393c-4…      395.0  40.0
8927   https://www.immowelt.de/expose/e877e1be-c4fd-4…      821.0  73.0
22007  https://www.immowelt.de/expose/6a01cfb1-c95b-4…      890.0  70.0

       rooms  floor free_from                        location  has_balkon  \
18771    2.5    7.0   unknown           Pankow, Berlin (13187)        True
22611    3.5    NaN   unknown       Zehlendorf, Berlin (12247)        True
8141     1.5    2.0   unknown            Mitte, Berlin (13353)       False
8927     2.5    2.0   unknown       Karlshorst, Berlin (10318)       False
22007    2.5    NaN   unknown  Prenzlauer Berg, Berlin (10405)       False

       has_terrasse  has_garten  …  has_built-in_kitchen  has_bathtub  \
18771          True       False  …                  True         True
22611          True       False  …                  True         True
8141          False       False  …                  True        False
8927          False       False  …                  True         True
22007         False       False  …                  True         True

       has_shower                    flooring_type  energy_source  \
18771       False                           parkett        Unknown
22611       False                           Unknown        Unknown
8141        False  fliesen, holzdielen, kunststoff        Unknown
8927        False                       holzdielen        Unknown
22007       False                 fliesen, laminat        Unknown

         heating_type  property_condition year_built schufa_check  \
18771         Unknown             Unknown       <NA>         True
22611         Unknown             Unknown       <NA>         True
8141   Zentralheizung             Unknown       <NA>         True
8927   Zentralheizung             Unknown       <NA>         True
22007  Zentralheizung             Unknown       <NA>         True

               scraped_at
18771 2026-01-27 21:33:40
22611 2026-01-31 17:47:52
8141  2026-01-16 20:29:49
8927  2026-01-16 22:12:56
22007 2026-01-31 16:01:55

[5 rows x 24 columns]
```

The values seem legitimate. Adding 0.5 values they might advertise small extra spaces. These definitely have their effect on the price so rounding the values up or down would alter the real information. I would still like to have a closer look to these to specific values '2,9' and '2.2', in case they meant 29 rooms and 22 rooms.

[104]:
```
df[(df["rooms"] == 2.2) | (df["rooms"] == 2.9)]
```

[104]:
```
                                          listing_url  cold_rent  area  \
21434  https://www.immowelt.de/expose/9d2b7253-4684-4…     1334.1  95.3
19323  https://www.immowelt.de/expose/9d2b7253-4684-4…     1334.1  95.3
19087  https://www.immowelt.de/expose/73f72612-5c96-4…     1799.0  94.0
11593  https://www.immowelt.de/expose/73f72612-5c96-4…     1799.0  94.0
7398   https://www.immowelt.de/expose/73f72612-5c96-4…     1899.0  94.0
5460   https://www.immowelt.de/expose/73f72612-5c96-4…     1899.0  94.0
141    https://www.immowelt.de/expose/73f72612-5c96-4…     1899.0  94.0

       rooms  floor        free_from  \
21434    2.2    NaN  not_immediately
19323    2.2   10.0  not_immediately
19087    2.9    8.0  not_immediately
11593    2.9    3.0  not_immediately
7398     2.9    2.0  not_immediately
5460     2.9    1.0  not_immediately
141      2.9    0.0  not_immediately

                                            location  has_balkon  \
21434    Reginhardstr. 32, Gesundbrunnen, Berlin (13409)        True
19323    Reginhardstr. 32, Gesundbrunnen, Berlin (13409)        True
19087  Mahlsdorferstr 7-8, Köpenick, Berlin Köpenick …        True
11593  Mahlsdorferstr 7-8, Köpenick, Berlin Köpenick …        True
7398   Mahlsdorferstr 7-8, Köpenick, Berlin Köpenick …        True
5460   Mahlsdorferstr 7-8, Köpenick, Berlin Köpenick …        True
141    Mahlsdorferstr 7-8, Köpenick, Berlin Köpenick …        True

       has_terrasse  has_garten  …  has_built-in_kitchen  has_bathtub  \
21434         False       False  …                  False         True
19323         False       False  …                  False         True
19087         False       False  …                  False         True
11593         False       False  …                  False         True
7398          False       False  …                  False         True
5460          False       False  …                  False         True
141           False       False  …                  False         True

       has_shower     flooring_type  energy_source    heating_type  \
21434       False           laminat            Gas         Unknown
19323       False           laminat            Gas         Unknown
19087        True  fliesen, parkett        Unknown  Zentralheizung
11593        True           Unknown        Unknown  Zentralheizung
7398         True           Unknown        Unknown  Zentralheizung
5460         True  fliesen, parkett        Unknown  Zentralheizung
141          True  fliesen, parkett        Unknown  Zentralheizung
```

```
       property_condition year_built schufa_check          scraped_at
21434            Gepflegt       1977         True 2026-01-31 13:53:28
19323            Gepflegt       1977         True 2026-01-30 17:28:10
19087           Erstbezug       2025         True 2026-01-30 16:35:07
11593           Erstbezug       2025         True 2026-01-20 21:00:32
7398            Erstbezug       2025         True 2026-01-16 18:22:47
5460            Erstbezug       2025         True 2026-01-11 16:53:20
141             Erstbezug       2025         True 2025-12-28 22:04:18

[7 rows x 24 columns]
```

[105]:
```
# It seems like they made some types; Wanting to write 2,0 instead of 2,2 and␣
↪2,0 instead of 2,9.
# So we just round them to 2.
```

[106]:
```
df.loc[(df["rooms"] == 2.2) | (df["rooms"] == 2.9),"rooms"] = 2
```

[107]:
```
df["rooms"].unique()
```

[107]:
```
array([ 2. ,  1. ,  4. ,  5. ,  3. ,  1.5,  3.5,  8. ,  7. ,  2.5,  6. ,
        5.5,  4.5, 35. ,  6.5, 10. , 22. ])
```

4. Formatting 'flooring_type' feature

[108]:
```
df["flooring_type"].unique()
```

[108]:
```
array(['Unknown', 'fliesen, parkett', 'fliesen', 'fliesen, laminat',
       'fliesen, holzdielen', 'holzdielen',
       'fliesen, parkett, kunststoff', 'laminat, kunststoff, teppich',
       'parkett', 'laminat', 'laminat, kunststoff', 'linoleum',
       'parkett, laminat', 'stein', 'kunststoff',
       'fliesen, holzdielen, parkett', 'fliesen, kunststoff',
       'holzdielen, parkett, stein',
       'fliesen, parkett, kunststoff, teppich',
       'fliesen, holzdielen, parkett, stein', 'fliesen, parkett, laminat',
       'parkett, stein', 'fliesen, parkett, laminat, teppich',
       'holzdielen, parkett', 'fliesen, laminat, kunststoff',
       'fliesen, parkett, stein', 'fliesen, parkett, teppich', 'estrich',
       'fliesen, holzdielen, kunststoff', 'fliesen, marmor, parkett',
       'holzdielen, laminat', 'fliesen, estrich, granit',
       'marmor, parkett', 'parkett, doppelboden',
       'fliesen, laminat, stein, teppich', 'fliesen, linoleum, teppich',
       'laminat, kunststoff, linoleum', 'kunststoff, linoleum',
       'fliesen, parkett, granit', 'holzdielen, stein',
       'parkett, laminat, stein', 'parkett, kunststoff',
       'fliesen, laminat, stein', 'holzdielen, parkett, laminat',
       'parkett, laminat, kunststoff', 'teppich',
       'fliesen, holzdielen, laminat',
```

```
       'fliesen, holzdielen, parkett, laminat, kunststoff',
       'holzdielen, kunststoff', 'marmor, holzdielen, stein',
       'holzdielen, terrakotta', 'fliesen, holzdielen, terrakotta',
       'fliesen, laminat, teppich',
       'fliesen, holzdielen, parkett, kunststoff',
       'fliesen, holzdielen, laminat, kunststoff',
       'laminat, kunststoff, stein',
       'fliesen, laminat, kunststoff, stein', 'fliesen, linoleum',
       'fliesen, parkett, laminat, kunststoff', 'holzdielen, teppich',
       'fliesen, teppich', 'laminat, stein', 'kunststoff, stein',
       'fliesen, holzdielen, kunststoff, linoleum', 'marmor, laminat',
       'marmor, holzdielen', 'holzdielen, parkett, terrakotta',
       'parkett, terrakotta', 'holzdielen, laminat, stein',
       'fliesen, laminat, terrakotta'], dtype=object)
```

We have so many types of values and some of them repeat under slightly different names. Also, we want them in english. We will try to map them in as little categories as possible, while trying not to lose to much information. We decided to map them into 8 pure categories since some of them have more we will end up having more than 8 categories, but that is not avoidable without losing information.

[109]:
```
floor_mapping = {
    # Tiles
    "fliesen": "Tiles",

    # Wood
    "parkett": "Wood",
    "holzdielen": "Wood",
    "doppelboden": "Wood",

    # Laminate
    "laminat": "Laminate",

    # Vinyl / plastic family
    "kunststoff": "Vinyl",
    "linoleum": "Vinyl",

    # Carpet
    "teppich": "Carpet",

    # Stone family
    "stein": "Stone",
    "marmor": "Stone",
    "granit": "Stone",
    "terrakotta": "Stone",

    # Concrete/raw
```

```
    "estrich": "Concrete",

    # Unknown
    "Unknown": "Unknown"
}
```

[110]:
```
def map_flooring(value):
    if value == "Unknown":
        return "unknown"
    types = [t.strip() for t in value.split(",")]
    mapped = sorted(set(floor_mapping.get(t, t) for t in types))
    return ", ".join(mapped)

df["flooring_type"] = df["flooring_type"].apply(map_flooring)
```

[111]:
```
df["flooring_type"].unique()
```

[111]:
```
array(['unknown', 'Tiles, Wood', 'Tiles', 'Laminate, Tiles', 'Wood',
       'Tiles, Vinyl, Wood', 'Carpet, Laminate, Vinyl', 'Laminate',
       'Laminate, Vinyl', 'Vinyl', 'Laminate, Wood', 'Stone',
       'Tiles, Vinyl', 'Stone, Wood', 'Carpet, Tiles, Vinyl, Wood',
       'Stone, Tiles, Wood', 'Laminate, Tiles, Wood',
       'Carpet, Laminate, Tiles, Wood', 'Laminate, Tiles, Vinyl',
       'Carpet, Tiles, Wood', 'Concrete', 'Concrete, Stone, Tiles',
       'Carpet, Laminate, Stone, Tiles', 'Carpet, Tiles, Vinyl',
       'Laminate, Stone, Wood', 'Vinyl, Wood', 'Laminate, Stone, Tiles',
       'Laminate, Vinyl, Wood', 'Carpet', 'Laminate, Tiles, Vinyl, Wood',
       'Carpet, Laminate, Tiles', 'Laminate, Stone, Vinyl',
       'Laminate, Stone, Tiles, Vinyl', 'Carpet, Wood', 'Carpet, Tiles',
       'Laminate, Stone', 'Stone, Vinyl'], dtype=object)
```

[112]:
```
len(df["flooring_type"].unique())
```

[112]: 37

When hot-encoding the feature 38 categories would be an overkill for our models. Let's try to reduce it more. That is why we will follow a slightly different approach, where we rank the types of flooring from the most valuable to the least valuable in order to keep the most valuable type of flooring when there are multiple types of flooring listed.

[113]:
```
floor_rank = {
    "Stone": 5,
    "Wood": 4,
    "Tiles": 3,
    "Laminate": 2,
    "Vinyl": 2,
    "Carpet": 1,
    "Concrete": 0,
```

```
    "unknown": -1
}
```

[114]:
```
def pick_most_expensive_floor(value):
    if pd.isna(value):
        return "unknown"

    materials = [v.strip() for v in value.split(",")]

    best_material = "unknown"
    best_score = -1

    for m in materials:
        score = floor_rank.get(m, -1)
        if score > best_score:
            best_score = score
            best_material = m

    return best_material
df["flooring_type"] = df["flooring_type"].apply(pick_most_expensive_floor)
```

[115]:
```
df["flooring_type"].unique()
```

[115]:
```
array(['unknown', 'Wood', 'Tiles', 'Laminate', 'Vinyl', 'Stone',
       'Concrete', 'Carpet'], dtype=object)
```

Now we have a reasonable number of categories.

6. Formatting 'energy_source' feature

[116]:
```
df["energy_source"].unique()
```

[116]:
```
array(['Gas', 'Fernwärme', 'Gas, Solar, Luft-/Wasser-Wärmepumpe',
       'fossile Brennstoffe', 'Unknown', 'Luft-/Wasser-Wärmepumpe',
       'Erdwärme', 'Elektro, Luft-/Wasser-Wärmepumpe', 'Öl',
       'Gas, Blockheizkraftwerk',
       'Elektro, Solar, Luft-/Wasser-Wärmepumpe', 'Elektro, Gas',
       'Solar, Blockheizkraftwerk', 'Fernwärme, Gas', 'Gas, Solar',
       'Fernwärme, Öl', 'Elektro', 'Solar, Luft-/Wasser-Wärmepumpe',
       'Elektro, Solar', 'Gas, Luft-/Wasser-Wärmepumpe', 'Elektro, Öl',
       'Fernwärme, Blockheizkraftwerk',
       'Erdwärme, Solar, Luft-/Wasser-Wärmepumpe',
       'Fernwärme, Luft-/Wasser-Wärmepumpe', 'Gas, fossile Brennstoffe',
       'Erdwärme, Luft-/Wasser-Wärmepumpe', 'Blockheizkraftwerk',
       'Fernwärme, Elektro, Solar', 'Gas, Öl', 'Gas, Flüssiggas, Solar',
       'Pellets', 'Fernwärme, Gas, Blockheizkraftwerk', 'Solar',
       'Fernwärme, Erdwärme', 'Fernwärme, Elektro', 'Gas, Flüssiggas',
       'Gas,', 'Kohle, Gas, Öl', 'Elektro, Gas, Blockheizkraftwerk',
```

```
       'Flüssiggas', 'Fernwärme, Elektro, Blockheizkraftwerk',
       'Fernwärme, Elektro, Gas'], dtype=object)
```

We are facing the same problem. We have too many features and in german. We can follow a similar approach with the 'flooring_type'.

[117]:
```python
def classify_heating_rent(value):
    if pd.isna(value) or value == "Unknown":
        return "Unknown"

    types = [t.strip() for t in value.split(",") if t.strip()]

    has_district = "Fernwärme" in types
    has_gas = "Gas" in types
    has_oil = "Öl" in types or "Kohle" in types
    has_heatpump = "Luft-/Wasser-Wärmepumpe" in types or "Erdwärme" in types
    has_electric = "Elektro" in types
    has_solar = "Solar" in types
    has_chp = "Blockheizkraftwerk" in types

    # District is its own strong category
    if has_district and len(types) == 1:
        return "District Heating"

    # Modern systems
    if has_heatpump:
        if len(types) > 1:
            return "Hybrid"
        return "Heat Pump / Geothermal"

    # Gas baseline
    if has_gas and not has_oil:
        if len(types) > 1:
            return "Hybrid"
        return "Gas"

    # Oil / Coal
    if has_oil:
        return "Oil/Coal"

    # Electric only
    if has_electric and len(types) == 1:
        return "Electric"

    # Mixed systems
    if len(types) > 1:
        return "Hybrid"
```

```
    return "Other"
```

[118]:
```
df["energy_source"] = df["energy_source"].apply(classify_heating_rent)

df["energy_source"].unique()
```

[118]:
```
array(['Gas', 'District Heating', 'Hybrid', 'Other', 'Unknown',
       'Heat Pump / Geothermal', 'Oil/Coal', 'Electric'], dtype=object)
```

7. Formatting 'heating_type' feature

[119]:
```
df["heating_type"].unique()
```

[119]:
```
array(['Unknown', 'Zentralheizung', 'Zentralheizung: Fußbodenheizung',
       'Fußbodenheizung', 'Zentralheizung: Heizkörper', 'Etagenheizung',
       'individuell', 'Etagenheizung: Fußbodenheizung',
       'individuell: Heizkörper', 'Ofen', 'individuell: Fußbodenheizung'],
      dtype=object)
```

[120]:
```
def classify_heating_system(value):
    if pd.isna(value) or value == "Unknown":
        return "Unknown"

    value = value.strip()

    # Premium
    if "Fußbodenheizung" in value:
        return "Underfloor Heating"

    # Central
    if value.startswith("Zentralheizung"):
        return "Central Heating"

    # Unit-level heating
    if value.startswith("Etagenheizung"):
        return "Unit Heating"

    # Stove
    if "Ofen" in value:
        return "Stove Heating"

    # Individual systems
    if value.startswith("individuell"):
        return "Individual"

    return "Other"
```

[121]:
```
df["heating_type"] = df["heating_type"].apply(classify_heating_system)
df["heating_type"].unique()
```

[121]:
```
array(['Unknown', 'Central Heating', 'Underfloor Heating', 'Unit Heating',
       'Individual', 'Stove Heating'], dtype=object)
```

8. Formatting 'property_condition' feature

[122]:
```
df["property_condition"].unique()
```

[122]:
```
array(['Gepflegt', 'Neubau', 'neuwertig', 'Erstbezug',
       'renoviert / saniert', 'Saniert', 'Neubau, Erstbezug',
       'Altbau (bis 1945), neuwertig', 'Neubau, neuwertig',
       'Altbau (bis 1945), Gepflegt',
       'Altbau (bis 1945), renoviert / saniert', 'Neubau, Massivhaus',
       'renovierungsbedürftig / sanierungsbedürftig',
       'Neubau, renoviert / saniert', 'Massivhaus, renoviert / saniert',
       'Altbau (bis 1945)', 'Massivhaus', 'teilsaniert',
       'Neubau, Massivhaus, Gepflegt', 'Neubau, Gepflegt',
       'Massivhaus, Gepflegt', 'Massivhaus, neuwertig',
       'Massivhaus, Erstbezug', 'Fertighaus, renoviert / saniert',
       'Erstbezug nach Renovierung', 'Modernisiert',
       'Neubau, Erstbezug, renoviert / saniert',
       'Altbau (bis 1945), neuwertig, renoviert / saniert',
       'Erstbezug, renoviert / saniert', 'neuwertig, renoviert / saniert',
       'Altbau (bis 1945), Saniert, renoviert / saniert',
       'Gepflegt, renoviert / saniert',
       'Neubau, neuwertig, renoviert / saniert',
       'Erstbezug nach Renovierung, renoviert / saniert',
       ', renoviert / saniert', 'Saniert, renoviert / saniert', 'Unknown'],
      dtype=object)
```

[123]:
```
def classify_condition(value):
    if pd.isna(value) or value == "Unknown":
        return "Unknown"

    value = value.strip()

    # Highest level (brand new)
    if "Erstbezug" in value or "Neubau" in value:
        return "New / First Occupancy"

    # Like new
    if "neuwertig" in value:
        return "Like New"

    # Fully renovated
```

```
    if "Saniert" in value or "renoviert / saniert" in value or "Modernisiert"␣
↪in value:
        return "Renovated"

    # Partially renovated
    if "teilsaniert" in value:
        return "Partially Renovated"

    # Needs renovation
    if "renovierungsbedürftig" in value or "sanierungsbedürftig" in value:
        return "Needs Renovation"

    # Old building
    if "Altbau" in value:
        return "Old Building"

    # Well maintained baseline
    if "Gepflegt" in value:
        return "Well Maintained"

    return "Other"
```

[124]:
```
df["property_condition"] = df["property_condition"].apply(classify_condition)
df["property_condition"].unique()
```

[124]:
```
array(['Well Maintained', 'New / First Occupancy', 'Like New',
       'Renovated', 'Old Building', 'Needs Renovation', 'Other',
       'Partially Renovated', 'Unknown'], dtype=object)
```

9. Formatting 'location' feature

[125]:
```
df["location"].sample(10)
```

[125]:
```
5227     Gustav-Adolf-Straße 114, Weißensee, Berlin (13…
5332       Kurfürstenstraße 41, Tiergarten, Berlin (10785)
17984          Carl-Herz-Ufer 3, Kreuzberg, Berlin (10961)
7966     Falkenberger Straße 145a, Weißensee, Berlin (1…
13880                     Prenzlauer Berg, Berlin (10407)
11383              Spandauer Damm, Westend, Berlin (14050)
18871              Eidechsenweg 3, Staaken, Berlin (13591)
3050                                Wedding, Berlin (13353)
11609       Herderstraße 5, Charlottenburg, Berlin (10625)
9016                             Kreuzberg, Berlin (10249)
Name: location, dtype: object
```

First, let's check if we have any samples that are not in Berlin

[126]:
```
mask_no_berlin = ~df["location"].str.contains("Berlin", case=False, na=False)

df_no_berlin = df.loc[mask_no_berlin, "location"]

df_no_berlin.head(20)
```

[126]:
```
18804    Kreuzberg, (10115)
13320    Kreuzberg, (10115)
8520        Moabit, (10551)
8374     Kreuzberg, (10115)
4882     Kreuzberg, (10115)
4457        Moabit, (10551)
22571     Mahlsdorf (12623)
14774     Mahlsdorf (12623)
9671      Mahlsdorf (12623)
Name: location, dtype: object
```

All samples in our data set are in Berlin, these 9 just happen to not have the word 'Berlin' explicitly written on them.

By observing many samples, it was noticed that this feature has this composition: [street_name + house number], [area], Berlin, [(ZIP Code)] We have to decide what information we should extract from this feature. Extracting street number as a seperate feature would mean too many categories since Berlin is a huge city. Extracting the area and ZIP code into too different features would cause an almost perfect collinearity, since they contain very similar information.

[127]:
```
# Remove word 'Berlin'
df["location_clean"] = df["location"].str.replace("Berlin", "", regex=True,␣
↪case=False)

# Extract ZIP
df["zip_code"] = df["location_clean"].str.extract(r"\((\d{5})\)\s*$")

# Extract text before the last comma => name of district
df["district"] = df["location_clean"].str.extract(r",\s*([^,]+?
↪)\s*\(\d{5}\)\s*$")

# Clean whitespace
df["district"] = df["district"].str.strip()
```

[128]:
```
print(len(df["zip_code"].unique()))
```

```
191
```

[129]:
```
print(len(df["district"].unique()))
```

```
52
```

[130]:
```
df["district"].unique()
```

[130]:
```
array(['', nan, '/ Bohnsdorf', '- Treptow', 'Tempelhof', '/ Mitte',
       '/ Charlottenburg', '/ Kreuzberg', '/ Lichtenrade',
       '/ Wilmersdorf', '-Spandau', '/ Mahlsdorf', 'Friedrichshain',
       '-Prenzlauer Berg', '-Reinickendorf', '/ Weißensee',
       '/ Friedrichshain', '-Treptow', '-Wilmersdorf', 'Kreuzberg',
       '-Zehlendorf', 'Köpenick', '/ Nikolassee', 'Kaulsdorf',
       '/ Tiergarten', '-Köpenick', '/ Adlershof', '-Dahlem',
       '/ Wendenschloss', '/ Köpenick', '-Mitte', '-Kaulsdorf',
       '-Hakenfelde', '-Friedrichsfelde', '/ Prenzlauer Berg',
       '/ Lichtenberg', '-Grünau', '/ Schmargendorf', '-12161', '-10711',
       '-12105', '-10589', '-12109', '-12203', '-12207', '-12205',
       '-12247', '-10627', '-12159', '/ Niederschönhausen', '-Weißensee',
       '/ Kladow'], dtype=object)
```

The most suitable way to deal with the location feature is to clean the 'district' feature using information from the internet to map the zip codes to their corresponding areas and for the missing values we map them specifically by inspecting them and condcluding the place.

[131]:
```
# keep only words and numbers
df["district"] = df["district"].str.extract(r'([A-Za-zÀ-ÖØ-öø-ÿ0-9]+)',␣
 ↪expand=False)
df["district"].unique()
```

[131]:
```
array([nan, 'Bohnsdorf', 'Treptow', 'Tempelhof', 'Mitte',
       'Charlottenburg', 'Kreuzberg', 'Lichtenrade', 'Wilmersdorf',
       'Spandau', 'Mahlsdorf', 'Friedrichshain', 'Prenzlauer',
       'Reinickendorf', 'Weißensee', 'Zehlendorf', 'Köpenick',
       'Nikolassee', 'Kaulsdorf', 'Tiergarten', 'Adlershof', 'Dahlem',
       'Wendenschloss', 'Hakenfelde', 'Friedrichsfelde', 'Lichtenberg',
       'Grünau', 'Schmargendorf', '12161', '10711', '12105', '10589',
       '12109', '12203', '12207', '12205', '12247', '10627', '12159',
       'Niederschönhausen', 'Kladow'], dtype=object)
```

[132]:
```
print(len(df["district"].unique()))
```

```
41
```

Now we have to map the ZIP codes to the corresponding area

[133]:
```
zip_dict = {
    "12161": "Tempelhof-Schöneberg",
    "10711": "Charlottenburg-Wilmersdorf",
    "12105": "Mariendorf",
    "10589": "Charlottenburg",
    "12109": "Mariendorf",
    "12203": "Steglitz",
    "12207": "Lichterfelde",
    "12205": "Lichterfelde",
    "12247": "Steglitz",
```

```
    "10627": "Charlottenburg",
    "12159": "Friedenau",
}

def map_zip_text(text):
    if pd.isna(text):
        return text

    text = str(text)
    for zip_code, district in zip_dict.items():
        text = text.replace(zip_code, district)
    return text

df["district"] = df["district"].apply(map_zip_text)
```

[134]:
```
df["district"].unique()
```

[134]:
```
array([nan, 'Bohnsdorf', 'Treptow', 'Tempelhof', 'Mitte',
       'Charlottenburg', 'Kreuzberg', 'Lichtenrade', 'Wilmersdorf',
       'Spandau', 'Mahlsdorf', 'Friedrichshain', 'Prenzlauer',
       'Reinickendorf', 'Weißensee', 'Zehlendorf', 'Köpenick',
       'Nikolassee', 'Kaulsdorf', 'Tiergarten', 'Adlershof', 'Dahlem',
       'Wendenschloss', 'Hakenfelde', 'Friedrichsfelde', 'Lichtenberg',
       'Grünau', 'Schmargendorf', 'Tempelhof-Schöneberg',
       'Charlottenburg-Wilmersdorf', 'Mariendorf', 'Steglitz',
       'Lichterfelde', 'Friedenau', 'Niederschönhausen', 'Kladow'],
      dtype=object)
```

[135]:
```
print(len(df["district"].unique()))
```

```
36
```

[136]:
```
mask = df["district"].isna()
df[mask][['district','zip_code','location']]
```

[136]:
```
      district zip_code                                          location
22653      NaN    10405                     Prenzlauer Berg, Berlin (10405)
22652      NaN    13347  Reinickendorfer Str. 1, Gesundbrunnen, Berlin …
22651      NaN    13629  Heinrich-Hertz-Straße 40, Siemensstadt, Berlin…
22642      NaN    10117                              Mitte, Berlin (10117)
22641      NaN    10318                         Karlshorst, Berlin (10318)
…          …      …                                                   …
8          NaN    10179         Am Köllnischen Park, Mitte, Berlin (10179)
9          NaN    13129          Karower Damm, Blankenburg, Berlin (13129)
6          NaN    10437        Topsstraße, Prenzlauer Berg, Berlin (10437)
1          NaN    12555              Spreestraße, Köpenick, Berlin (12555)
3          NaN    12555            Bahnhofstraße, Köpenick, Berlin (12555)
```

```
[15522 rows x 3 columns]
```

All these need to be mapped

[137]:
```
zip_dict = {
    # Charlottenburg-Wilmersdorf (106xx-107xx)
    "10551": "Tiergarten",
    "10553": "Tiergarten",
    "10555": "Tiergarten",
    "10557": "Tiergarten",
    "10559": "Tiergarten",
    "10585": "Charlottenburg",
    "10587": "Charlottenburg",
    "10589": "Charlottenburg",
    "10623": "Charlottenburg",
    "10625": "Charlottenburg",
    "10627": "Charlottenburg",
    "10629": "Charlottenburg",
    "10707": "Wilmersdorf",
    "10709": "Wilmersdorf",
    "10711": "Wilmersdorf",
    "10713": "Wilmersdorf",
    "10715": "Wilmersdorf",
    "10717": "Wilmersdorf",
    "10719": "Wilmersdorf",
    "10777": "Schöneberg",
    "10779": "Schöneberg",
    "10781": "Schöneberg",
    "10783": "Schöneberg",
    "10785": "Schöneberg",
    "10787": "Schöneberg",
    "10789": "Schöneberg",

    # Tempelhof-Schöneberg (107xx, 108xx, 121xx)
    "10823": "Schöneberg",
    "10825": "Schöneberg",
    "10827": "Schöneberg",
    "10829": "Schöneberg",
    "12043": "Neukölln",
    "12045": "Neukölln",
    "12047": "Neukölln",
    "12049": "Neukölln",
    "12051": "Neukölln",
    "12053": "Neukölln",
    "12055": "Neukölln",
    "12057": "Neukölln",
    "12059": "Neukölln",
```

```
    "12099": "Tempelhof",
    "12101": "Tempelhof",
    "12103": "Tempelhof",
    "12105": "Tempelhof",
    "12107": "Tempelhof",
    "12109": "Tempelhof",
    "12157": "Steglitz",
    "12159": "Friedenau",
    "12161": "Friedenau",
    "12163": "Steglitz",
    "12165": "Steglitz",
    "12167": "Steglitz",
    "12169": "Steglitz",

    # Steglitz-Zehlendorf (121xx, 122xx, 141xx)
    "12203": "Steglitz",
    "12205": "Lichterfelde",
    "12207": "Lichterfelde",
    "12209": "Lichterfelde",
    "12247": "Steglitz",
    "12249": "Steglitz",
    "12277": "Marienfelde",
    "12279": "Marienfelde",
    "12305": "Lichtenrade",
    "12307": "Lichtenrade",
    "12309": "Lichtenrade",
    "12347": "Britz",
    "12349": "Britz",
    "12351": "Britz",
    "12353": "Britz",
    "12355": "Britz",
    "12357": "Britz",
    "12359": "Britz",

    # Tempelhof-Schöneberg (124xx)
    "12435": "Alt-Treptow",
    "12437": "Alt-Treptow",
    "12439": "Niederschöneweide",
    "12459": "Oberschöneweide",
    "12487": "Johannisthal",
    "12489": "Johannisthal",

    # Neukölln (120xx, 123xx)
    "12524": "Altglienicke",
    "12526": "Bohnsdorf",
    "12527": "Grünau",
    "12555": "Köpenick",
```

```
    "12557": "Köpenick",
    "12559": "Köpenick",
    "12587": "Friedrichshagen",
    "12589": "Rahnsdorf",

    # Treptow-Köpenick (124xx, 125xx)
    "12619": "Hellersdorf",
    "12621": "Hellersdorf",
    "12623": "Mahlsdorf",
    "12627": "Hellersdorf",
    "12629": "Hellersdorf",
    "12679": "Marzahn",
    "12681": "Marzahn",
    "12683": "Biesdorf",
    "12685": "Marzahn",
    "12687": "Marzahn",
    "12689": "Marzahn",

    # Marzahn-Hellersdorf (126xx)
    "13051": "Hohenschönhausen",
    "13053": "Hohenschönhausen",
    "13055": "Hohenschönhausen",
    "13057": "Hohenschönhausen",
    "13059": "Hohenschönhausen",
    "13086": "Weißensee",
    "13088": "Weißensee",
    "13089": "Weißensee",

    # Lichtenberg (103xx, 130xx)
    "13125": "Buch",
    "13127": "Französisch Buchholz",
    "13129": "Blankenburg",
    "13156": "Niederschönhausen",
    "13158": "Niederschönhausen",
    "13159": "Niederschönhausen",
    "13187": "Pankow",
    "13189": "Pankow",

    # Pankow (101xx, 104xx, 131xx)
    "13347": "Wedding",
    "13349": "Wedding",
    "13351": "Wedding",
    "13353": "Wedding",
    "13355": "Gesundbrunnen",
    "13357": "Gesundbrunnen",
    "13359": "Gesundbrunnen",
```

```
    # Mitte (101xx, 133xx)
    "13403": "Reinickendorf",
    "13405": "Reinickendorf",
    "13407": "Reinickendorf",
    "13409": "Reinickendorf",
    "13435": "Wittenau",
    "13437": "Wittenau",
    "13439": "Wittenau",
    "13465": "Hermsdorf",
    "13467": "Hermsdorf",
    "13469": "Hermsdorf",

    # Reinickendorf (134xx, 135xx)
    "13503": "Tegel",
    "13505": "Tegel",
    "13507": "Tegel",
    "13509": "Tegel",
    "13581": "Spandau",
    "13583": "Spandau",
    "13585": "Spandau",
    "13587": "Spandau",
    "13589": "Spandau",
    "13591": "Staaken",
    "13593": "Staaken",
    "13595": "Staaken",
    "13597": "Spandau",
    "13599": "Spandau",

    # Spandau (135xx, 140xx)
    "14050": "Charlottenburg",
    "14052": "Charlottenburg",
    "14055": "Charlottenburg",
    "14057": "Charlottenburg",
    "14059": "Charlottenburg",
    "14089": "Kladow",

    # Charlottenburg-Wilmersdorf (140xx)
    "14109": "Wannsee",
    "14129": "Nikolassee",
    "14163": "Zehlendorf",
    "14165": "Zehlendorf",
    "14167": "Zehlendorf",
    "14169": "Zehlendorf",
    "14193": "Grunewald",
    "14195": "Dahlem",
    "14197": "Wilmersdorf",
    "14199": "Wilmersdorf",
```

```
    # Steglitz-Zehlendorf (141xx)
    "10115": "Mitte",
    "10117": "Mitte",
    "10119": "Mitte",
    "10178": "Mitte",
    "10179": "Mitte",
    "10243": "Friedrichshain",
    "10245": "Friedrichshain",
    "10247": "Friedrichshain",
    "10249": "Friedrichshain",
    "10315": "Lichtenberg",
    "10317": "Lichtenberg",
    "10318": "Lichtenberg",
    "10319": "Lichtenberg",
    "10365": "Lichtenberg",
    "10367": "Lichtenberg",
    "10369": "Lichtenberg",
    "10405": "Prenzlauer Berg",
    "10407": "Prenzlauer Berg",
    "10409": "Prenzlauer Berg",
    "10435": "Prenzlauer Berg",
    "10437": "Prenzlauer Berg",
    "10439": "Prenzlauer Berg",
    "10551": "Tiergarten",
    "10553": "Tiergarten",
    "10555": "Tiergarten",
    "10557": "Tiergarten",
    "10559": "Tiergarten",
    "10585": "Charlottenburg",
    "10587": "Charlottenburg",
    "10589": "Charlottenburg",
    "10623": "Charlottenburg",
    "10625": "Charlottenburg",
    "10627": "Charlottenburg",
    "10629": "Charlottenburg",
    "10707": "Wilmersdorf",
    "10709": "Wilmersdorf",
    "10711": "Wilmersdorf",
    "10713": "Wilmersdorf",
    "10715": "Wilmersdorf",
    "10717": "Wilmersdorf",
    "10719": "Wilmersdorf",
    "10777": "Schöneberg",
    "10779": "Schöneberg",
    "10781": "Schöneberg",
    "10783": "Schöneberg",
```

```
    "10785": "Schöneberg",
    "10787": "Schöneberg",
    "10789": "Schöneberg",
    "10799": "Schöneberg",
    "10823": "Schöneberg",
    "10825": "Schöneberg",
    "10827": "Schöneberg",
    "10829": "Schöneberg",
    "10961": "Kreuzberg",
    "10963": "Kreuzberg",
    "10965": "Kreuzberg",
    "10967": "Kreuzberg",
    "10969": "Kreuzberg",
    "10997": "Kreuzberg",
    "10999": "Kreuzberg",
    "12043": "Neukölln",
    "12045": "Neukölln",
    "12047": "Neukölln",
    "12049": "Neukölln",
    "12051": "Neukölln",
    "12053": "Neukölln",
    "12055": "Neukölln",
    "12057": "Neukölln",
    "12059": "Neukölln",
    "12099": "Tempelhof",
    "12101": "Tempelhof",
    "12103": "Tempelhof",
    "12105": "Tempelhof",
    "12107": "Tempelhof",
    "12109": "Tempelhof",
    "12157": "Steglitz",
    "12159": "Friedenau",
    "12161": "Friedenau",
    "12163": "Steglitz",
    "12165": "Steglitz",
    "12167": "Steglitz",
    "12169": "Steglitz",
    "12203": "Steglitz",
    "12205": "Lichterfelde",
    "12207": "Lichterfelde",
    "12209": "Lichterfelde",
    "12247": "Steglitz",
    "12249": "Steglitz",
    "12277": "Marienfelde",
    "12279": "Marienfelde",
    "12305": "Lichtenrade",
    "12307": "Lichtenrade",
```

```
    "12309": "Lichtenrade",
    "12347": "Britz",
    "12349": "Britz",
    "12351": "Britz",
    "12353": "Britz",
    "12355": "Britz",
    "12357": "Britz",
    "12359": "Britz",
    "12435": "Alt-Treptow",
    "12437": "Alt-Treptow",
    "12439": "Niederschöneweide",
    "12459": "Oberschöneweide",
    "12487": "Johannisthal",
    "12489": "Johannisthal",
    "12524": "Altglienicke",
    "12526": "Bohnsdorf",
    "12527": "Grünau",
    "12555": "Köpenick",
    "12557": "Köpenick",
    "12559": "Köpenick",
    "12587": "Friedrichshagen",
    "12589": "Rahnsdorf",
    "12619": "Hellersdorf",
    "12621": "Hellersdorf",
    "12623": "Mahlsdorf",
    "12627": "Hellersdorf",
    "12629": "Hellersdorf",
    "12679": "Marzahn",
    "12681": "Marzahn",
    "12683": "Biesdorf",
    "12685": "Marzahn",
    "12687": "Marzahn",
    "12689": "Marzahn",
    "13051": "Hohenschönhausen",
    "13053": "Hohenschönhausen",
    "13055": "Hohenschönhausen",
    "13057": "Hohenschönhausen",
    "13059": "Hohenschönhausen",
    "13086": "Weißensee",
    "13088": "Weißensee",
    "13089": "Weißensee",
    "13125": "Buch",
    "13127": "Französisch Buchholz",
    "13129": "Blankenburg",
    "13156": "Niederschönhausen",
    "13158": "Niederschönhausen",
    "13159": "Niederschönhausen",
```

```
    "13187": "Pankow",
    "13189": "Pankow",
    "13347": "Wedding",
    "13349": "Wedding",
    "13351": "Wedding",
    "13353": "Wedding",
    "13355": "Gesundbrunnen",
    "13357": "Gesundbrunnen",
    "13359": "Gesundbrunnen",
    "13403": "Reinickendorf",
    "13405": "Reinickendorf",
    "13407": "Reinickendorf",
    "13409": "Reinickendorf",
    "13435": "Wittenau",
    "13437": "Wittenau",
    "13439": "Wittenau",
    "13465": "Hermsdorf",
    "13467": "Hermsdorf",
    "13469": "Hermsdorf",
    "13503": "Tegel",
    "13505": "Tegel",
    "13507": "Tegel",
    "13509": "Tegel",
    "13581": "Spandau",
    "13583": "Spandau",
    "13585": "Spandau",
    "13587": "Spandau",
    "13589": "Spandau",
    "13591": "Staaken",
    "13593": "Staaken",
    "13595": "Staaken",
    "13597": "Spandau",
    "13599": "Spandau",
    "13627":"Spandau",
    "13629": "Siemensstadt",
    "14050": "Charlottenburg",
    "14052": "Charlottenburg",
    "14055": "Charlottenburg",
    "14057": "Charlottenburg",
    "14059": "Charlottenburg",
    "14089": "Kladow",
    "14109": "Wannsee",
    "14129": "Nikolassee",
    "14163": "Zehlendorf",
    "14165": "Zehlendorf",
    "14167": "Zehlendorf",
    "14169": "Zehlendorf",
```

```
    "14193": "Grunewald",
    "14195": "Dahlem",
    "14197": "Wilmersdorf",
    "14199": "Wilmersdorf"
}
```

[138]:
```
def map_zip_text(row):
    # Check if area is NaN and zip code exists
    if pd.isna(row['district']) and pd.notna(row['zip_code']):
        zip_code = str(row['zip_code']).strip()

        if zip_code in zip_dict:
            return f"{zip_dict[zip_code]}"

    return row['district']

# Apply the function row-wise
df["district"] = df.apply(map_zip_text, axis=1)
```

[139]:
```
df["district"].unique()
```

[139]:
```
array(['Prenzlauer Berg', 'Wedding', 'Siemensstadt', 'Mitte',
       'Lichtenberg', 'Weißensee', 'Charlottenburg', 'Wilmersdorf',
       'Bohnsdorf', 'Friedrichshain', 'Tempelhof', 'Schöneberg',
       'Französisch Buchholz', 'Tegel', 'Kreuzberg', 'Friedenau',
       'Zehlendorf', 'Britz', 'Friedrichshagen', 'Köpenick', 'Mahlsdorf',
       'Treptow', 'Neukölln', 'Grünau', 'Gesundbrunnen', 'Staaken',
       'Pankow', 'Steglitz', 'Reinickendorf', 'Tiergarten', 'Lichtenrade',
       'Johannisthal', 'Kladow', 'Biesdorf', 'Hohenschönhausen',
       'Lichterfelde', 'Niederschöneweide', 'Oberschöneweide',
       'Blankenburg', 'Grunewald', 'Wannsee', 'Dahlem', 'Hermsdorf',
       'Spandau', 'Altglienicke', 'Niederschönhausen', 'Hellersdorf',
       'Alt-Treptow', 'Prenzlauer', 'Marzahn', 'Marienfelde', 'Wittenau',
       'Buch', 'Nikolassee', 'Rahnsdorf', 'Kaulsdorf', 'Adlershof',
       'Wendenschloss', 'Hakenfelde', 'Friedrichsfelde', 'Schmargendorf',
       'Tempelhof-Schöneberg', 'Charlottenburg-Wilmersdorf', 'Mariendorf',
       nan], dtype=object)
```

[140]:
```
mask = df["district"].isna()
df[mask][['district','zip_code','location']]
```

[140]:

	district	zip_code	location
13752	NaN	NaN	Prenzlauer Berg, Berlin (100001199)
10778	NaN	NaN	Wilmersdorf, Berlin (10)
10570	NaN	NaN	Niederschönhausen, Berlin (13)
9311	NaN	NaN	Prenzlauer Berg, Berlin (100001199)
7359	NaN	NaN	Französisch Buchholz, Berlin (13)

```
7321       NaN      NaN             Wilmersdorf, Berlin (10)
2598       NaN      NaN  Prenzlauer Berg, Berlin (100001199)
2159       NaN      NaN       Niederschönhausen, Berlin (13)
1431       NaN      NaN          Friedrichshain, Berlin (10)
1327       NaN      NaN                           Berlin (13)
1068       NaN      NaN             Wilmersdorf, Berlin (10)
879        NaN      NaN         Prenzlauer Berg, Berlin (10)
625        NaN      NaN       Schöneberg, Berlin (100001199)
387        NaN      NaN    Französisch Buchholz, Berlin (13)
```

There are still nan values left. These are all valid samples expect sample number 1327 which we can simply drop

[141]:
```
df = df.drop(index=1327)
```

[142]:
```
# Since we dont have many feautres we edit them manually.
df.loc[[13752, 9311, 2598, 879], 'district'] = 'Prenzlauer Berg'
df.loc[[10778, 7321, 1068], 'district'] = 'Wilmersdorf'
df.loc[[10570, 2159], 'district'] = 'Niederschönhausen'
df.loc[[7359, 387], 'district'] = 'Französisch Buchholz'
df.loc[1431, 'district'] = 'Friedrichshain'
df.loc[625, 'district'] = 'Schöneberg'
```

[143]:
```
df[df['district'].isna()][['location', 'district']]
```

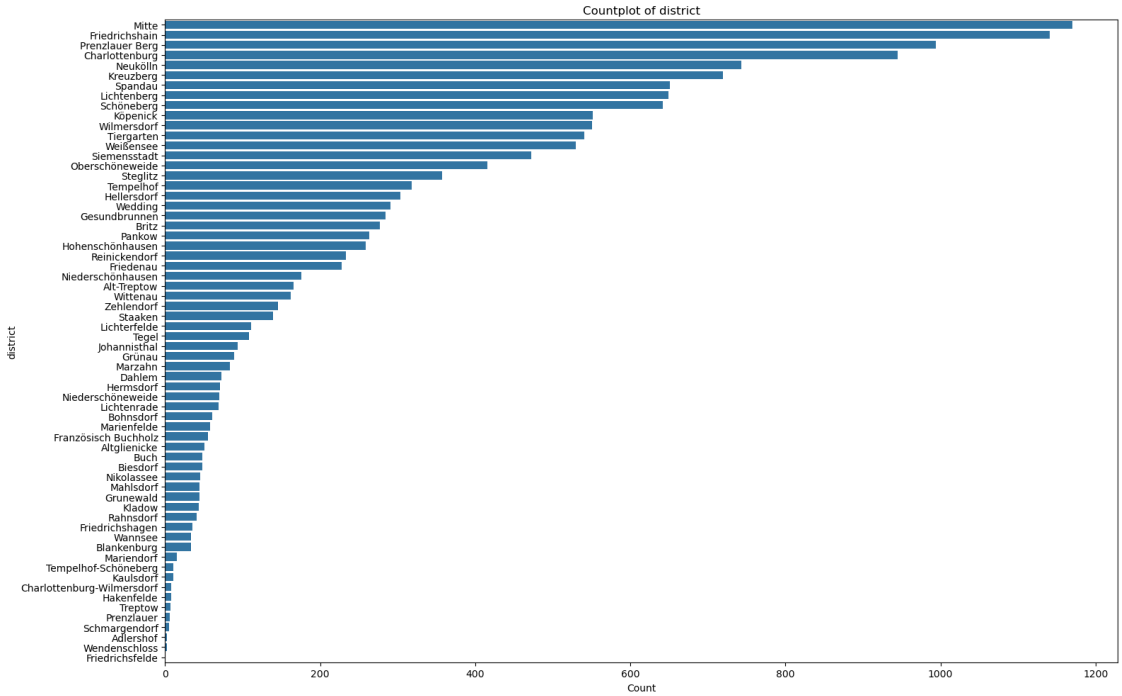
[143]:
```
Empty DataFrame
Columns: [location, district]
Index: []
```

[144]:
```
len(df["district"].unique())
```

[144]:
```
64
```

We managed to clean all the values, but we still have a problem, that being having 64 categories (districts). Maybe we have some values which we can cluster together.

[145]:
```
plt.figure(figsize=(16, 10))
sns.countplot(data=df, y="district", order=df["district"].value_counts().index)
plt.title('Countplot of district')
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```

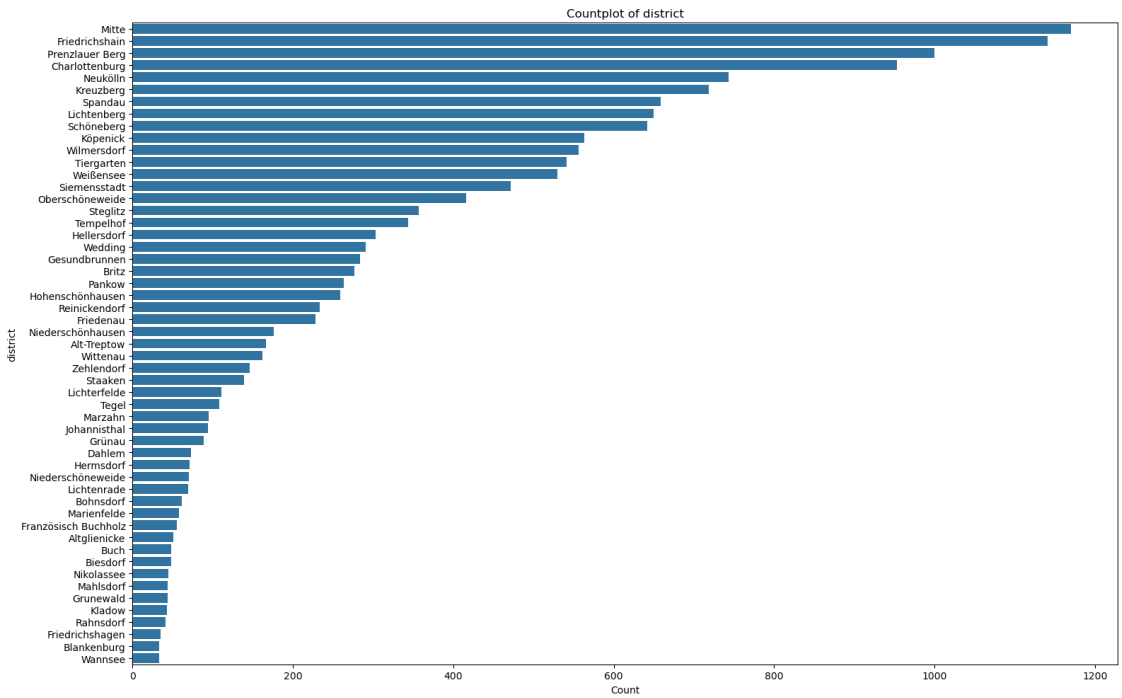


[146]:
```
# We have 'Prenzlauer Berg' written also as 'Prenzlauer', so we can just merge␣
↪the categories together.
# df[df['district'] == 'Prenzlauer']['district']
df.loc[df['district'] == 'Prenzlauer','district'] = 'Prenzlauer Berg'
```

[147]:
```
# We should also group categories with very few occuranes into more general␣
↪districts to avoid too many categories.
df.loc[df['district'] == 'Friedrichsfelde','district'] = 'Lichtenberg'
df.loc[df['district'] == 'Wendenschloss','district'] = 'Köpenick'
df.loc[df['district'] == 'Adlershof','district'] = 'Köpenick'
df.loc[df['district'] == 'Schmargendorf','district'] = 'Wilmersdorf'
df.loc[df['district'] == 'Treptow','district'] = 'Köpenick'
df.loc[df['district'] == 'Hakenfelde','district'] = 'Spandau'
df.loc[df['district'] == 'Charlottenburg-Wilmersdorf','district'] =␣
↪'Charlottenburg'
df.loc[df['district'] == 'Kaulsdorf','district'] = 'Marzahn'
df.loc[df['district'] == 'Tempelhof-Schöneberg','district'] = 'Tempelhof'
df.loc[df['district'] == 'Mariendorf','district'] = 'Tempelhof'    # 15 values

# As noticed some arbitrary decisions were made, like converting␣
↪'Tempelhof-Schöneberg' into 'Tempelhof'. I picked 'Tempelhof'
# instead of 'Schöneberg' to balance the categories more
# Also this impact is relevant since they were less than 15 samples.
```

[148]:
```
plt.figure(figsize=(16, 10))
sns.countplot(data=df, y="district", order=df["district"].value_counts().index)
plt.title('Countplot of district')
plt.xlabel('Count')
plt.tight_layout()
plt.show()
```



[149]:
```
len(df["district"].unique())
```

[149]: 53

[150]:
```
len(df[df["district"] == "Blankenburg"])
```

[150]: 33

[151]:
```
df.shape
```

[151]: (15802, 27)

The categories with the lowest number of samples are Wannsee and Blankenburg with 33 samples which is not very low. We have 53 categories which is quite a lot for a feature to have, but the type of the feature justifies it. I would prefer not to cluter any more categories together to preserve information. There are many ML models out there that work well with features with this many categories. A ratio of 53 categories to 15000 samples is also a good ratio.

Part 4: Handling outliers

1. Handling 'cold_rent' feature

[152]:
```
df['cold_rent'].describe()
```

[152]:
```
count    15802.00000
mean      1313.84175
std       1188.08090
min          1.09500
25%        677.50000
50%       1100.00000
75%       1642.00000
max      35000.00000
Name: cold_rent, dtype: float64
```

We clearly have outliers and a lot of them. We have to find a way to deal with them. But first we need to inspect them into detial to see wether it is a type or just an invalid sample.

[153]:
```
# Knowing Berlin anything below 300 euros is not possible.
len(df[df['cold_rent'] < 300])
```

[153]:
```
163
```

[154]:
```
# We have to inspect at least 163 values. Every sample is valuable and should␣
↪not be dropped, so will
# to inspect them one by one.
# We will start with a threshold of '2' and increase it gradually until we␣
↪reach 300.
df[df['cold_rent'] < 2]
```

[154]:
```
                                          listing_url  cold_rent  area  \
28     https://www.immowelt.de/expose/ed2ff766-9412-4…      1.349  65.8
26     https://www.immowelt.de/expose/7a4aec64-ab72-4…      1.449  66.5
15     https://www.immowelt.de/expose/d4043120-352d-4…      1.095  45.0
17     https://www.immowelt.de/expose/0d680cb5-7a9c-4…      1.595  67.5
14     https://www.immowelt.de/expose/94b19a7d-b5d4-4…      1.095  40.0
4      https://www.immowelt.de/expose/03a3d0b6-9fab-4…      1.399  65.8
0      https://www.immowelt.de/expose/f6e9dd76-dbe1-4…      1.499  58.3
22169  https://www.immowelt.de/expose/f899bf21-d112-4…      1.250  80.0
18694  https://www.immowelt.de/expose/f899bf21-d112-4…      1.250  80.0
14493  https://www.immowelt.de/expose/f899bf21-d112-4…      1.250  80.0
13267  https://www.immowelt.de/expose/f899bf21-d112-4…      1.250  80.0
29     https://www.immowelt.de/expose/a7088184-d5a8-4…      1.530  61.1
12     https://www.immowelt.de/expose/1a0ba5cb-3931-4…      1.150  40.9
8      https://www.immowelt.de/expose/aba79292-351c-4…      1.330  49.3
6      https://www.immowelt.de/expose/49ae5e0a-6f97-4…      1.645  85.8
1      https://www.immowelt.de/expose/56cf72dd-00ea-4…      1.178  45.7

       rooms  floor        free_from  \
```

```
28       2.0    0.0      immediately
26       2.0    0.0  not_immediately
15       2.0    0.0      immediately
17       2.0    0.0      immediately
14       2.0    0.0      immediately
4        2.0   -1.0  not_immediately
0        2.0   -1.0  not_immediately
22169    3.0    NaN          unknown
18694    3.0    6.0          unknown
14493    3.0    4.0          unknown
13267    3.0    3.0          unknown
29       3.0    0.0  not_immediately
12       2.0    0.0  not_immediately
8        2.0   -1.0  not_immediately
6        3.0   -1.0  not_immediately
1        2.0   -1.0  not_immediately

                                                 location  has_balkon  \
28         Friedenauer Höhe 8, Friedenau, Berlin (12159)        True
26        Friedenauer Höhe 10, Friedenau, Berlin (12159)        True
15     Kienhorststr. 103A, Reinickendorf, Berlin (13403)        True
17     Kienhorststr. 103A, Reinickendorf, Berlin (13403)       False
14     Kienhorststr. 103A, Reinickendorf, Berlin (13403)        True
4         Friedenauer Höhe 16, Friedenau, Berlin (12159)        True
0        Friedenauer Höhe 23a, Friedenau, Berlin (12159)        True
22169                              Mitte, Berlin (10115)       False
18694                              Mitte, Berlin (10115)       False
14493                              Mitte, Berlin (10115)       False
13267                              Mitte, Berlin (10115)       False
29     Allee der Kosmonauten, Friedrichsfelde, Berlin…       False
12               Spandauer Damm, Westend, Berlin (14050)       False
8             Am Köllnischen Park, Mitte, Berlin (10179)       False
6            Topsstraße, Prenzlauer Berg, Berlin (10437)       False
1                  Spreestraße, Köpenick, Berlin (12555)       False

       has_terrasse  has_garten  …  flooring_type           energy_source  \
28            False       False  …        unknown        District Heating
26            False        True  …        unknown        District Heating
15             True        True  …          Tiles  Heat Pump / Geothermal
17            False        True  …          Tiles  Heat Pump / Geothermal
14            False        True  …          Tiles  Heat Pump / Geothermal
4             False       False  …        unknown        District Heating
0             False       False  …           Wood        District Heating
22169         False       False  …        unknown                 Unknown
18694         False       False  …        unknown                 Unknown
14493         False       False  …        unknown                 Unknown
13267         False       False  …        unknown                 Unknown
```

29	False	False	…	Carpet	Unknown
12	False	False	…	unknown	Unknown
8	False	False	…	Carpet	Unknown
6	False	False	…	Wood	Unknown
1	False	False	…	Wood	Unknown

	heating_type	property_condition	year_built	schufa_check	\
28	Underfloor Heating	Like New	2023	True	
26	Underfloor Heating	Like New	2023	True	
15	Underfloor Heating	New / First Occupancy	2025	True	
17	Underfloor Heating	New / First Occupancy	2025	True	
14	Underfloor Heating	New / First Occupancy	2025	True	
4	Underfloor Heating	Like New	2024	True	
0	Underfloor Heating	New / First Occupancy	2025	True	
22169	Unknown	Unknown	<NA>	True	
18694	Unknown	Unknown	<NA>	True	
14493	Unknown	Unknown	<NA>	True	
13267	Unknown	Unknown	<NA>	True	
29	Unit Heating	Unknown	2020	True	
12	Unit Heating	Unknown	2025	True	
8	Unit Heating	Unknown	2021	True	
6	Unit Heating	Unknown	2023	True	
1	Unit Heating	Unknown	2025	True	

	scraped_at	location_clean	\
28	2025-12-28 21:33:00	Friedenauer Höhe 8, Friedenau,	(12159)
26	2025-12-28 21:32:00	Friedenauer Höhe 10, Friedenau,	(12159)
15	2025-12-28 21:30:00	Kienhorststr. 103A, Reinickendorf,	(13403)
17	2025-12-28 21:30:00	Kienhorststr. 103A, Reinickendorf,	(13403)
14	2025-12-28 21:29:00	Kienhorststr. 103A, Reinickendorf,	(13403)
4	2025-12-28 21:27:00	Friedenauer Höhe 16, Friedenau,	(12159)
0	2025-12-28 21:25:00	Friedenauer Höhe 23a, Friedenau,	(12159)
22169	2026-01-31 16:33:37	Mitte,	(10115)
18694	2026-01-27 21:15:53	Mitte,	(10115)
14493	2026-01-23 10:44:24	Mitte,	(10115)
13267	2026-01-22 20:00:10	Mitte,	(10115)
29	2025-12-28 21:33:00	Allee der Kosmonauten, Friedrichsfelde,	(10315)
12	2025-12-28 21:29:00	Spandauer Damm, Westend,	(14050)
8	2025-12-28 21:28:00	Am Köllnischen Park, Mitte,	(10179)
6	2025-12-28 21:28:00	Topsstraße, Prenzlauer Berg,	(10437)
1	2025-12-28 21:26:00	Spreestraße, Köpenick,	(12555)

	zip_code	district
28	12159	Friedenau
26	12159	Friedenau
15	13403	Reinickendorf
17	13403	Reinickendorf

```
14       13403    Reinickendorf
4        12159        Friedenau
0        12159        Friedenau
22169    10115            Mitte
18694    10115            Mitte
14493    10115            Mitte
13267    10115            Mitte
29       10315      Lichtenberg
12       14050   Charlottenburg
8        10179            Mitte
6        10437  Prenzlauer Berg
1        12555         Köpenick

[16 rows x 27 columns]
```

For this one the problem and solution are obvious, we just have to multiple the values by 1000.

[155]:
```
df.loc[df["cold_rent"] < 2, "cold_rent"] *= 1000
```

[156]:
```
df[df['cold_rent'] < 5]
```

[156]:
```
                                          listing_url  cold_rent   area  \
22   https://www.immowelt.de/expose/d5dbcc47-60a6-4…      2.300  115.0
18   https://www.immowelt.de/expose/94a3c69c-14a9-4…      2.040   88.6
13   https://www.immowelt.de/expose/25d8e8ff-8de6-4…      2.110   87.8
7    https://www.immowelt.de/expose/7e269628-a21a-4…      2.249   85.8
778  https://www.immowelt.de/expose/00aa6fd9-a5ab-4…      3.000   50.0

     rooms  floor        free_from  \
22     3.0    0.0          unknown
18     3.0    0.0      immediately
13     3.0    0.0      immediately
7      3.0   -1.0  not_immediately
778    2.0    0.0          unknown

                                          location  has_balkon  has_terrasse  \
22      Rauschener Allee 4, Westend, Berlin (14055)       False         False
18           Heidestraße 39, Moabit, Berlin (10557)        True         False
13           Heidestraße 39, Moabit, Berlin (10557)        True         False
7    Friedenauer Höhe 24, Friedenau, Berlin (12159)        True         False
778                                  Berlin (13349)       False         False

     has_garten  …  flooring_type           energy_source  \
22        False  …        unknown  Heat Pump / Geothermal
18        False  …        unknown        District Heating
13        False  …        unknown        District Heating
7         False  …        unknown        District Heating
778       False  …        unknown                 Unknown
```

```
           heating_type     property_condition  year_built  schufa_check  \
22   Underfloor Heating              Renovated        1970          True
18   Underfloor Heating  New / First Occupancy        2024          True
13   Underfloor Heating  New / First Occupancy        2024          True
7    Underfloor Heating               Like New        2023          True
778             Unknown                Unknown        <NA>          True

             scraped_at                              location_clean zip_code  \
22  2025-12-28 21:31:00     Rauschener Allee 4, Westend,  (14055)    14055
18  2025-12-28 21:30:00          Heidestraße 39, Moabit,  (10557)    10557
13  2025-12-28 21:29:00          Heidestraße 39, Moabit,  (10557)    10557
7   2025-12-28 21:28:00  Friedenauer Höhe 24, Friedenau,  (12159)    12159
778 2025-12-29 00:05:18                                   (13349)    13349

           district
22   Charlottenburg
18       Tiergarten
13       Tiergarten
7         Friedenau
778         Wedding

[5 rows x 27 columns]
```

[157]:
```
# the last one looks weird I can't tell if it is 3000 or 300. Let's just␣
↪inspect the listing if it is still available.
with pd.option_context('display.max_colwidth', None):
    print(df.loc[778]['listing_url'])
```

```
https://www.immowelt.de/expose/00aa6fd9-a5ab-4533-9d8f-
f853047e4aa0?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D149#ln=classified_search_results
&m=classified_search_results_classified_classified_detail_L
```

[158]:
```
# even after inspecting the sample, I cannot tell and I don't want to risk␣
↪making any arbitrary decision here so
# I will simply delete the sample.
df = df.drop(index=778)
```

[159]:
```
df.loc[df["cold_rent"] < 5, "cold_rent"] *= 1000
```

[160]:
```
df[df['cold_rent'] < 5]
```

[160]:
```
Empty DataFrame
Columns: [listing_url, cold_rent, area, rooms, floor, free_from, location,
has_balkon, has_terrasse, has_garten, elevator, parking, has_basement,
is_barrier-free, has_built-in_kitchen, has_bathtub, has_shower, flooring_type,
energy_source, heating_type, property_condition, year_built, schufa_check,
```

```
scraped_at, location_clean, zip_code, district]
Index: []

[0 rows x 27 columns]
```

[161]:
```python
# Let's inspect the ones with small values
with pd.option_context('display.max_colwidth', None):
    print(df.loc[[11, 2]]['listing_url'])
```

```
11    https://www.immowelt.de/expose/2605b650-de83-4aeb-8bc8-
bc8966e1d8a0?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634#ln=classified_search_results&m=classified
_search_results_classified_classified_detail_XL
2     https://www.immowelt.de/expose/591c45d0-01dd-4baf-8f0b-
626cf4b3d692?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634#ln=classified_search_results&m=classified
_search_results_classified_classified_detail_XL
Name: listing_url, dtype: object
```

[162]:
```python
# Sample 11 was not available anymore, while sample 2 was available and it had␣
↪the adjusted data this time.
df = df.drop(index=11)
df.loc[2, ["cold_rent", "floor"]] = [4100, 3]
```

[163]:
```python
with pd.option_context('display.max_colwidth', None):
    print(df[df['cold_rent'] < 200].sample(5)['listing_url'])
```

```
2013     https://www.immowelt.de/expose/92d3ddc5-302c-437b-abb0-
cc87623a9864?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D80&feature_priceDropVariantB=tru
e#ln=classified_search_results&m=classified_search_results_classified_classified
_detail_XL
322                                     https://www.immowelt.de/expose/6243b2cf-
1754-437e-b94e-
6a55381f09a1?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D164#ln=classified_search_results
&m=classified_search_results_classified_classified_detail_L
12966                                   https://www.immowelt.de/expose/0a01010e-
a4ba-493c-9d08-
348ec6167f6d?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D14#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_XL
9384                                     https://www.immowelt.de/expose/932e5f77
-56e1-4521-bd03-
5859d065861b?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D73#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
10724                                    https://www.immowelt.de/expose/f7cd459a
```

```
-1fd9-4003-91f7-
791440ab5922?serp_view=list&search=distributionTypes%3DRent%26estateTypes%3DHous
e%2CApartment%26locations%3DAD08DE8634%26page%3D96#ln=classified_search_results&
m=classified_search_results_classified_classified_detail_M
Name: listing_url, dtype: object
```

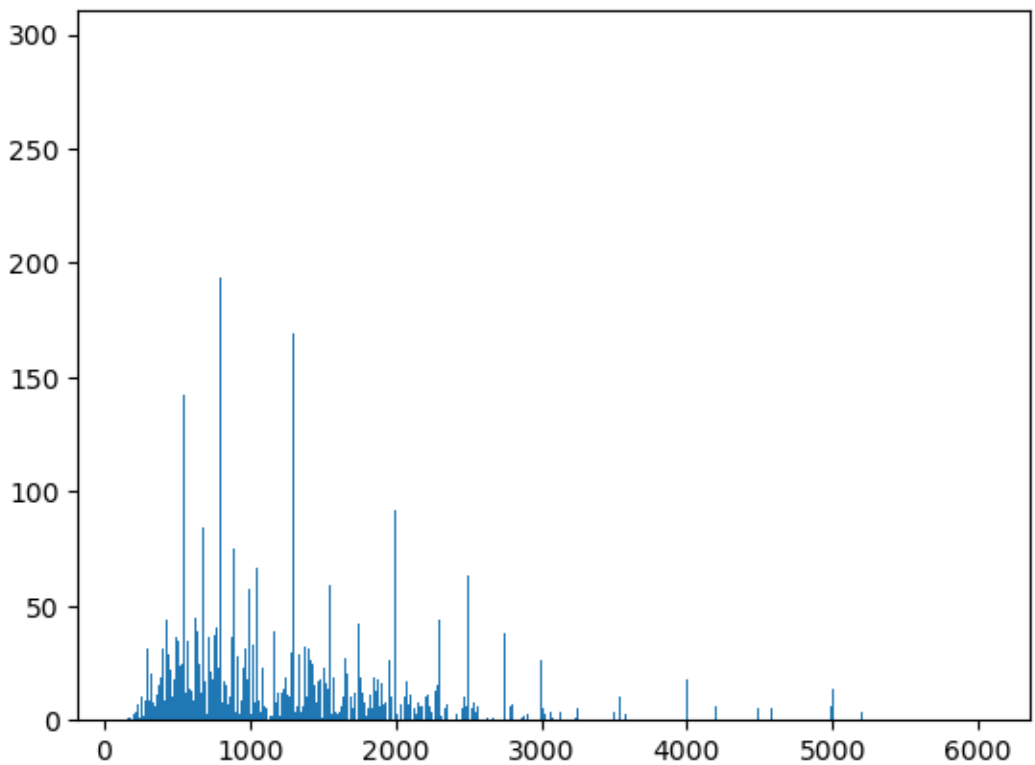
[164]:
```
# They were all legit, they explicitly mentioned the price even in the␣
↪description, so technically they were all legitimate.
# But somehow there were some duplicates, which need to be removed.
df = df.drop(index=[10740, 1713, 1857])
```

[165]:
```
# We know that 25% of the samples have a cold rent higher than 1642 and the␣
↪maximum value is 35000.
df["cold_rent"].quantile([0.90, 0.95, 0.975, 0.985, 0.99, 0.995, 0.999])
```

[165]:
```
0.900     2240.00
0.950     2700.00
0.975     3300.00
0.985     4750.00
0.990     6060.44
0.995     7175.00
0.999    12000.00
Name: cold_rent, dtype: float64
```

[166]:
```
# Let's go with the 99% winsorization, since it only means removing 1% of about␣
↪16000 which is 160 samples
# Also these samples do not help our model in any way because it does not␣
↪interest in our application. Also some of them were
# just different floors of the same building.
threshold = df["cold_rent"].quantile(0.99)
df = df[df["cold_rent"] <= threshold]
```

[167]:
```
plt.hist(df['cold_rent'],bins=2000)
plt.show()
```



[168]:
```
df["cold_rent"].quantile([0.90, 0.95, 0.975, 0.985, 0.99, 0.995, 0.999])
```

[168]:
```
0.900    2200.000
0.950    2595.000
0.975    2990.000
0.985    3300.000
0.990    3762.000
0.995    4750.000
0.999    5416.422
Name: cold_rent, dtype: float64
```

[169]:
```
# Now the percentiles are have more fair values and the graph of price␣
↪distribution seems more fair. Before we move
# to the next feature let's just have one more look at the statistics of the␣
↪prices.
df["cold_rent"].describe()
```

[169]:
```
count    15639.000000
mean      1239.285980
std        743.418851
min        120.000000
25%        675.000000
```

```
50%       1100.000000
75%       1611.355000
max       6060.000000
Name: cold_rent, dtype: float64
```

2. Handling 'area' feature

[170]:
```python
df["area"].describe()
```

[170]:
```
count    15639.00000
mean        78.65274
std        117.60875
min          1.00000
25%         50.50000
50%         66.00000
75%         86.80000
max       5251.00000
Name: area, dtype: float64
```

[171]:
```python
# anything below 10 wouldn't make sense
df[df["area"] < 10]
```

[171]:
```
                                          listing_url  cold_rent  area  \
12941  https://www.immowelt.de/expose/bf9188d4-e4b2-4…     2099.0   1.0
5550   https://www.immowelt.de/expose/bf9188d4-e4b2-4…     2099.0   1.0
4604   https://www.immowelt.de/expose/76343a42-0fc7-4…     2600.0   1.0
69     https://www.immowelt.de/expose/bf9188d4-e4b2-4…     2099.0   1.0
1457   https://www.immowelt.de/expose/e29bc640-6917-4…      418.0   1.0

       rooms  floor    free_from  \
12941    5.0    3.0  immediately
5550     5.0    1.0  immediately
4604     6.0    1.0  immediately
69       5.0    0.0  immediately
1457     3.0    0.0      unknown

                                              location  has_balkon  \
12941  Abendseglersteig 50, Wilhelmshagen, Berlin (12…        True
5550   Abendseglersteig 50, Wilhelmshagen, Berlin (12…        True
4604                      Charlottenburg, Berlin (10623)       False
69     Abendseglersteig 50, Wilhelmshagen, Berlin (12…        True
1457                                Buch, Berlin (13125)       False

       has_terrasse  has_garten  …  flooring_type     energy_source  \
12941         False        True  …           Wood  District Heating
5550           True        True  …           Wood  District Heating
4604          False        True  …           Wood           Unknown
69            False        True  …           Wood  District Heating
```

```
1457          False       False  …        unknown           Unknown

             heating_type  property_condition  year_built  schufa_check  \
12941  Underfloor Heating            Like New        2021          True
5550   Underfloor Heating            Like New        2021          True
4604         Unit Heating           Renovated        1907          True
69     Underfloor Heating            Like New        2021          True
1457              Unknown             Unknown        <NA>          True

               scraped_at                                location_clean  \
12941 2026-01-22 18:49:25  Abendseglersteig 50, Wilhelmshagen,  (12589)
5550  2026-01-11 17:09:51  Abendseglersteig 50, Wilhelmshagen,  (12589)
4604  2026-01-11 14:05:05                      Charlottenburg,  (10623)
69    2025-12-28 21:49:24  Abendseglersteig 50, Wilhelmshagen,  (12589)
1457  2025-12-31 17:20:53                                Buch,  (13125)

      zip_code        district
12941    12589       Rahnsdorf
5550     12589       Rahnsdorf
4604     10623  Charlottenburg
69       12589       Rahnsdorf
1457     13125            Buch

[5 rows x 27 columns]
```

[172]:
```
# These are all legitimate samples with legitimate valuse except area. Keeping␣
↪them would confuse our model so we delete them.
df = df[df["area"] >= 10]
```

Since we decided to keep the luxary listings we cannot remove anything from the higher tail.

3. Handling 'rooms' feature

[173]:
```
df['rooms'].unique()
```

[173]:
```
array([ 2. ,  1. ,  4. ,  5. ,  3. ,  1.5,  3.5,  8. ,  7. ,  2.5,  6. ,
        5.5,  4.5,  6.5, 10. ])
```

All of them are legitimate values.

4. Handling 'floor' feature

[174]:
```
df['floor'].unique()
```

[174]:
```
array([nan, 21., 20., 18., 17., 16., 15., 14., 13., 12., 11., 10.,  9.,
        8.,  7.,  6.,  5.,  4.,  3.,  2.,  1.,  0., -1.])
```

All of them are legitimate values.

5. Handling 'property_condition' feature

[175]:
```
df['property_condition'].value_counts()
```

[175]:
```
property_condition
Unknown                  9297
New / First Occupancy    1796
Like New                 1609
Well Maintained          1453
Renovated                1306
Old Building               56
Partially Renovated        44
Needs Renovation           38
Other                      35
Name: count, dtype: int64
```

Every category has it's own meaning, so removing any of them would cause information loss. Removing the 'Other' category would cause losing valueable samples so there is nothing we can adjust in this feature.

6. Handling 'year_built' feature

[176]:
```
df['year_built'].describe()
```

[176]:
```
count         7688.0
mean     1996.926119
std        43.860206
min           1765.0
25%           1987.5
50%           2020.0
75%           2024.0
max           2026.0
Name: year_built, dtype: Float64
```

All of them are legitimate values.

Now let's just remove the features that aren't needed and the dataset will be completely cleaned.

[177]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 15634 entries, 22653 to 3
Data columns (total 27 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   listing_url           15634 non-null  object
 1   cold_rent             15634 non-null  float64
 2   area                  15634 non-null  float64
 3   rooms                 15634 non-null  float64
 4   floor                 13686 non-null  float64
 5   free_from             15634 non-null  object
 6   location              15634 non-null  object
```

```
 7   has_balkon            15634 non-null  boolean
 8   has_terrasse          15634 non-null  boolean
 9   has_garten            15634 non-null  boolean
 10  elevator              15634 non-null  boolean
 11  parking               15634 non-null  boolean
 12  has_basement          15634 non-null  boolean
 13  is_barrier-free       15634 non-null  boolean
 14  has_built-in_kitchen  15634 non-null  boolean
 15  has_bathtub           15634 non-null  boolean
 16  has_shower            15634 non-null  boolean
 17  flooring_type         15634 non-null  object
 18  energy_source         15634 non-null  object
 19  heating_type          15634 non-null  object
 20  property_condition    15634 non-null  object
 21  year_built            7688 non-null   Int64
 22  schufa_check          15634 non-null  bool
 23  scraped_at            15634 non-null  datetime64[ns]
 24  location_clean        15634 non-null  object
 25  zip_code              15621 non-null  object
 26  district              15634 non-null  object
dtypes: Int64(1), bool(1), boolean(10), datetime64[ns](1), float64(4),
object(10)
memory usage: 2.4+ MB
```

[ ]:
```
df = df.drop(columns=["location","location_clean","scraped_at","zip_code"])
```

[182]:
```
df.to_csv("rent_listings.csv", encoding="utf-8-sig", index=False)
```